sherpa.

Автотесты

от первого теста до системы качества в команде

Практическое руководство

Содержание

01	Unit-тесты: назначение и границы ответственности	5
02	Integration-тесты: проверка взаимодействий	10
03	API-тесты: цели и сценарии применения	15
04	UI-тесты: возможности и ограничения	19
05	End-to-End тесты	24
06	Smoke, Regression, Sanity тесты	29
07	Пирамида тестирования и её практическое применение	34
08	Критерии выбора инструментов автотестирования	37
09	Языки программирования для автотестов	40
10	Фреймворки для UI-автотестов	43
11	Инструменты для API-тестирования	46
12	Инструменты для unit-тестов	49
13	Сравнение популярных стеков	52
14	Когда использовать low-code / no-code решения	55
15	Структура автотестового проекта	58
16	Test layers и separation of concerns	61
17	Page Object и альтернативные паттерны	64
18	Работа с тестовыми данными	67
19	Конфигурация окружений	70
20	Управление зависимостями	73
21	Локальное окружение для запуска автотестов	76
22	Тестовые стенды и их виды	79
23	Работа с тестовыми базами данных	82
24	Моки, стабы, фейки	85
25	Управление тестовыми пользователями	88
26	Создание проекта с нуля	91
27	Первый unit-тест	94
28	Первый API-тест	97
29	Первый UI-тест	100
30	Структура теста: arrange, act, assert	103
31	Чтение и интерпретация результатов	106
32	Архитектура REST и HTTP-основы	109
33	Проверка статусов и контрактов	112
34	Валидация схем и данных	115
35	Авторизация и работа с токенами	118
36	Негативные и пограничные сценарии	121
37	Связь API-тестов с UI- и unit-тестами	124
38	Как писать стабильные тесты	127
39	Борьба с flaky-тестами	130
40	Рефакторинг автотестов	133
41	Обновление тестов при изменении продукта	136
42	Управление техническим долгом в автотестах	139

43	Зачем запускать автотесты в пайплайне	142
44	Интеграция с CI-системами	145
45	Разделение тестов по стадиям	148
46	Параллельный запуск тестов	151
47	Отчёты и артефакты выполнения	154
48	Автотесты как критерий качества релиза	157
49	Виды отчётов по автотестам	160
50	Интерпретация падений	163
51	Отличие дефекта от нестабильного теста	166
52	Коммуникация результатов с командой	169
53	Метрики автотестирования	172
54	Взаимодействие QA, разработчиков и аналитиков	175
55	Ответственность за автотесты	178
56	Code review автотестов	181
57	Автотесты как часть культуры качества	184
58	Автотесты без ценности	187
59	Дублирование тестов	190
60	Хрупкие UI-тесты	193
61	Отсутствие стратегии	196
62	Игнорирование поддержки тестов	199
63	Рост автотестовой базы	202
64	Работа с большими проектами	205
65	Оптимизация времени прогона	208
66	Поддержка нескольких продуктов	211
67	Автотесты в микросервисной архитектуре	214

ПРИЛОЖЕНИЯ

А	Шаблоны документов	218
Б	Чек-листы по этапам	221
В	Глоссарий	223

КАК ЧИТАТЬ

- правило или ключевое число
- важно: типичная ошибка
- мировой кейс или пример из практики
- классика: идея, автор, год
- f формула: сначала общий вид, затем значения символов

ПРАКТИКА задание после главы

СКВОЗНОЙ КЕЙС проходит через все 67 глав

«Полка»

Сервис доставки продуктов из магазинов района: веб-сайт, приложения для iOS и Android, 14 микросервисов на Kotlin. Релиз выходит раз в две недели, потому что перед каждым четыре тестировщика три дня проходят ручной регресс, а в продакшен всё равно утекает полтора десятка дефектов в месяц. Нина, руководитель QA, получила задачу: за год перейти на ежедневные релизы и не потерять качество. Мы проходим этот путь от первых тестов до системы.

РУКОВОДИТЕЛЬ QA
Нина
хочет релизы каждый день без ночных откатов

SDET
Тимофей
хочет тесты, которым команда верит

ТЕХЛИД БЭКЕНДА
Марат
хочет, чтобы тесты не тормозили разработку

Бэкенд 14 сервисов, Kotlin **Веб** React + TypeScript **Мобильные** iOS, Android
Команда 38 инженеров, 6 QA

ВОСЕМЬ ВВОДНЫХ И РЕЗУЛЬТАТ

1 400 ручных регрессионных кейсов	**3 дня** ручной регресс перед релизом	**2 недели** цикл релиза
18 дефектов в продакшене в месяц	**14%** нестабильных UI-тестов	**95 мин** прогон 120 UI-тестов
60 000 ₽ средняя цена дефекта в проде	**1,08 млн ₽** потери от дефектов в месяц	**3 дня → 25 мин** цель: регресс в пайплайне, релиз каждый день

Компания вымышлена, но собрана из типичных ситуаций продуктовой разработки. Числа согласованы: время регресса, стоимость дефектов, вложения в автоматизацию и её эффект выводятся друг из друга.

01 Unit-тесты: назначение и границы ответственности

После неё вы отличаете unit-тест от остальных, решаете, что считать «юнитом» в своём коде, и пишете тесты, которые проверяют поведение, а не реализацию.

1.1 Что такое «юнит» 1.2 Границы ответственности 1.3 Кейс «Полки»: расчёт стоимости доставки 1.4 Формулировки: плохо и хорошо 1.5 Бенчмарки 1.6 Анти-паттерны

Unit-тест проверяет небольшой фрагмент кода в изоляции: функцию, класс, модуль. Он не ходит в сеть, не пишет в базу и не открывает браузер, поэтому выполняется за миллисекунды. Тысяча таких тестов проходит быстрее, чем один UI-тест логина. Именно скорость делает их основой всей системы автотестов: разработчик запускает их после каждого изменения, а не раз в день.

В «Полке» к началу истории было 900 unit-тестов на бэкенде, и покрывали они 31% строк. Марат честно признался: половину писали ради отчёта о покрытии, и они проверяли геттеры и конструкторы. Расчёт стоимости доставки, скидки, правила слотов — то, где на самом деле ломалось, — оставались без тестов.

1.1 Что такое «юнит»

Границу юнита каждая команда определяет сама. Есть две школы, и обе рабочие, если команда договорилась.

Классическая (детройтская)	Юнит — единица поведения. Тест может задействовать несколько классов, если они работают вместе и не трогают внешний мир. Подменяются только медленные и недетерминированные зависимости: сеть, база, часы.
Лондонская (mockist)	Юнит — один класс. Все его соседи заменяются моками, тест проверяет, какие вызовы класс сделал. Быстро локализует ошибку, но тесты сильнее привязаны к устройству кода.

КЛАССИКА Владимир Хориков, «Unit Testing: Principles, Practices, and Patterns», 2020

Хороший unit-тест

Хориков формулирует четыре свойства хорошего теста: защита от регрессий, устойчивость к рефакторингу, быстрая обратная связь и простота поддержки. Первые три конфликтуют, и максимизировать все сразу нельзя.

Главный вывод книги: устойчивость к рефакторингу не обсуждается. Тест, который краснеет при переименовании приватного метода, хуже отсутствия теста — он учит команду игнорировать красный цвет.

1.2 Границы ответственности

UNIT-ТЕСТ ПРОВЕРЯЕТ	UNIT-ТЕСТ НЕ ПРОВЕРЯЕТ
Бизнес-правила: расчёт, валидацию, ветвления	Что SQL-запрос корректен для настоящей базы
Граничные значения и обработку ошибок	Что сервисы договорились о формате JSON
Преобразования данных	Что кнопка видна и нажимается
Алгоритмы и состояния (конечные автоматы)	Что конфигурация окружения верна

Ошибки из правой колонки ловят интеграционные, контрактные и UI-тесты. Попытка проверить их юнитами с моками даёт ложное спокойствие: мок базы всегда отвечает так, как вы его научили, и никогда — как настоящая база с её типами, индексами и транзакциями.

1.3 Кейс «Полки»: расчёт стоимости доставки

Первым модулем для настоящих unit-тестов Марат выбрал DeliveryPriceCalculator: за

квартал из него пришли 4 дефекта в продакшен. Цена зависит от суммы корзины, расстояния, времени слота и подписки. До тестов правила жили в голове аналитика Алисы и в 180 строках вложенных if.

КАК КОМАНДА ПОКРЫЛА КАЛЬКУЛЯТОР

1 **Таблица правил**
Алиса выписала все правила в таблицу: 11 условий, 23 комбинации, которые реально встречаются.

2 **Тест на каждую строку**
Параметризованный тест JUnit 5: одна строка таблицы — один кейс, 23 кейса за вечер.

3 **Границы**
Отдельно: корзина ровно 1 500 ₽ (порог бесплатной доставки), 1 499,99 ₽, расстояние 0 км, слот в полночь.

4 **Рефакторинг под защитой**
180 строк if переписали в таблицу правил; все 31 тест остались зелёными.

✓ **Итог**
0 дефектов калькулятора за следующие два квартала; прогон 31 теста — 0,4 с.

```
@ParameterizedTest
@CsvSource(
 "1499.99, 3.0, DAY,   false, 199",
 "1500.00, 3.0, DAY,   false,   0",
 "1500.00, 8.5, DAY,   false, 149",
 " 900.00, 3.0, NIGHT, true,    0"
)
fun `price by rules`(cart: BigDecimal, km: Double, slot: Slot, sub: Boolean, exp:
Int) {
 val price = calculator.price(Order(cart, km, slot, sub))
 assertEquals(exp.toBigDecimal(), price)
}
```

ПРАВИЛО

Тестируйте через публичный контракт модуля: вход → выход или вход → наблюдаемое изменение состояния. Если для теста нужно открыть приватный метод, это сигнал, что внутри прячется отдельный модуль со своим контрактом.

1.4 Формулировки: плохо и хорошо

ПЛОХО

Тест реализации

verify(repo, times(1)).findById(42)

verify(mapper).toDto(any())

Проверяет, как код устроен; ломается при любом рефакторинге.

ХОРОШО

Тест поведения

assertEquals(0, price(cart = 1500))

assertThrows<SlotUnavailable> { book(pastSlot) }

Проверяет, что получает пользователь кода.

1.5 Бенчмарки

МЕТРИКА	ОРИЕНТИР	«ПОЛКА» ДО	«ПОЛКА» ЧЕРЕЗ ГОД
Время одного unit-теста	до 50 мс	120 мс	8 мс
Прогон всех unit-тестов сервиса	до 1–2 мин	6 мин	40 с
Покрытие строк в доменной логике	70–90%	31%	84%
Доля unit среди всех автотестов	60–80%	88%	71%

Покрытие доменной логики выросло, а доля unit-тестов в общей массе упала. Это не противоречие: команда удалила около 400 тестов на геттеры и DTO и параллельно добавила интеграционные и API-тесты, которых раньше не было.

1.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Тесты на геттеры и DTO	Растят покрытие, не ловят ни одной ошибки	Тестировать логику, а не структуру данных

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Мок на каждый класс	Тест повторяет код; рефакторинг ломает десятки тестов	Мокать только внешний мир
Unit-тест с базой «на минутку»	Медленно, нестабильно, зависит от окружения	Вынести в интеграционные
Один тест на десять проверок	Падение не говорит, что сломалось	Один сценарий — один тест

ПРАКТИКА 1 после главы 01

Найдите в своём проекте модуль, из которого чаще всего приходят дефекты, и покройте его по таблице правил.

ЧТО СДЕЛАТЬ

01	Выпишите правила модуля в таблицу: условия, комбинации, ожидаемый результат.	☐
02	Напишите параметризованный тест: одна строка таблицы — один кейс.	☐
03	Добавьте граничные значения: порог, порог ± минимальный шаг, ноль, пусто.	☐
04	Проверьте, что ни один тест не обращается к приватным методам и не проверяет вызовы через verify.	☐
05	Замерьте время прогона: если тест дольше 50 мс, найдите, что он трогает снаружи.	☐

РЕЗУЛЬТАТ Хороший результат — тесты переживают рефакторинг модуля без единой правки.

02 Integration-тесты: проверка взаимодействий

После неё вы проверяете стыки кода с базой, очередями и соседними сервисами на настоящих зависимостях и знаете, где граница между интеграционным тестом и E2E.

2.1 Что считается интеграцией · 2.2 Настоящие зависимости в контейнерах · 2.3 Кейс «Полки»: двойное бронирование слота · 2.4 Как ускорить интеграционные тесты · 2.5 Формулировки: плохо и хорошо · 2.6 Бенчмарки · 2.7 Анти-паттерны

Большинство дефектов живёт не внутри функций, а на стыках: запрос написан под одну версию базы, а в продакшене другая; сериализатор отдаёт дату строкой, а потребитель ждёт число; транзакция закрывается раньше, чем отправлено событие. Unit-тест с моками этого не увидит, потому что мок ведёт себя так, как его научили. Интеграционный тест подключает настоящую зависимость и проверяет, что код с ней действительно работает.

2.1 Что считается интеграцией

Код ↔ инфраструктура	Репозиторий и PostgreSQL, продюсер и Kafka, кэш и Redis, клиент и S3. Самый частый и самый полезный вид.
Модули внутри сервиса	HTTP-слой, сервисы и репозитории вместе: запрос на эндпоинт проходит весь стек сервиса до настоящей базы.
Сервис ↔ внешний API	Клиент платёжного шлюза против эмулятора (WireMock) или sandbox-окружения провайдера.

Граница с E2E простая: интеграционный тест проверяет один сервис и его прямые зависимости. Если в тесте участвуют несколько ваших сервисов, развёрнутых вместе, это уже системный или E2E-тест, и стоит он на порядок дороже.

КЛАССИКА Мартин Фаулер, «IntegrationTest», 2018

Узкие и широкие интеграционные тесты

Фаулер различает узкие интеграционные тесты — проверку кода, работающего с одной внешней системой, с подменой остальных, — и широкие, где поднимаются все сервисы. Термин «интеграционный тест» люди используют для обоих, и из-за этого спорят, не понимая друг друга.

Узких тестов должно быть много, широких — мало. Широкие медленны, нестабильны и плохо локализуют ошибку.

2.2 Настоящие зависимости в контейнерах

Ещё десять лет назад интеграционные тесты гоняли на общей тестовой базе или на H2 в памяти. Первое делает тесты зависимыми друг от друга, второе — от отличий диалекта. Сегодня стандарт — Testcontainers: тест сам поднимает PostgreSQL, Kafka или Redis нужной версии в Docker, работает с ними и выбрасывает после прогона.

```
@Testcontainers
class SlotRepositoryIT {
 companion object {
  @Container val pg = PostgreSQLContainer("postgres:16.4")
 }
 @Test fun `two orders cannot book the last slot`() {
  repo.createSlot(store = 7, capacity = 1)
  val results = parallel(2) { repo.book(store = 7, order = it) }
  assertEquals(1, results.count { it.isSuccess })
 }
}
```

2.3 Кейс «Полки»: двойное бронирование слота

В марте два клиента получили подтверждение на один и тот же последний слот доставки, и курьер приехал с двумя заказами при вместимости в один. Unit-тест сервиса бронирования был зелёным: мок репозитория честно возвращал «слот свободен» на каждый вызов. Ошибка жила в SQL: проверка и запись шли без блокировки строки.

КАК ТИМОФЕЙ ЗАКРЫЛ ДЫРУ

1 **Воспроизвести**
Интеграционный тест на настоящем PostgreSQL 16 с двумя параллельными бронированиями — падает в 7 прогонах из 10.

2 **Исправить**
SELECT ... FOR UPDATE и уникальный индекс на (slot_id, seq); тест зелёный 200 раз подряд.

3 **Расширить**
Такие же тесты на отмену и перенос слота: ещё одна гонка найдена до продакшена.

✓ **Итог**
38 интеграционных тестов на сервис слотов, прогон 54 с с учётом старта контейнера.

ВАЖНО · ТИПИЧНАЯ ОШИБКА

Подменять базу H2 или SQLite «для скорости». Разница в типах, блокировках и функциях маскирует именно те ошибки, ради которых пишут интеграционный тест. Контейнер PostgreSQL стартует за 2–4 секунды и переиспользуется всем классом тестов.

2.4 Как ускорить интеграционные тесты

- Один контейнер на весь прогон, а не на каждый тест; изоляция данных — через транзакцию с откатом или очистку таблиц.
- Миграции применяются один раз при старте, а не перед каждым тестом.
- Образы зависимостей кэшируются на CI-раннерах, чтобы не тянуть их из сети при каждом запуске.
- Тяжёлые сценарии (Kafka с несколькими брокерами) выносятся в отдельную стадию пайплайна.

2.5 Формулировки: плохо и хорошо

ПЛОХО

Общая тестовая база

Все тесты пишут в одну базу на стенде.

Тест зелёный, если перед ним никто не удалил нужную запись.

Параллельный запуск невозможен.

ХОРОШО

Своя база на прогон

Контейнер нужной версии поднимается тестом.

Каждый тест сам создаёт свои данные и не видит чужих.

Параллельно идут 8 потоков без конфликтов.

2.6 Бенчмарки

МЕТРИКА	ОРИЕНТИР	«ПОЛКА» ЧЕРЕЗ ГОД
Время одного интеграционного теста	0,1–2 с	0,6 с
Старт контейнера PostgreSQL	2–5 с	3 с
Интеграционных тестов на сервис	20–60	30 в среднем
Доля среди всех автотестов	10–25%	9%

2.7 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Тесты зависят от порядка	Падают при параллельном или выборочном запуске	Каждый тест создаёт свои данные
Поднимать все сервисы ради одного стыка	Медленно и хрупко	Соседей подменять WireMock
Проверять только happy path	Гонки и таймауты не ловятся	Параллельные вызовы, обрывы, повторы
База в памяти вместо настоящей	Ошибки диалекта уходят в прод	Testcontainers той же версии

ПРАКТИКА 2 после главы 02

Найдите самый рискованный стык своего сервиса с инфраструктурой и проверьте его на настоящей зависимости.

ЧТО СДЕЛАТЬ

01	Выпишите внешние зависимости сервиса: базы, очереди, кэши, внешние API.	☐
02	Для каждой вспомните последний инцидент: был бы он пойман unit-тестом?	☐
03	Напишите интеграционный тест с Testcontainers для самого рискованного стыка.	☐
04	Добавьте сценарий конкуренции или сбоя: два параллельных запроса, обрыв соединения, таймаут.	☐
05	Замерьте время прогона и настройте переиспользование контейнера.	☐

РЕЗУЛЬТАТ Цель — один тест, который упал бы на вашем последнем инциденте.

03 API-тесты: цели и сценарии применения

После неё вы понимаете, почему API-тесты — лучший баланс скорости и уверенности, и выбираете для них сценарии: контракт, бизнес-цепочки, права, негативные случаи.

3.1 Зачем нужны API-тесты · 3.2 Где API-тесты сильнее UI · 3.3 Кейс «Полки»: перенос логики из UI в API · 3.4 Сценарии, которые часто забывают · 3.5 Формулировки: плохо и хорошо · 3.6 Бенчмарки · 3.7 Анти-паттерны

API-тест обращается к работающему сервису по сети так же, как это сделает фронтенд, мобильное приложение или другой сервис: отправляет HTTP-запрос и проверяет ответ. Он не видит внутренностей кода и не зависит от вёрстки. Поэтому API-тесты дают почти ту же уверенность, что UI-тесты, но выполняются в десятки раз быстрее и почти не страдают от нестабильности.

В «Полке» API-тестов не было совсем. Всё, что не покрывали юниты, проверяли через 120 UI-тестов на Selenium. Тимофей посчитал: 83 из них на самом деле проверяли бизнес-логику бэкенда — расчёт корзины, промокоды, статусы заказа — и проходили через браузер только потому, что другого способа не было.

3.1 Зачем нужны API-тесты

ЦЕЛЬ	ЧТО ПРОВЕРЯЕМ	ПРИМЕР В «ПОЛКЕ»
Контракт	Статусы, поля, типы, обязательность	POST /orders возвращает 201 и id в формате UUID
Бизнес-цепочки	Последовательность вызовов как у пользователя	Корзина → слот → оплата → статус «собирается»
Права доступа	Кто может, а кто нет	Курьер не видит заказы чужого магазина
Негативные случаи	Ошибки ввода и состояния	Промокод истёк → 422 с кодом PROMO_EXPIRED

ЦЕЛЬ	ЧТО ПРОВЕРЯЕМ	ПРИМЕР В «ПОЛКЕ»
Нефункциональное	Время ответа, лимиты, идемпотентность	Повтор оплаты с тем же ключом не списывает дважды

3.2 Где API-тесты сильнее UI

Через интерфейс сложно проверить то, что пользователь не видит, но что ломает продукт: права, обработку ошибок, граничные значения, идемпотентность. Чтобы проверить через UI, что курьер не видит чужой заказ, нужно найти способ туда «попасть». Через API это один запрос с токеном курьера и ожидание 403.

```
test('courier cannot read order of another store', async ({ api }) => {
 const order = await api.as('customer').createOrder({ store: 7 });
 const res = await api.as('courier', { store: 12 }).get(`/orders/${order.id}`);
 expect(res.status()).toBe(403);
 expect(await res.json()).toMatchObject({ code: 'FORBIDDEN_STORE' });
});
```

3.3 Кейс «Полки»: перенос логики из UI в API

КАК ТИМОФЕЙ РАЗОБРАЛ 120 UI-ТЕСТОВ

1. **Классификация**
   Каждый UI-тест получил метку: проверяет интерфейс, бизнес-логику или и то и другое.

2. **Перенос логики**
   83 сценария бизнес-логики переписаны как 214 API-тестов: разбили по вариантам данных.

3. **Сокращение UI**
   Из UI-набора удалено 64 теста, 19 упрощены до проверки отображения.

✓ **Итог**
214 API-тестов идут 3 мин 40 с против 61 мин у удалённых UI; flaky — 0,3%.

ПРАВИЛО

Если сценарий можно проверить без браузера — проверяйте без браузера. UI-тест оставляйте для того, что видит пользователь: отображение, навигация, ключевой путь целиком.

3.4 Сценарии, которые часто забывают

- Пустые и огромные значения: пустая строка, 0, отрицательное число, строка в 10 000 символов.
- Повторный запрос с тем же ключом идемпотентности.
- Изменение сущности, которую параллельно изменил другой пользователь.
- Запрос с просроченным и с отозванным токеном.
- Пагинация на границах: первая, последняя, за пределами.

3.5 Формулировки: плохо и хорошо

ПЛОХО

Проверка «что-то пришло»

expect(res.status()).toBe(200)

Тест зелёный, даже если тело ответа пустое или с ошибкой внутри.

ХОРОШО

Проверка смысла

Статус, схема ответа и ключевые поля.

expect(body.total).toBe(1690)

expect(body.status).toBe('ASSEMBLING')

3.6 Бенчмарки

МЕТРИКА	ОРИЕНТИР	«ПОЛКА» ЧЕРЕЗ ГОД
Время одного API-теста	50–500 мс	180 мс
Нестабильность	до 1%	0,3%
Доля среди всех автотестов	10–20%	14%
Покрытие эндпоинтов хотя бы одним тестом	90–100% публичных	96%

3.7 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Только статус 200	Пропускает ошибки в теле	Проверять схему и ключевые поля
Данные из прошлого теста	Цепочка падает целиком	Каждый тест готовит своё
Только позитивные сценарии	Права и ошибки не проверены	На каждый эндпоинт — негативные случаи
Дублировать unit-тесты через API	Медленнее без новой уверенности	API — для стыков и цепочек

ПРАКТИКА 3 после главы 03

Найдите UI-тесты, которые на самом деле проверяют логику бэкенда, и перенесите хотя бы три на уровень API.

ЧТО СДЕЛАТЬ

01	Пометьте каждый UI-тест: интерфейс, логика или оба.	☐
02	Для трёх тестов с меткой «логика» опишите эквивалентную цепочку API-запросов.	☐
03	Напишите API-тесты с проверкой статуса, схемы и ключевых полей.	☐
04	Добавьте к каждому по негативному сценарию: права, неверные данные, повтор.	☐
05	Сравните время прогона старых UI и новых API-тестов.	☐

РЕЗУЛЬТАТ Цифры сравнения — хороший аргумент для команды, почему пирамиду стоит перестраивать.

04 UI-тесты: возможности и ограничения

После неё вы понимаете, что UI-тест проверяет лучше остальных, почему он дороже всех в поддержке, и выбираете для интерфейса узкий набор сценариев.

4.1 Что UI-тест умеет лучше всех
4.2 Почему UI-тесты дорогие
4.3 Кейс «Полки»: 95 минут и 14% нестабильности
4.4 Формулировки: плохо и хорошо
4.5 Бенчмарки
4.6 Анти-паттерны

UI-тест управляет настоящим браузером или мобильным приложением: открывает страницу, кликает, вводит текст и проверяет, что увидел пользователь. Это единственный вид автотестов, который подтверждает, что продукт работает так, как его воспринимает человек: кнопка видна, форма отправляется, ошибка показана рядом с полем. Но за это приходится платить скоростью, стабильностью и стоимостью поддержки.

4.1 Что UI-тест умеет лучше всех

Сборка фронтенда целиком	Код, стили, бандл, роутинг и конфигурация вместе — как у пользователя.
Пользовательские пути	Регистрация, оформление заказа, оплата: ключевые сценарии от первого клика до результата.
Отображение и состояния	Ошибки валидации, пустые списки, загрузка, блокировка кнопки при отправке.
Совместимость	Разные браузеры, размеры экрана, локали.

4.2 Почему UI-тесты дорогие

ПРОБЛЕМА	ОТКУДА БЕРЁТСЯ	КАК СМЯГЧИТЬ
Медленно	Браузер, сеть, рендер, анимации	Готовить данные через API, в UI проверять только суть
Нестабильно	Асинхронность, гонки, анимации, сторонние скрипты	Автоожидания, стабильные локаторы, изоляция данных
Хрупко	Любая правка вёрстки ломает селекторы	data-testid и роли вместо CSS-путей
Плохая диагностика	Падение «элемент не найден» ничего не объясняет	Трейс, скриншот, видео, логи сети

КЛАССИКА Алистер Скотт, «Testing Pyramids & Ice-Cream Cones», 2012

Мороженое вместо пирамиды

Скотт нарисовал антипаттерн рожка мороженого: много ручных проверок сверху, толстый слой UI-тестов, тонкий слой интеграционных и почти нет unit-тестов. Такая система медленная и дорогая, и в ней каждый релиз зависит от долгого прогона.

«Полка» в начале истории выглядела именно так: три дня ручного регресса и 120 UI-тестов поверх 900 юнитов, которые почти ничего не проверяли.

4.3 Кейс «Полки»: 95 минут и 14% нестабильности

Старый набор на Selenium писали три года разные люди. Тимофей разобрал последние 60 прогонов и выяснил, откуда берутся случайные падения: тесты ждали элементы через Thread.sleep, искали их по XPath из 8–12 уровней и логинились через форму в каждом тесте.

ПРИЧИНА ПАДЕНИЙ	ДОЛЯ	ИСПРАВЛЕНИЕ
Фиксированные паузы и гонки	41%	Автоожидания Playwright, ожидание ответа API
Хрупкие локаторы	27%	data-testid и getByRole
Общие тестовые данные	18%	Свой пользователь и заказ на каждый тест
Нестабильный стенд	14%	Ретрай инфраструктурных ошибок, мониторинг стенда

ЧТО СДЕЛАЛИ С UI-НАБОРОМ

1 **Сократить**
64 теста бизнес-логики ушли в API (глава 3); осталось 56.

2 **Переписать**
Selenium → Playwright + TypeScript: автоожидания, трейсы, параллельность из коробки.

3 **Логин через API**
Состояние авторизации сохраняется один раз и подгружается в тесты: -35 с на тест.

✓ **Итог**
56 тестов, 4 параллельных воркера, 11 мин вместо 95; flaky 14% → 1,8%.

ПРАВИЛО

В UI-тесте готовьте всё, кроме проверяемого шага, через API: пользователя, корзину, адрес. Через интерфейс проходите только то, что действительно проверяете.

4.4 Формулировки: плохо и хорошо

ПЛОХО

Привязка к вёрстке

page.locator('#root > div > div:nth-child(3) > button')

await page.waitForTimeout(3000)

Ломается при переносе кнопки и тормозит на быстром стенде.

ХОРОШО

Привязка к смыслу

page.getByRole('button', { name: 'Оформить заказ' })

await expect(page.getByTestId('order-status')).toHaveText('Собирается')

Ждёт ровно столько, сколько нужно.

4.5 Бенчмарки

МЕТРИКА	ОРИЕНТИР	«ПОЛКА» ДО	ЧЕРЕЗ ГОД
Время одного UI-теста	10–60 с	48 с	14 с
Нестабильность	до 2%	14%	0,9%

МЕТРИКА	ОРИЕНТИР	«ПОЛКА» ДО	ЧЕРЕЗ ГОД
Доля среди автотестов	5–10%	12%	2%
Полный UI-прогон	до 15 мин	95 мин	11 мин

4.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Sleep вместо ожиданий	Медленно и всё равно нестабильно	Ожидать состояние элемента или ответ сети
Логин через форму в каждом тесте	+30–40 с на тест, лишняя точка отказа	Сохранённая сессия, логин через API
UI для проверки расчётов	Медленная проверка логики бэкенда	Перенести в API или unit
Один огромный сценарий	Падение в начале скрывает остальное	Короткие независимые тесты

ПРАКТИКА 4 после главы 04

Разберите причины нестабильности своего UI-набора по последним прогонам.

ЧТО СДЕЛАТЬ

01	Соберите результаты последних 30–60 прогонов и выпишите все случайные падения.	☐
02	Разложите их по причинам: ожидания, локаторы, данные, стенд, сам продукт.	☐
03	Найдите все фиксированные паузы и замените их ожиданием состояния.	☐
04	Переведите логин и подготовку данных на API.	☐
05	Посчитайте, сколько UI-тестов проверяют логику, которую можно проверить через API.	☐

РЕЗУЛЬТАТ Обычно 60–80% нестабильности дают две первые причины, и их можно убрать за пару недель.

05 End-to-End тесты

После неё вы отличаете E2E от UI-теста, отбираете для него 10–20 критичных путей и строите сквозные проверки так, чтобы они не превратились в главный источник ложных падений.

5.1 Почему E2E должно быть мало
5.2 Как отбирать сценарии
5.3 Кейс «Полки»: 12 сквозных путей
5.4 Синтетический мониторинг
5.5 Формулировки: плохо и хорошо
5.6 Бенчмарки
5.7 Анти-паттерны

End-to-End тест проходит бизнес-процесс от начала до конца через все системы, которые в нём участвуют. В «Полке» заказ — это фронтенд, API-шлюз, сервис корзины, сервис слотов, платёжный сервис, сборка в магазине, назначение курьера и уведомления. E2E-тест создаёт заказ и доводит его до статуса «доставлен», проверяя, что все звенья договорились друг с другом.

Часто E2E путают с UI-тестом. UI-тест может работать с подменённым бэкендом, а E2E может вообще не трогать браузер: сценарий из цепочки API-вызовов через несколько сервисов — тоже сквозной. Признак E2E — не интерфейс, а настоящая сборка всех участников процесса.

5.1 Почему E2E должно быть мало

ФОРМУЛА 5.1 · ВЕРОЯТНОСТЬ ЛОЖНОГО ПАДЕНИЯ СКВОЗНОГО ТЕСТА

$$P_{\text{flaky}} = 1 - (1 - p)^n \tag{5.1}$$

где

p — вероятность случайного сбоя одного звена — доля

n — сколько звеньев проходит тест — шт.

P_{flaky} — вероятность, что тест упадёт не из-за дефекта — доля

```
Заказ в «Полке»: n = 8 сервисов, p = 0,5% → P = 1 - 0,995^{8} ≈ 3,9%. Набор из 50 таких тестов почти всегда будет красным.
```

Формула объясняет, почему стабильность E2E нельзя «докрутить» до нуля: каждое звено вносит свою долю. Поэтому сквозные тесты оставляют для нескольких критичных путей, а всё остальное проверяют ниже — контрактами и API-тестами отдельных сервисов.

5.2 Как отбирать сценарии

КРИТЕРИЙ	ВОПРОС	ПРИМЕР
Деньги	Ломает ли сбой выручку прямо сейчас?	Оплата заказа картой
Частота	Сколько пользователей проходит путь в день?	Поиск товара и добавление в корзину
Сложность стыков	Сколько сервисов участвует?	Заказ → сборка → курьер → доставлен
История	Были ли инциденты на этом пути?	Возврат денег при отмене

5.3 Кейс «Полки»: 12 сквозных путей

КАК НИНА СОБРАЛА E2E-НАБОР

1 **Карта путей**
С продактами выписали 31 пользовательский путь и оценили по четырём критериям.

КАК НИНА СОБРАЛА E2E-НАБОР

2 **Отбор**
В E2E попали 12 путей: заказ, оплата, отмена с возвратом, замена товара, промокод, подписка и ещё шесть.

3 **Остальное — ниже**
19 путей покрыли API-тестами сервисов и контрактами между ними.

4 **Стенд**
E2E идут на отдельном стенде после деплоя; платёжный шлюз — в режиме sandbox.

✓ **Итог**
12 тестов, 7 минут, ложные падения 2,1% на тест; запуск после каждого деплоя на стенд.

ВАЖНО · ТИПИЧНАЯ ОШИБКА

Отвечать на каждый инцидент в продакшене новым E2E-тестом. Через год набор вырастает до сотен тестов, прогон занимает час, и красный цвет становится нормой. Правильный вопрос после инцидента: на каком самом низком уровне эту ошибку можно было поймать.

5.4 Синтетический мониторинг

Часть сквозных тестов полезно гонять и в продакшене, по расписанию, от имени служебного пользователя. «Полка» каждые 5 минут создаёт тестовый заказ в служебном магазине и отменяет его. Если сценарий падает дважды подряд, дежурный получает сигнал раньше, чем первый клиент напишет в поддержку.

5.5 Формулировки: плохо и хорошо

ПЛОХО

E2E на всё

«Добавим E2E на каждую фичу».

180 сквозных тестов, прогон 70 минут.

Каждый второй прогон — красный без дефекта.

ХОРОШО

E2E на критичное

«E2E — только для 12 путей, которые приносят деньги».

Прогон 7 минут, падение — повод остановить релиз.

Остальное — контракты и API.

5.6 Бенчмарки

МЕТРИКА	ОРИЕНТИР	«ПОЛКА»
Количество E2E-тестов	5–30 на продукт	12 (+ мобильные 8)
Время прогона	до 10–15 мин	7 мин
Ложные падения на тест	до 2–3%	2,1%
Частота синтетики в продакшене	1–5 мин	5 мин

5.7 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Новый E2E на каждый баг	Медленный и нестабильный набор	Ловить ошибку на нижнем уровне
Длинная цепочка в одном тесте	Падение в начале скрывает остальные шаги	Разбить путь на этапы с подготовкой через API
Общий стенд с ручным тестированием	Чужие данные ломают тесты	Отдельный стенд или изоляция по магазину
Внешние сервисы вживую	Падения из-за чужих сбоев	Sandbox или эмулятор провайдера

ПРАКТИКА 5 после главы 05

Соберите короткий список сквозных путей, которые стоит проверять E2E-тестами.

ЧТО СДЕЛАТЬ

01	Выпишите все пользовательские пути продукта.	☐
02	Оцените каждый по деньгам, частоте, сложности стыков и истории инцидентов.	☐
03	Оставьте в E2E не больше 10–20 путей с наивысшей оценкой.	☐
04	Для остальных определите, каким уровнем их покрыть: API, контракт, unit.	☐
05	Посчитайте по формуле (5.1) ожидаемую долю ложных падений вашего набора.	☐

РЕЗУЛЬТАТ Если расчёт даёт больше 3% на тест, сокращайте число звеньев или переносите проверки ниже.

06 Smoke, Regression, Sanity тесты

После неё вы различаете наборы тестов по цели, а не по уровню, и собираете smoke, sanity и регресс так, чтобы каждый отвечал на свой вопрос за своё время.

6.1 Три набора и три вопроса
6.2 Smoke: дым из коробки
6.3 Sanity: узкая проверка исправления
6.4 Regression: защита от возврата ошибок
6.5 Кейс «Полки»: три дня ручного регресса
6.6 Формулировки: плохо и хорошо
6.7 Бенчмарки
6.8 Анти-паттерны

Unit, API, UI и E2E — это уровни: что именно тест трогает. Smoke, sanity и regression — это наборы: зачем мы запускаем тесты в данный момент. Один и тот же API-тест может входить и в smoke, и в регресс. Путаница между уровнями и наборами — частая причина споров, поэтому договоритесь о терминах до того, как начнёте собирать пайплайн.

6.1 Три набора и три вопроса

НАБОР	ВОПРОС	КОГДА	ВРЕМЯ
Smoke	Сборка вообще живая? Есть смысл тестировать дальше?	Сразу после деплоя	1–5 мин
Sanity	Конкретное исправление работает и не сломало соседнее?	После хотфикса или точечной правки	5–15 мин
Regression	Всё, что работало раньше, работает и сейчас?	Перед релизом, ночью, на merge	15–60 мин

6.2 Smoke: дым из коробки

Название пришло из электроники: включили плату — если пошёл дым, дальше не проверяем. Smoke-набор отвечает на один вопрос: стоит ли тратить время на остальные тесты. Он короткий, широкий и неглубокий: сервисы отвечают, главная открывается, логин проходит, товар добавляется в корзину.

```
smoke (после деплоя на стенд, 2 мин)
 GET /health у 14 сервисов → 200
 логин тестового пользователя через API → токен
 поиск «молоко» в магазине 7 → не пусто
 добавить в корзину → total > 0
 UI: главная открывается, корзина показывает товар
```

6.3 Sanity: узкая проверка исправления

Sanity запускают, когда поменялось что-то конкретное и нужно быстро убедиться, что изменение работает и не задело ближайшее окружение. Его не собирают заранее целиком: берут тесты затронутого модуля и соседних. Хорошо, когда тесты размечены тегами по функциональным областям, — тогда sanity собирается фильтром за секунды.

6.4 Regression: защита от возврата ошибок

Регресс проверяет, что новое не сломало старое. Это самый большой набор, и именно его автоматизация даёт главный экономический эффект. Важно, что регресс не равен «все тесты подряд»: его состав пересматривают, удаляя устаревшие и дублирующиеся проверки.

6.5 Кейс «Полки»: три дня ручного регресса

Ручной регресс «Полки» содержал 1 400 кейсов. Четыре тестировщика проходили их три дня перед каждым из 26 релизов в год. Нина посчитала стоимость.

ФОРМУЛА 6.1 · СТОИМОСТЬ РУЧНОГО РЕГРЕССА В ГОД

$$C = q \cdot d \cdot h \cdot r \cdot w \tag{6.1}$$

где

- q — тестировщиков на регрессе — чел.
- d — дней регресса — дн.
- h — часов в дне — ч
- r — релизов в год — шт.
- w — стоимость часа с налогами — ₽

`C = 4 · 3 · 8 · 26 · 1 200 ≈ 3,0 млн ₽ в год, и это без учёта того, что релиз ждёт три дня`

КАК РАЗОБРАЛИ 1 400 КЕЙСОВ

1. **Чистка**
   312 кейсов устарели или дублировали друг друга — удалены.
2. **Приоритет**
   Остальные 1 088 размечены: критичный, важный, второстепенный.
3. **Автоматизация**
   За 9 месяцев автоматизировано 870 кейсов на разных уровнях, больше всего — API.
4. **Остаток**
   218 кейсов остались ручными: вёрстка, новые фичи, исследовательские сессии.

✓ **Итог**
Автоматический регресс 25 мин на merge; ручной — 4 часа исследовательского тестирования.

6.6 Формулировки: плохо и хорошо

ПЛОХО

Наборы без цели

«Smoke — это наши самые важные тесты».

Smoke растёт до 300 тестов и 40 минут.

Никто не помнит, почему тест в наборе.

ХОРОШО

Наборы по вопросу

«Smoke отвечает за 2 минуты: живая ли сборка».

Добавить тест в smoke можно, только убрав другой.

Наборы собираются тегами, а не списками.

6.7 Бенчмарки

НАБОР	ОРИЕНТИР ПО ВРЕМЕНИ	«ПОЛКА»
Smoke	1–5 мин	2 мин, 22 проверки
Sanity	5–15 мин	по тегу модуля, 3–8 мин
Regression на merge	15–30 мин	25 мин
Полный ночной	до 90 мин	70 мин

6.8 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Smoke разрастается	Теряет смысл быстрого сигнала	Жёсткий лимит по времени
Регресс только растёт	Время прогона увеличивается каждый месяц	Квартальная чистка набора
Списки тестов в CI-файле	Новые тесты не попадают в наборы	Теги: @smoke, @regress, @cart
Путать уровни и наборы	Споры, дубли, дыры в покрытии	Глоссарий команды

ПРАКТИКА 6 после главы 06

Разметьте свои тесты тегами наборов и посчитайте стоимость ручного регресса.

ЧТО СДЕЛАТЬ

01	Договоритесь в команде об определениях smoke, sanity и regression.	☐
02	Соберите smoke-набор, который отвечает за 2–5 минут, жива ли сборка.	☐
03	Добавьте теги функциональных областей, чтобы sanity собирался фильтром.	☐
04	Посчитайте по формуле (6.1) стоимость вашего ручного регресса в год.	☐
05	Выберите 10% ручных кейсов, которые стоит автоматизировать первыми.	☐

РЕЗУЛЬТАТ Расчёт стоимости регресса — главный аргумент, когда вы будете просить ресурсы на автоматизацию.

07 Пирамида тестирования и её практическое применение

После неё вы строите свою пропорцию уровней тестов от рисков продукта, а не от картинки, и знаете, когда пирамида превращается в ромб или трофей.

7.1 Формы и когда они уместны
7.2 Кейс «Полки»: пропорция до и после
7.3 Анти-паттерны

Пирамида говорит простую вещь: чем ниже уровень, тем тестов больше, они быстрее и дешевле; чем выше — тем их меньше. Внизу тысячи unit-тестов, в середине сотни интеграционных и API, наверху десятки UI и E2E. Пропорция не догма, а следствие экономики: каждый уровень выше стоит на порядок дороже в написании, прогоне и поддержке.

КЛАССИКА — Майк Кон, «Succeeding with Agile», 2009

Пирамида автоматизации

Кон нарисовал три слоя: unit внизу, сервисный слой посередине, UI наверху. Его главный тезис — большинство проверок должно идти мимо интерфейса, потому что UI-тесты медленные, хрупкие и дорогие.

Позже появились варианты: «трофей тестирования» Кента Доддса (2018) с упором на интеграционные тесты для фронтенда и «соты» Spotify для микросервисов, где основу составляют интеграционные тесты сервиса.

7.1 Формы и когда они уместны

ФОРМА	ОСНОВА	КОГДА УМЕСТНА
Пирамида	Unit	Много доменной логики: расчёты, правила, алгоритмы
Трофей	Интеграционные компонентов	Фронтенд, где логика живёт в связке компонентов
Соты	Интеграционные сервиса	Микросервисы с тонкой логикой и толстыми стыками

ФОРМА	ОСНОВА	КОГДА УМЕСТНА
Рожок мороженого	UI и ручные	Никогда — это анти-паттерн

7.2 Кейс «Полки»: пропорция до и после

УРОВЕНЬ	ДО	ЧЕРЕЗ ГОД	ДОЛЯ ЧЕРЕЗ ГОД
Unit (бэкенд + фронт)	900	3 200	71%
Интеграционные	0	420	9%
API	0	640	14%
Контрактные	0	180	4%
UI и E2E	120	90	2%
Итого	**1 020**	**4 530**	**100%**

Форма получилась пирамидой с толстой серединой: у «Полки» много логики в расчётах корзины и слотов (это unit), но ещё больше рисков на стыках 14 сервисов (это интеграция, API и контракты). Нина не подгоняла цифры под картинку — каждый уровень рос там, откуда приходили дефекты.

ПРАВИЛО

Смотрите на пирамиду дефектов, а не тестов. Разложите инциденты за полгода по уровню, где их можно было поймать раньше всего. Тесты добавляйте туда, где больше всего таких дефектов.

ПЛОХО

Пирамида как KPI

«Доля unit должна быть 70%».

Команда пишет тесты на геттеры, чтобы попасть в норму.

ХОРОШО

Пирамида как проверка

«Почему 40% дефектов ловит только UI?»

Команда переносит проверки ниже, и доли меняются сами.

7.3 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Рожок мороженого	Медленный регресс, дорогая поддержка	Переносить проверки ниже
Песочные часы	Нет середины, дыры на стыках	Добавить API и интеграционные
Пропорция как цель	Тесты ради чисел	Анализ дефектов по уровням

ПРАКТИКА 7 после главы 07

Нарисуйте текущую пирамиду тестов и пирамиду дефектов своего продукта.

ЧТО СДЕЛАТЬ

01	Посчитайте тесты по уровням и время их прогона.	☐
02	Разложите инциденты за полгода по уровню, где их можно было поймать раньше всего.	☐
03	Сравните две пирамиды: где тестов мало, а дефектов много?	☐
04	Выберите один уровень для усиления в ближайший квартал.	☐

РЕЗУЛЬТАТ Повторяйте сравнение раз в квартал — пирамида должна меняться вслед за рисками.

08 Критерии выбора инструментов автотестирования

После неё вы выбираете инструменты по взвешенной матрице критериев, проверяете кандидатов пилотом и не выбираете фреймворк «потому что модно».

8.1 Критерии
8.2 Кейс «Полки»: матрица для UI-фреймворка

Инструмент автотестов живёт в проекте годами, и цена неудачного выбора — не лицензия, а сотни тестов, которые придётся переписывать. Поэтому выбирать стоит по критериям, взвешенным под контекст команды, и проверять кандидатов на реальных сценариях продукта, а не на демо из документации.

8.1 Критерии

Совместимость со стеком	Поддерживает ли инструмент ваш язык, браузеры, платформы, протоколы.
Кто будет писать	Разработчики охотнее пишут тесты на языке продукта; отдельная QA-команда может выбрать свой.
Стабильность и диагностика	Автоожидания, трейсы, видео, понятные сообщения об ошибках.
Скорость и параллельность	Встроенный параллельный запуск, шардирование, запуск без UI.
Экосистема	Сообщество, плагины, интеграция с CI и отчётами, скорость ответа на баги.
Стоимость владения	Лицензии, инфраструктура, обучение, найм людей с этим навыком.

8.2 Кейс «Полки»: матрица для UI-фреймворка

КРИТЕРИЙ	ВЕС	SELENIUM	CYPRESS	PLAYWRIGHT
Стабильность, автоожидания	5	2	4	5
Диагностика (трейс, видео)	4	2	4	5
Параллельность из коробки	4	3	2	5
Язык фронтенда (TS)	3	4	5	5
Несколько вкладок, домены	2	4	2	5
Опыт команды	2	5	2	2
Итого (взвешенно)	—	**56**	**66**	**94**

КАК ПРОВЕРИЛИ ВЫБОР

1 **Пилот**
Два кандидата, по 5 одинаковых сценариев «Полки», по 3 дня на каждый.

2 **Замеры**
Время прогона, число случайных падений на 50 запусках, время на диагностику падения.

✓ **Итог**
Playwright: 0 случайных падений из 250 прогонов, диагностика по трейсу — 2 мин против 15.

ВАЖНО · ТИПИЧНАЯ ОШИБКА

Выбирать по числу звёзд на GitHub или по докладу с конференции. Популярность говорит об экосистеме, но не о том, подходит ли инструмент вашим сценариям. Пилот на трёх-пяти реальных тестах занимает неделю и экономит год.

ПЛОХО

Выбор по вкусу

«Я раньше работал на Selenium, давайте его».

Решение одного человека, без замеров.

ХОРОШО

Выбор по данным

Матрица с весами, согласованная с командой.

Пилот на реальных сценариях, решение записано в ADR.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Зоопарк инструментов	Каждая команда со своим стеком, нет переиспользования	Один стек на уровень
Выбор без разработчиков	Тесты на чужом языке никто не поддерживает	Решать вместе
Нет записи решения	Через год спор начинается заново	ADR с критериями и замерами

ПРАКТИКА 8 после главы 08

Соберите матрицу выбора для инструмента, который вам предстоит выбрать или пересмотреть.

ЧТО СДЕЛАТЬ

- [ ] 01 Выпишите 5–7 критериев и согласуйте веса с командой.
- [ ] 02 Оцените 2–3 кандидата по документации и отзывам.
- [ ] 03 Проведите пилот двух лучших на 3–5 реальных сценариях.
- [ ] 04 Запишите решение и замеры в ADR.

РЕЗУЛЬТАТ ADR пригодится, когда через год кто-то предложит «перейти на новый модный фреймворк».

09 Языки программирования для автотестов

После неё вы выбираете язык автотестов исходя из того, кто их будет писать и поддерживать, и понимаете, где один язык с продуктом выгоднее, а где нет.

9.1 Популярные языки
9.2 Кейс «Полки»: два языка вместо трёх

Язык автотестов определяет, кто сможет их читать, править и ревьюить. Если тесты написаны на языке продукта, разработчик чинит упавший тест сам, не дожидаясь автоматизатора. Если на другом — тесты живут в отдельном мире, и их поддержка целиком ложится на QA.

9.1 Популярные языки

ЯЗЫК	СИЛЬНЫЕ СТОРОНЫ	ТИПИЧНЫЙ СТЕК
Java / Kotlin	Строгая типизация, зрелая экосистема, язык многих бэкендов	JUnit 5, RestAssured, Selenide, Allure
Python	Низкий порог входа, лаконичность, много QA-специалистов	pytest, requests/httpx, Playwright
TypeScript / JS	Язык фронтенда, лучшие современные UI-инструменты	Playwright, Vitest, Jest
C#	Стек .NET	NUnit, xUnit, Playwright for .NET
Go	Быстрые тесты сервисов на Go	testing, testify

9.2 Кейс «Полки»: два языка вместо трёх

У «Полки» бэкенд на Kotlin, фронтенд на TypeScript, а старые UI-тесты — на Java. Марат предлагал писать API-тесты на Kotlin, Тимофей — на TypeScript вместе с UI. Решили так: unit и интеграционные тесты сервисов — на Kotlin, рядом с кодом; API, UI и E2E — на TypeScript в одном репозитории с общими фикстурами. Java ушла вместе со старым Selenium-набором.

АРГУМЕНТЫ РЕШЕНИЯ

1 **Unit и интеграция на Kotlin**

Пишут разработчики, тесты лежат в модуле сервиса и идут в его сборке.

2 **API и UI на TypeScript**

Один клиент API и одни фикстуры для обоих уровней: заказ для UI-теста готовится тем же кодом.

✓ **Итог**

Фронтендеры начали сами чинить UI-тесты: 40% правок через полгода — от них.

ПРАВИЛО

Выбирайте язык, на котором тесты будут ревьюить разработчики. Тесты, которые никто кроме автора не читает, быстро становятся обузой.

ПЛОХО

Язык по привычке QA

«У нас все автоматизаторы на Python».

Разработчики на Kotlin не трогают тесты; падение ждёт QA.

ХОРОШО

Язык по владельцу

Unit — на языке сервиса.

Системные тесты — на языке, где есть лучшие инструменты и общие фикстуры.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Три языка на три уровня	Дублирование клиентов и данных	Не больше двух языков
Тесты без типизации в большом проекте	Ошибки рефакторинга видны только в рантайме	TypeScript вместо JS, типы в Python
Язык, которого нет на рынке найма	Некому поддерживать	Учитывать рынок специалистов

ПРАКТИКА 9 после главы 09

Определите, на каком языке должны жить тесты каждого уровня в вашем проекте.

ЧТО СДЕЛАТЬ

01	Выпишите языки продукта и уровни тестов.	☐
02	Для каждого уровня решите, кто основной автор и кто ревьюер.	☐
03	Выберите язык, понятный обоим, и запишите решение.	☐
04	Если языков больше двух — составьте план сокращения.	☐

РЕЗУЛЬТАТ Ключевой показатель — сколько упавших тестов чинят разработчики без помощи QA.

10 Фреймворки для UI-автотестов

После неё вы различаете архитектуры UI-фреймворков, понимаете их сильные и слабые стороны и выбираете инструмент для веба и мобильных платформ.

10.1 Веб · 10.2 Мобильные платформы · 10.3 Кейс «Полки»

Все UI-фреймворки решают одну задачу — управлять интерфейсом как пользователь, — но устроены по-разному. От архитектуры зависит, насколько тесты стабильны, быстры и что вообще можно проверить: несколько вкладок, iframe, загрузку файла, сетевые запросы.

10.1 Веб

ФРЕЙМВОРК	КАК УСТРОЕН	СИЛЬНО	СЛАБО
Selenium WebDriver	Протокол W3C WebDriver, драйвер на каждый браузер	Стандарт, любые языки, Grid	Нет автоожиданий, медленнее, больше flaky
Selenide	Обёртка над Selenium на Java/Kotlin	Автоожидания, лаконичный API	Наследует ограничения WebDriver
Cypress	Работает внутри браузера, рядом с приложением	Удобная отладка, тайм-тревел	Одна вкладка, слабая параллельность бесплатно
Playwright	CDP и собственные протоколы браузеров	Автоожидания, трейсы, вкладки, сеть, шардирование	Моложе экосистема плагинов
WebdriverIO	WebDriver и BiDi, JS/TS	Веб и мобильные одним API	Сложнее настройка

10.2 Мобильные платформы

Espresso (Android) и XCUITest (iOS)	Нативные фреймворки: быстрые и стабильные, тесты пишут на Kotlin и Swift рядом с кодом приложения.
Appium	Кросс-платформенный, протокол WebDriver: один язык для двух платформ, но медленнее и капризнее нативных.
Maestro	Декларативные сценарии в YAML, низкий порог входа для простых путей.

10.3 Кейс «Полки»

Для веба выбрали Playwright (матрица в главе 8). Для мобильных Тимофей сначала хотел Appium — один набор на две платформы. Пилот показал 9% случайных падений и 3 минуты на тест. Мобильные разработчики предложили нативные Espresso и XCUITest: 8 ключевых сценариев на каждой платформе, 40 секунд на тест, flaky ниже 1%. Тесты пишут сами мобильщики.

ПЛОХО

Один инструмент на всё

Appium для веба, iOS и Android ради «единого стека».

Медленно, нестабильно, тесты никто кроме QA не трогает.

ХОРОШО

Лучший инструмент на платформу

Playwright для веба, нативные фреймворки для мобильных.

Тесты рядом с кодом, авторы — разработчики платформы.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Своя обёртка над WebDriver на 5 000 строк	Поддерживать фреймворк дороже тестов	Брать готовый инструмент с автоожиданиями
Запись тестов рекордером без правки	Хрупкие селекторы и лишние шаги	Рекордер — черновик, не результат
Выбор без пилота на мобильных	Неожиданная нестабильность эмуляторов	Пилот на реальных устройствах и фермах

ПРАКТИКА 10 после главы 10

Проверьте, подходит ли ваш UI-фреймворк вашим сценариям.

ЧТО СДЕЛАТЬ

01	Выпишите 5 сценариев, которые сложно автоматизировать: вкладки, iframe, файлы, сеть.	☐
02	Проверьте, как текущий фреймворк справляется с каждым.	☐
03	Если есть проблемы — проведите пилот альтернативы на этих же сценариях.	☐

РЕЗУЛЬТАТ Переход оправдан, если пилот снижает время и нестабильность хотя бы вдвое.

11 Инструменты для API-тестирования

После неё вы отличаете инструменты для ручного исследования API от инструментов для автотестов и собираете стек для HTTP, gRPC, контрактов и моков.

11.1 Кейс «Полки»: от коллекции к коду

Инструменты для API делятся на две группы. Первые нужны человеку: отправить запрос, посмотреть ответ, сохранить коллекцию. Вторые — коду: библиотеки, на которых пишутся автотесты, живущие в репозитории и запускаемые в CI. Ошибка многих команд — растить автотесты в первой группе, пока коллекция Postman не становится неуправляемой.

ЗАДАЧА	ИНСТРУМЕНТЫ	КОММЕНТАРИЙ
Ручное исследование	Postman, Insomnia, Bruno, curl, HTTPie	Bruno хранит коллекции в git как файлы
Автотесты на Java/Kotlin	RestAssured, Ktor client + JUnit	Зрелый DSL для проверок
Автотесты на Python	pytest + httpx/requests	Быстрый старт, фикстуры pytest
Автотесты на TypeScript	Playwright APIRequestContext, supertest	Общие фикстуры с UI-тестами
gRPC	grpcurl, ghz, клиенты из proto	Проверка контракта через сгенерированный код
Контракты	Pact, Spring Cloud Contract	Consumer-driven контракты между сервисами
Моки внешних API	WireMock, MockServer, Prism	Prism поднимает мок по OpenAPI
Нагрузка	k6, Gatling, JMeter	Отдельная тема, но стек лучше общий

11.1 Кейс «Полки»: от коллекции к коду

У аналитика Алисы была коллекция Postman на 340 запросов с тестами на JavaScript внутри. Она работала, пока ею пользовался один человек. Когда коллекцию запустили в CI

через Newman, оказалось, что 60 запросов зависят от переменных, которые Алиса выставляла руками.

ЧТО СДЕЛАЛИ

1 **Разделили назначение**
Postman остался для исследования; автотесты — в репозитории на Playwright.

2 **Клиент API**
Типизированный клиент сгенерировали из OpenAPI-спецификаций 14 сервисов.

3 **Перенос**
340 запросов превратились в 190 тестов: дубли схлопнулись, проверки стали строже.

✓ **Итог**
Изменение схемы ломает компиляцию тестов, а не прогон через неделю.

ПЛОХО

Коллекция как автотесты

Тесты в JSON-файле на 12 000 строк.

Ревью невозможно, конфликты при каждом изменении.

ХОРОШО

Код как автотесты

Тесты в репозитории, клиент из спецификации.

Ревью в merge request, история изменений в git.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Сырые HTTP-вызовы в каждом тесте	URL и заголовки дублируются сотни раз	Клиент API с методами
Мок вручную на каждый тест	Моки расходятся со спецификацией	Генерировать моки из OpenAPI
Отдельный стек для нагрузки	Дублирование сценариев и данных	Общий клиент и данные

ПРАКТИКА 11 после главы 11

Соберите свой стек API-тестирования по задачам.

ЧТО СДЕЛАТЬ

01	Перечислите задачи: исследование, автотесты, контракты, моки, нагрузка.	☐
02	Подберите инструмент на каждую, ориентируясь на язык из главы 9.	☐
03	Если есть OpenAPI — попробуйте сгенерировать из него клиент для тестов.	☐

РЕЗУЛЬТАТ Сгенерированный клиент окупается при первом же изменении схемы.

12 Инструменты для unit-тестов

После неё вы собираете набор для unit-тестов: раннер, библиотеку проверок, моки, генерацию данных, мутационное тестирование — и знаете, зачем каждый элемент.

12.1 Покрытие и мутации · 12.2 Кейс «Полки»

Unit-тесты пишут разработчики, поэтому инструменты для них выбирают по языку продукта. Набор почти всегда одинаков: раннер запускает тесты, библиотека проверок формулирует ожидания, библиотека моков подменяет зависимости. Сверху добавляют параметризацию, генерацию данных и метрики качества тестов.

ЭЛЕМЕНТ	KOTLIN / JAVA	TYPESCRIPT	PYTHON
Раннер	JUnit 5, Kotest	Vitest, Jest	pytest
Проверки	AssertJ, Kotest assertions, Strikt	expect (встроен)	assert + pytest
Моки	MockK, Mockito	vi.fn, MSW для сети	unittest.mock, pytest-mock
Property-based	Kotest property, jqwik	fast-check	Hypothesis
Покрытие	JaCoCo, Kover	v8 / istanbul	coverage.py
Мутации	PIT	Stryker	mutmut

12.1 Покрытие и мутации

Покрытие показывает, какие строки выполнились во время тестов, но не то, проверили ли тесты результат. Тест без единого assert даёт такое же покрытие, как тест с десятком проверок. Мутационное тестирование отвечает на другой вопрос: инструмент вносит в код маленькие изменения (меняет > на >=, + на -) и смотрит, упадёт ли хоть один тест.

ФОРМУЛА 12.1 · ДОЛЯ УБИТЫХ МУТАНТОВ

$$MS = \frac{M_{\text{k}}}{M - M_{\text{e}}} \tag{12.1}$$

где

M	— сколько мутантов создано	шт.
M_{k}	— мутантов, на которых упал хотя бы один тест	шт.
M_{e}	— эквивалентных мутантов, не меняющих поведение	шт.

```
Калькулятор доставки: 212 мутантов, 6 эквивалентных, убито 189 → MS = 189 / 206 ≈ 92%
```

12.2 Кейс «Полки»

Марат запустил PIT на модуле промокодов с покрытием 78%. Мутационный счёт оказался 41%: половина тестов вызывала код и проверяла только, что исключения нет. После доработки тестов покрытие выросло лишь до 82%, а мутационный счёт — до 88%, и именно этот показатель команда стала смотреть на ревью для доменных модулей.

ПЛОХО

Покрытие как цель

«Покрытие не ниже 80% на каждый MR».

Тесты без проверок, ради процента.

ХОРОШО

Покрытие как сигнал

Непокрытые ветки — повод посмотреть, а не блокер.

Для доменной логики — мутационный счёт раз в неделю.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Mockito и MockK в одном проекте	Два стиля, путаница	Один инструмент моков
Мутации на каждый MR по всему коду	Прогон на часы	Только изменённые модули или ночью
assertTrue(result != null)	Проверка ни о чём	Проверять конкретное значение

ПРАКТИКА 12 после главы 12

Оцените качество unit-тестов одного модуля мутационным тестированием.

ЧТО СДЕЛАТЬ

01	Запустите PIT, Stryker или mutmut на одном доменном модуле.	☐
02	Посмотрите выживших мутантов: какие проверки отсутствуют?	☐
03	Допишите проверки и сравните покрытие и мутационный счёт до и после.	☐

РЕЗУЛЬТАТ Разница между покрытием и мутационным счётом показывает, сколько тестов «просто выполняют код».

13 Сравнение популярных стеков

После неё вы сравниваете готовые связки инструментов по типовым контекстам команды и выбираете стек целиком, а не по одному инструменту.

13.1 Сравнение по критериям · 13.2 Кейс «Полки»: итоговый стек

Инструменты работают в связке: язык, раннер, UI-фреймворк, HTTP-клиент, отчёты и CI. Выбирать их по отдельности рискованно — лучший раннер может плохо дружить с лучшим отчётом. Поэтому полезно смотреть на устоявшиеся стеки, которые сообщество уже проверило вместе.

СТЕК	СОСТАВ	КОМУ ПОДХОДИТ
JVM-классика	Java/Kotlin, JUnit 5, Selenide, RestAssured, Allure	Бэкенд на JVM, большая QA-команда, корпоративная среда
TypeScript-современный	TS, Playwright (UI + API), Vitest, Allure или встроенный отчёт	Фронтенд на TS, разработчики пишут тесты сами
Python-универсальный	Python, pytest, Playwright, httpx, Allure	Смешанный стек, много QA без сильного опыта разработки
Cypress-фронтенд	TS, Cypress, компонентные тесты	Одностраничное приложение, фронтенд-команда
Мобильный нативный	Espresso, XCUITest, Firebase Test Lab или фермы устройств	Приложения с собственными мобильными командами

13.1 Сравнение по критериям

КРИТЕРИЙ	JVM	TS + PLAYWRIGHT	PYTHON
Порог входа для QA	средний	средний	низкий
Скорость UI-тестов	средняя	высокая	высокая
Разработчики фронтенда пишут сами	редко	часто	редко

КРИТЕРИЙ	JVM	TS + PLAYWRIGHT	PYTHON
Рынок специалистов в РФ	широкий	растёт	широкий
Типизация	строгая	строгая	по желанию

13.2 Кейс «Полки»: итоговый стек

```
unit бэкенда       Kotlin · JUnit 5 · MockK · Kover · PIT (ночью)
unit фронтенда     TypeScript · Vitest · Testing Library · MSW
интеграция         Kotlin · Testcontainers · WireMock
контракты          Pact (consumer — фронт и сервисы, provider — сервисы)
API, UI, E2E       TypeScript · Playwright · клиент из OpenAPI
мобильные          Espresso · XCUITest
отчёты и CI        Allure TestOps · GitLab CI · Docker-раннеры
```

ПРАВИЛО

Стек должен сокращать число мест, где живёт одно и то же знание. Если адрес тестового магазина записан в трёх репозиториях на трёх языках, стек выбран неудачно.

ПЛОХО

Лучшее от каждого

Пять языков и семь фреймворков.

Каждый инструмент лучший, вместе — несовместимы.

ХОРОШО

Связка, которая работает вместе

Два языка, общий клиент и фикстуры.

Единый формат отчётов для всех уровней.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Стек под одного автоматизатора	Уходит человек — уходит экспертиза	Стек, понятный команде
Разные отчёты по уровням	Нет общей картины качества	Общий формат, например Allure
Миграция «за выходные»	Половина тестов в старом стеке навсегда	План миграции с датой отключения

ПРАКТИКА 13 после главы 13

Опишите свой стек одной таблицей, как в кейсе «Полки».

ЧТО СДЕЛАТЬ

01	Перечислите уровни тестов и инструменты на каждом.	☐
02	Отметьте места, где одно и то же знание дублируется.	☐
03	Решите, что убрать или объединить в ближайшие полгода.	☐

РЕЗУЛЬТАТ Таблица стека — хороший первый слайд для онбординга новых автоматизаторов.

14 Когда использовать low-code / no-code решения

После неё вы понимаете, где low-code инструменты автотестов экономят время, где создают ловушку, и оцениваете их по стоимости владения.

14.1 Виды решений | 14.2 Где уместно, а где нет | 14.3 Кейс «Полки»

Low-code инструменты обещают автотесты без программирования: тест записывается кликами, шаги описываются на естественном языке, а ИИ сам чинит локаторы. Для части задач это правда. Для других — ловушка, которая обнаруживается через год, когда тестов сотни, а перенести их некуда.

14.1 Виды решений

Рекордеры	Записывают действия в браузере и воспроизводят. Playwright codegen и Selenium IDE — бесплатные, их результат — обычный код.
Облачные платформы	Визуальный редактор сценариев, самолечение локаторов, запуск в облаке, подписка.
BDD и ключевые слова	Gherkin или Robot Framework: сценарий на почти естественном языке, за шагами стоит код.
ИИ-генерация	Агенты пишут тест по описанию или по сессии пользователя; результат требует ревью.

14.2 Где уместно, а где нет

УМЕСТНО	НЕУМЕСТНО
Команда без программистов, простой продукт	Сложные данные и цепочки API
Smoke-проверки маркетинговых страниц	Регресс на тысячи кейсов

УМЕСТНО	НЕУМЕСТНО
Быстрый прототип перед выбором стека	Тесты, которые должны писать разработчики
Мониторинг внешнего вида в нескольких браузерах	Продукт с частыми изменениями интерфейса

14.3 Кейс «Полки»

Маркетинг «Полки» постоянно менял промостраницы, и раз в месяц на одной из них ломалась кнопка «Скачать приложение». Нина отдала эти проверки low-code платформе: маркетолог сам записал 14 сценариев за день. Основной продукт остался на коде: Тимофей оценил, что перенос 640 API-тестов в визуальный редактор занял бы больше времени, чем их написание.

ФОРМУЛА 14.1 · СТОИМОСТЬ ВЛАДЕНИЯ ЗА ГОРИЗОНТ

$$TCO = L \cdot m + n \cdot (c_w + c_s \cdot m) \tag{14.1}$$

где

- L — подписка в месяц, ₽
- m — горизонт, мес.
- n — число тестов, шт.
- c_w, c_s — стоимость написания теста и его поддержки в месяц, ₽

Сравнивайте TCO двух вариантов на 24 месяца, а не цену первого теста

ПЛОХО

Low-code для всего

«Пусть ручные тестировщики записывают регресс».

Через год 900 записанных тестов без переиспользования шагов.

ХОРОШО

Low-code для краёв

Промостраницы, мониторинг, прототипы.

Ядро продукта — в коде с ревью.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Привязка к вендору	Тесты нельзя вынести, цена растёт	Проверить экспорт до покупки
Рекордер без правки	Дубли шагов, хрупкие локаторы	Рекорд → рефакторинг
ИИ-тесты без ревью	Проверяют не то, что нужно	Ревью как у обычного кода

ПРАКТИКА 14 после главы 14

Найдите в своём продукте зону, где low-code сэкономит время, и посчитайте TCO.

ЧТО СДЕЛАТЬ

- [ ] 01 Выпишите проверки, которые сейчас никто не автоматизирует из-за нехватки рук.
- [ ] 02 Отберите те, что просты и редко меняются.
- [ ] 03 Посчитайте TCO на 24 месяца для low-code и для кода по формуле (14.1).

РЕЗУЛЬТАТ Отдельно проверьте, можно ли выгрузить тесты из платформы, если вы от неё откажетесь.

15 Структура автотестового проекта

После неё вы раскладываете проект автотестов по папкам и модулям так, чтобы новый человек находил нужное за минуту, а тесты, данные и инфраструктура не смешивались.

15.1 Где живут тесты
15.2 Кейс «Полки»: репозиторий системных тестов
15.3 Соглашения об именах

Проект автотестов — это такой же код, как продукт, и ему нужна такая же структура. Когда тестов двадцать, порядок не важен. Когда их тысяча, от структуры зависит, сколько времени уходит на поиск теста, понимание его данных и правку при изменении продукта.

15.1 Где живут тесты

Рядом с кодом	Unit и интеграционные тесты сервиса — в его репозитории, в папке test. Падают в сборке сервиса, чинятся его командой.
Отдельный репозиторий	Системные API, UI и E2E-тесты, которые проверяют несколько сервисов вместе.
Монорепозиторий	Если продукт в монорепо, тесты лежат там же, но в отдельном пакете с явными границами.

15.2 Кейс «Полки»: репозиторий системных тестов

```
polka-tests/
 api/              клиенты, сгенерированные из OpenAPI
 fixtures/         фабрики данных и Playwright-фикстуры
 pages/            объекты страниц и компонентов
 tests/
  api/cart/        cart.positive.spec.ts, cart.promo.spec.ts
  api/orders/
  ui/checkout/
  e2e/              12 сквозных путей
 config/           env.local.ts, env.stage.ts, env.prod-synthetic.ts
 utils/            ожидания, генераторы, хелперы отчёта
 playwright.config.ts · .gitlab-ci.yml · README.md
```

ПРАВИЛО

Тесты группируйте по функциональным областям продукта, а не по типу действий. Папка tests/cart понятна продакту и разработчику; папка tests/clicks — никому.

15.3 Соглашения об именах

ЧТО	СОГЛАШЕНИЕ	ПРИМЕР
Файл теста	<область>.<аспект>.spec.ts	cart.promo.spec.ts
Название теста	что происходит при каком условии	applies 10% promo to cart over 1 500 ₽
Тег набора	@smoke, @regress, @e2e	test('...', { tag: '@smoke' })
Тег области	@cart, @slots, @payments	для sanity и владельцев

ПЛОХО

Плоская папка

tests/test1.spec.ts … test384.spec.ts

Названия вида «test order 2 new».

ХОРОШО

Карта продукта

tests/api/orders/orders.cancel.spec.ts

«refunds payment when order is cancelled before assembly».

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
utils.ts на 3 000 строк	Свалка, которой боятся касаться	Делить по назначению
Конфиги окружений в коде тестов	Тест нельзя запустить на другом стенде	Отдельная папка config
Нет README	Новичок тратит день на запуск	README с запуском за 3 команды

ПРАКТИКА 15 после главы 15

Проведите ревизию структуры своего проекта автотестов.

ЧТО СДЕЛАТЬ

01	Нарисуйте дерево папок до второго уровня.	☐
02	Проверьте, можно ли по пути к файлу понять, какую часть продукта он проверяет.	☐
03	Введите соглашения об именах файлов, тестов и тегов.	☐
04	Напишите README: запуск локально, на стенде, выборочный запуск по тегу.	☐

РЕЗУЛЬТАТ Проверка: новый человек запускает тесты по README без вопросов за 15 минут.

16 Test layers и separation of concerns

После неё вы делите код автотестов на слои — сценарий, бизнес-действия, взаимодействие с системой, инфраструктура — и знаете, какое изменение продукта затронет какой слой.

16.1 Четыре слоя · 16.2 Как это выглядит · 16.3 Кейс «Полки»: переименование эндпоинта

Тест, в котором в одной функции перемешаны URL, селекторы, генерация данных и проверки, работает. Но когда меняется адрес эндпоинта, правка расходится по сотне файлов. Разделение на слои решает это: каждое знание о системе живёт ровно в одном месте, и изменение продукта затрагивает один слой.

16.1 Четыре слоя

СЛОЙ	ОТВЕЧАЕТ НА ВОПРОС	ПРИМЕР	МЕНЯЕТСЯ, КОГДА
Сценарий (тест)	Что проверяем?	Отмена заказа возвращает деньги	Меняются требования
Бизнес-действия	Как это сделать в терминах продукта?	orders.create(), orders.cancel()	Меняется процесс
Взаимодействие	Как говорить с системой?	HTTP-клиент, page objects	Меняются API или интерфейс
Инфраструктура	Где и с какими настройками?	Конфиг, авторизация, логирование	Меняется окружение

16.2 Как это выглядит

```
test('refunds payment when order is cancelled', async ({ customer }) ⇒ {
 const order = await customer.placeOrder({ items: 3, pay: 'card' });   // действие
 await customer.cancel(order);
 await expect.poll(() ⇒ customer.refundOf(order)).toBe(order.total);  // сценарий
});

// customer.placeOrder внутри — вызовы api.cart, api.slots, api.payments
// api.* сгенерированы из OpenAPI; базовый URL и токены — в config
```

16.3 Кейс «Полки»: переименование эндпоинта

Когда сервис корзины перешёл с /cart/v1 на /baskets/v2, в старом Selenium-наборе путь встречался в 47 файлах. В новом наборе изменение коснулось одного файла сгенерированного клиента, и его пересобрали из новой спецификации. Ни один тест не потребовал правки, потому что тесты знали только о действии «положить в корзину».

ПРАВИЛО

Проверки — только в слое сценария. Если бизнес-действие или page object сам делает assert, тест перестаёт быть читаемым: непонятно, что он проверяет, а что делает попутно.

ПЛОХО

Всё в тесте

fetch('https://stage.polka/cart/v1/items', {headers: {...}})

page.click('.btn-primary') в 30 тестах.

ХОРОШО

Каждое знание один раз

customer.addToCart(milk)

Адрес, заголовки и селекторы — в своих слоях.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Слишком много слоёв	Чтобы понять тест, нужно открыть 6 файлов	3–4 слоя, не больше
Проверки в page objects	Неясно, что проверяет тест	assert только в тесте
Бизнес-действия знают про UI	Нельзя переиспользовать в API-тестах	Действия поверх API, UI отдельно

ПРАКТИКА 16 после главы 16

Разметьте слои в одном своём тесте.

ЧТО СДЕЛАТЬ

01	Возьмите длинный тест и подсветите строки по слоям: сценарий, действия, взаимодействие, инфраструктура.	☐
02	Вынесите каждую группу в свой слой.	☐
03	Проверьте, сколько файлов затронет переименование одного эндпоинта до и после.	☐

РЕЗУЛЬТАТ Хорошее разделение делает тело теста читаемым для продакта.

17 Page Object и альтернативные паттерны

После неё вы применяете Page Object и компонентные объекты без раздувания, знаете альтернативы — Screenplay, App Actions, фикстуры — и выбираете паттерн под размер проекта.

17.1 Компонентные объекты · 17.2 Альтернативы · 17.3 Кейс «Полки»

Page Object прячет устройство страницы за методами. Тест говорит «войти как пользователь», а не «найти поле email, ввести, найти поле пароля…». Когда вёрстка меняется, правится один объект, а не все тесты. Паттерн старый и надёжный, но у него есть типичные болезни, которые стоит знать заранее.

КЛАССИКА — Мартин Фаулер, «PageObject», 2013; Selenium wiki, 2009

Page Object

Фаулер описал паттерн как объект, который оборачивает HTML-страницу или её фрагмент API, специфичным для приложения. Он подчеркнул: страница — не обязательно целая страница; объекты стоит делать для значимых фрагментов.

Он же предупредил, что проверки лучше держать в тестах, а не в объектах страниц.

17.1 Компонентные объекты

В современных SPA одна и та же карточка товара встречается в каталоге, поиске, корзине и рекомендациях. Если описывать её в каждом Page Object, получаются четыре копии. Поэтому объекты строят из компонентов: ProductCard, CartWidget, SlotPicker — а страница собирает их.

```
class ProductCard {
 constructor(private root: Locator) {}
 price = () ⇒ this.root.getByTestId('price');
 add   = () ⇒ this.root.getByRole('button', { name: 'В корзину' }).click();
}
class CatalogPage {
 constructor(private page: Page) {}
 card = (name: string) ⇒ new ProductCard(
  this.page.getByTestId('product-card').filter({ hasText: name }));
}
```

17.2 Альтернативы

ПАТТЕРН	ИДЕЯ	КОГДА ВЫБИРАТЬ
Page Object	Объект на страницу или компонент	Большинство веб-проектов
Screenplay	Актёры выполняют задачи через способности	Большие наборы, много ролей, BDD
App Actions	Тест вызывает внутренние функции приложения для подготовки	Cypress, когда подготовка через UI слишком долгая
Фикстуры	Готовые объекты и состояния внедряются в тест	Playwright, pytest: подготовка без наследования

17.3 Кейс «Полки»

В старом наборе был BasePage на 1 900 строк, от которого наследовались 34 страницы. Тимофей заменил наследование композицией: 21 компонентный объект и 9 страниц, собранных из них, плюс фикстуры Playwright для авторизованных ролей. Средний объект — 40 строк.

ПЛОХО

Божественный BasePage

Все хелперы в базовом классе.

Каждая страница наследует 200 методов, из которых нужны 5.

ХОРОШО

Композиция

Страница собрана из компонентов.

Общее — во фикстурах и утилитах, а не в наследовании.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Метод на каждый клик	Объект повторяет вёрстку	Методы — действия пользователя
Глубокое наследование	Изменение базы ломает всё	Композиция компонентов
Локаторы в тестах	Page Object теряет смысл	Все локаторы — в объектах

ПРАКТИКА 17 после главы 17

Найдите повторяющийся фрагмент интерфейса и вынесите его в компонентный объект.

ЧТО СДЕЛАТЬ

01	Найдите компонент, который встречается на 3+ страницах.	☐
02	Опишите его как отдельный объект с методами-действиями.	☐
03	Замените дубли в page objects на этот компонент.	☐
04	Посчитайте, сколько строк ушло.	☐

РЕЗУЛЬТАТ Если объект страницы больше 150 строк, в нём почти наверняка прячутся компоненты.

18 Работа с тестовыми данными

После неё вы готовите данные для каждого теста отдельно, выбираете между фабриками, снимками и генерацией и не зависите от записей, созданных руками на стенде.

18.1 Способы подготовки
18.2 Кейс «Полки»: фабрики с разумными значениями
18.3 Очистка

Половина нестабильных тестов падает не из-за кода, а из-за данных: кто-то удалил тестовый товар, другой тест потратил промокод, магазин закрыли на ремонт. Правило простое: тест сам создаёт данные, которые ему нужны, и не рассчитывает на то, что кто-то оставил их на стенде.

18.1 Способы подготовки

Способ	Как	Плюсы	Минусы
Фабрики через API	Тест создаёт сущности публичными или служебными вызовами	Реалистично, независимо	Медленнее прямой записи
Прямая запись в базу	SQL-скрипты или репозитории	Быстро	Обходит бизнес-правила, ломается при миграциях
Снимок базы	Восстановление эталонного дампа перед прогоном	Сложные данные за секунды	Дамп устаревает, тяжело поддерживать
Генераторы	Faker, property-based	Разнообразие, ловит граничные случаи	Нужен seed для воспроизведения

18.2 Кейс «Полки»: фабрики с разумными значениями

```
const order = await factory.order({
 store: await factory.store({ zone: 'center' }),   // своя тестовая лавка
 items: [{ sku: 'milk-1l', qty: 2 }],
});
// всё остальное — по умолчанию: пользователь, адрес, слот, оплата картой
// у каждого объекта уникальный префикс прогона: t-20261031-0712-...
```

Фабрика задаёт разумные значения по умолчанию, а тест указывает только то, что важно для проверки. Читая тест, сразу видно, от чего зависит результат. Уникальный префикс прогона позволяет параллельным тестам не мешать друг другу и находить мусор для очистки.

18.3 Очистка

- Тест удаляет то, что создал, в блоке afterEach — если сущность вообще можно удалить.
- Ночная задача удаляет всё с префиксом t- старше суток: страховка от упавших тестов.
- Для неудаляемых сущностей (платежи) — отдельный тестовый мерчант, который периодически сбрасывают.

ВАЖНО · ТИПИЧНАЯ ОШИБКА

Держать в тестах идентификаторы конкретных записей: userId = 1842, sku = 'A-77'. Через месяц запись изменит ручной тестировщик, и тест упадёт без единого изменения кода.

ПЛОХО

Данные со стенда

«Используем пользователя test@polka.ru, у него всё настроено».

Параллельные тесты меняют его корзину одновременно.

ХОРОШО

Данные теста

Каждый тест создаёт пользователя фабрикой.

Важные для проверки поля видны в коде теста.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Захардкоженные id	Падения после ручных правок стенда	Фабрики
Случайные данные без seed	Падение нельзя повторить	Логировать seed
Персональные данные из прода	Нарушение 152-ФЗ, утечки	Синтетика или маскирование

ПРАКТИКА 18 после главы 18

Уберите из тестов зависимость от заранее созданных данных.

ЧТО СДЕЛАТЬ

01	Найдите в тестах все захардкоженные идентификаторы и логины.	☐
02	Напишите фабрики для трёх самых частых сущностей с разумными значениями по умолчанию.	☐
03	Добавьте префикс прогона и ночную очистку.	☐

РЕЗУЛЬТАТ Проверка: тесты проходят на только что развёрнутом пустом стенде.

19 Конфигурация окружений

После неё вы запускаете один и тот же набор тестов локально, на стенде и в продакшене без правки кода и храните секреты так, чтобы они не попадали в репозиторий.

19.1 Что выносится в конфигурацию · 19.2 Иерархия источников · 19.3 Секреты · 19.4 Кейс «Полки»

Один тест должен уметь идти в разных местах: на ноутбуке разработчика против локально поднятых сервисов, в CI против временного стенда, после релиза — синтетикой в продакшене. Всё, что отличается между окружениями, выносится из кода в конфигурацию: адреса, учётные данные, таймауты, включённые функции.

19.1 Что выносится в конфигурацию

ПАРАМЕТР	LOCAL	STAGE	PROD-SYNTHETIC
Базовый URL	http://localhost:8080	https://stage-mr-412.polka.dev	https://polka.ru
Платёжный шлюз	WireMock	sandbox провайдера	служебный мерчант
Таймаут ожидания	5 с	10 с	15 с
Набор тестов	любой	@regress	@synthetic
Создание данных	фабрики	фабрики	только служебный магазин

19.2 Иерархия источников

ОТКУДА БЕРЁТСЯ ЗНАЧЕНИЕ

1 **Значения по умолчанию**
В коде конфига: работают для локального запуска.

2 **Файл окружения**
config/env.stage.ts — то, что отличает стенд.

3 **Переменные окружения**
Секреты и параметры конкретного прогона из CI.

✓ **Правило**
Каждый следующий источник переопределяет предыдущий.

19.3 Секреты

Токены, пароли и ключи никогда не лежат в репозитории, даже в тестовом. Их передают через защищённые переменные CI или хранилище секретов (Vault, облачный менеджер секретов) и маскируют в логах. Если секрет попал в git, его считают скомпрометированным и меняют, а не удаляют коммит.

19.4 Кейс «Полки»

В старом наборе адрес стенда был записан в 12 местах, а пароль служебного пользователя — в открытом виде в properties-файле. Тимофей свёл всё в типизированный конфиг: тест, которому нужен параметр, получает его из одного объекта, а при запуске с отсутствующим секретом прогон падает сразу с понятной ошибкой, а не через 10 минут на 401.

ПЛОХО

Конфиг в коде

if (process.env.CI) url = 'https://stage...'

Пароль в properties-файле в git.

ХОРОШО

Конфиг отдельно

ENV=stage npx playwright test

Секреты — в защищённых переменных CI, проверка при старте.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Ветвления по окружению в тестах	Тесты ведут себя по-разному	Различия — только в конфиге
Секреты в репозитории	Утечка доступа к стендам и продакшену	Хранилище секретов, маскирование
Молчаливые значения по умолчанию	Тест идёт не туда, куда думали	Печатать конфиг прогона в отчёт

ПРАКТИКА 19 после главы 19

Сделайте так, чтобы окружение выбиралось одной переменной.

ЧТО СДЕЛАТЬ

01	Найдите в тестах все адреса, таймауты и учётные данные.	☐
02	Вынесите их в конфиг с иерархией: умолчания → файл окружения → переменные.	☐
03	Перенесите секреты в хранилище и включите маскирование в логах CI.	☐
04	Добавьте печать итогового конфига (без секретов) в начало отчёта.	☐

РЕЗУЛЬТАТ Проверка: один и тот же коммит тестов проходит на двух окружениях без правок.

20 Управление зависимостями

После неё вы фиксируете версии библиотек и браузеров, обновляете их регулярно и безопасно и не получаете «у меня работает» из-за разных окружений.

20.1 Что фиксировать · 20.2 Как обновлять

Проект автотестов зависит от раннера, фреймворка, браузеров, драйверов, Docker-образов и десятков библиотек. Если версии не зафиксированы, один и тот же коммит сегодня проходит, а завтра падает, потому что вышла новая минорная версия чего-то в глубине дерева. Если не обновлять — через год обновление превращается в проект на месяц.

20.1 Что фиксировать

Lock-файлы	package-lock.json, poetry.lock, gradle.lockfile: точные версии всего дерева зависимостей, лежат в git.
Браузеры	Версия Playwright определяет версии браузеров; на CI — образ с той же версией.
Docker-образы	postgres:16.4, а не postgres:latest; в идеале — по digest.
Инструменты CI	Версии раннеров, плагинов отчётов, Node и JDK.

20.2 Как обновлять

ПРОЦЕСС «ПОЛКИ»

1. **Бот**
   Renovate каждый понедельник открывает MR с обновлениями, сгруппированными по риску.

ПРОЦЕСС «ПОЛКИ»

2 **Патчи**
Патч-версии мержатся автоматически, если прогон зелёный.

3 **Минорные и мажорные**
Ревью автоматизатора, прогон полного набора три раза подряд.

✓ **Итог**
Отставание от актуальных версий — не больше месяца; обновление Playwright — 20 минут вместо двух дней.

ПРАВИЛО

Обновляйте часто и маленькими шагами. Одно обновление на одну версию легко откатить и понять; обновление сразу на два года вперёд превращается в археологию.

ПЛОХО

Плавающие версии

"@playwright/test": "^1.40.0" без lock-файла.

image: node:latest на CI.

ХОРОШО

Точные версии

Lock-файл в git, npm ci на CI.

image: mcr.microsoft.com/playwright:v1.56.0-noble

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
latest в образах	Внезапные падения без изменений кода	Конкретная версия или digest
Обновление раз в год	Миграция на недели	Еженедельный бот
Разные версии локально и в CI	«У меня работает»	Запуск в том же контейнере

ПРАКТИКА 20 после главы 20

Проверьте воспроизводимость своего прогона.

ЧТО СДЕЛАТЬ

01	Убедитесь, что lock-файл лежит в git и CI ставит зависимости строго по нему.	☐
02	Замените latest во всех образах на конкретные версии.	☐
03	Настройте бота обновлений с автомержем патчей при зелёном прогоне.	☐

РЕЗУЛЬТАТ Проверка: прогон месячной давности на том же коммите даёт тот же результат.

21 Локальное окружение для запуска автотестов

После неё вы настраиваете запуск автотестов на ноутбуке за 15 минут, поднимаете зависимости одной командой и отлаживаете тест прямо в браузере.

21.1 Что нужно для локального запуска
21.2 Кейс «Полки»: от дня к 15 минутам

Если тесты можно запустить только в CI, разработчик узнаёт о падении через 20 минут после пуша и чинит его вслепую, коммит за коммитом. Локальный запуск сокращает цикл до секунд: изменил код — запустил нужный тест — увидел результат. Поэтому простота локального окружения напрямую влияет на то, будут ли разработчики вообще трогать тесты.

21.1 Что нужно для локального запуска

- Одна команда установки: npm ci && npx playwright install.
- Docker Compose с зависимостями: базы, очереди, WireMock для внешних API.
- Скрипт запуска тестируемого сервиса или переход на общий стенд одной переменной.
- Конфиг local по умолчанию: без переменных тесты идут на localhost.
- Режим отладки: UI-режим Playwright, пошаговое выполнение, трейс.

21.2 Кейс «Полки»: от дня к 15 минутам

Новый автоматизатор в «Полке» раньше тратил на первый запуск тестов около дня: ставил нужную JDK, искал актуальный адрес стенда, просил пароль служебного пользователя. Тимофей собрал devcontainer и Makefile.

```
make up          # docker compose: postgres, kafka, redis, wiremock, 3 сервиса
make seed        # служебный магазин и справочники
make test        # весь набор против localhost
make test T=cart # только тесты с тегом @cart
make ui          # Playwright UI mode: выбор теста, шаги, DOM, сеть
```

Локально поднимаются только три сервиса, нужные для большинства сценариев. Для полного набора тесты одной переменной переключаются на общий стенд. Первый запуск по README у трёх следующих новичков занял от 11 до 18 минут.

ПРАВИЛО

Если тест падает в CI, его должно быть можно повторить локально одной командой с тем же seed и конфигом. Тест, который воспроизводится только в CI, отлаживается в десять раз дольше.

ПЛОХО

Только CI

«Запушь и посмотри, что скажет пайплайн».

Цикл правки — 20 минут, отладка через логи.

ХОРОШО

Локально за секунды

make test T=cart за 40 секунд.

Падение открывается в трейсе с DOM и сетью.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Инструкция на 3 страницы	Новички не запускают тесты неделями	make up && make test
Весь продукт локально	Ноутбук не тянет 14 сервисов	Минимальный набор + стенд
Разные версии у всех	Падения «только у меня»	Devcontainer или Docker-образ

ПРАКТИКА 21 после главы 21

Сократите время первого локального запуска.

ЧТО СДЕЛАТЬ

01	Попросите коллегу запустить тесты с нуля и засеките время.	☐
02	Запишите все вопросы, которые у него возникли.	☐
03	Закройте каждый вопрос скриптом или строкой в README.	☐
04	Добавьте команду запуска по тегу и режим отладки.	☐

РЕЗУЛЬТАТ Цель — не больше 15 минут от клонирования до зелёного прогона.

22 Тестовые стенды и их виды

После неё вы различаете стенды по назначению, выбираете между общими и временными окружениями и понимаете, сколько стоит каждый вариант.

22.1 Виды стендов · 22.2 Временные стенды · 22.3 Кейс «Полки»: стоимость стендов

Стенд — это развёрнутая копия продукта, на которой идут тесты. От того, как устроены стенды, зависит, сколько тестов можно гонять параллельно, насколько они стабильны и как быстро разработчик получает обратную связь. Общий стенд на всех — самый дешёвый и самый проблемный вариант.

22.1 Виды стендов

СТЕНД	НАЗНАЧЕНИЕ	ЖИЗНЬ	ДАННЫЕ
Dev	Разработчики пробуют свои изменения	Постоянный	Любые
Временный (preview)	Тесты одного merge request	Часы	Пустые + сиды
Stage / pre-prod	Приёмка релиза, E2E, ручное исследование	Постоянный	Близкие к продакшену, обезличенные
Нагрузочный	Производительность	По расписанию	Объём как в продакшене
Продакшен	Синтетические проверки	—	Служебные аккаунты

22.2 Временные стенды

Kubernetes и инфраструктура как код позволяют поднимать окружение на каждый merge request: изменённый сервис в новой версии, остальные — из последнего релиза. Тесты не мешают друг другу, а стенд удаляется после мержа. Цена — вычислительные ресурсы и время старта.

22.3 Кейс «Полки»: стоимость стендов

ВАРИАНТ	ЗАТРАТЫ В МЕСЯЦ	ОЖИДАНИЕ СТЕНДА	FLAKY ИЗ-ЗА СТЕНДА
Один общий stage	40 тыс. ₽	до 2 ч в очереди	6%
3 общих стенда по командам	120 тыс. ₽	до 30 мин	3%
Временный стенд на MR	75 тыс. ₽ (при 18 MR в день, 2 ч жизни)	4 мин старт	0,4%

Временные стенды оказались дешевле трёх постоянных: они живут два часа и поднимают только изменённый сервис, остальные берут общими. Нина сохранила один общий stage для ручного исследования и E2E перед релизом.

ВАЖНО · ТИПИЧНАЯ ОШИБКА

Гонять автотесты на том же стенде, где работают ручные тестировщики. Человек меняет настройки магазина для своего сценария — и десяток автотестов падает без дефекта.

ПЛОХО

Очередь на стенд

«Не деплойте на stage, я тестирую».

Стенд в чужой ветке, тесты падают.

ХОРОШО

Стенд на изменение

Каждый MR получает своё окружение.

Stage — только для релиза и исследования.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Стенд отличается от продакшена	Дефекты конфигурации проходят	Та же инфраструктура как код
Нет мониторинга стенда	Тесты падают из-за упавшей базы	Health-check перед прогоном
Временные стенды не удаляются	Счёт за облако растёт	TTL и автоудаление

ПРАКТИКА 22 после главы 22

Оцените, сколько стоят ваши стенды и сколько из-за них теряется.

ЧТО СДЕЛАТЬ

- [ ] **01** Опишите все стенды: назначение, кто пользуется, кто деплоит.
- [ ] **02** Посчитайте ожидание стенда и долю падений из-за него за месяц.
- [ ] **03** Сравните стоимость общих и временных стендов для вашей частоты MR.

РЕЗУЛЬТАТ Учитывайте не только счёт за инфраструктуру, но и часы людей в очереди.

23 Работа с тестовыми базами данных

После неё вы изолируете данные тестов, применяете миграции, готовите обезличенные копии и знаете, как тестировать сами миграции.

23.1 Изоляция · 23.2 Схема и миграции · 23.3 Кейс «Полки»: обезличенная копия

База данных — самая сложная зависимость тестов: у неё есть состояние, которое переживает прогон, схема, которая меняется миграциями, и данные, которые нельзя просто скопировать из продакшена. Хорошая стратегия отвечает на три вопроса: как изолировать тесты, откуда брать схему и откуда — данные.

23.1 Изоляция

СПОСОБ	КАК РАБОТАЕТ	КОГДА
Транзакция с откатом	Тест работает внутри транзакции, в конце — rollback	Интеграционные тесты репозиториев
Очистка таблиц	TRUNCATE изменённых таблиц после теста	Код, который сам управляет транзакциями
База на поток	Каждый воркер получает свою схему или базу	Параллельные прогоны
Шаблонная база	CREATE DATABASE … TEMPLATE за доли секунды	PostgreSQL, много тестов с чистой базой

23.2 Схема и миграции

Тестовая база получает схему тем же способом, что продакшен: миграциями Flyway или Liquibase. Тогда каждый прогон заодно проверяет, что миграции применяются. Отдельно стоит тестировать миграции на копии с реальным объёмом: ALTER TABLE на 40 млн строк может занять полчаса и заблокировать таблицу.

23.3 Кейс «Полки»: обезличенная копия

КАК ГОТОВЯТ ДАННЫЕ ДЛЯ STAGE

1 **Срез**
Каждую неделю — копия 5% заказов и связанных сущностей из продакшена.

2 **Маскирование**
Имена, телефоны, адреса, email заменяются синтетикой с сохранением формата; карты не копируются вовсе.

3 **Проверка**
Скрипт ищет остатки персональных данных по шаблонам; при находке выгрузка останавливается.

✓ **Итог**
Stage с реалистичным распределением данных; проверка миграций на 2 млн заказов.

ПРАВИЛО

Персональные данные клиентов не попадают на тестовые стенды ни в каком виде. Это требование 152-ФЗ, а утечка со стенда случается чаще, чем из продакшена: доступ к нему шире, а защита слабее.

ПЛОХО

Дамп продакшена

«Залили вчерашний бэкап на stage».

Телефоны клиентов доступны всем подрядчикам.

ХОРОШО

Маскированный срез

Структура и распределение как в продакшене.

Ни одного реального имени и телефона.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Схема создаётся отдельным скриптом	Расходится с миграциями	Только миграции
Тесты зависят от данных дампа	Падают после обновления дампа	Фабрики поверх дампа

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Миграции не проверяются на объёме	Блокировки в продакшене	Прогон на срезе перед релизом

ПРАКТИКА 23 после главы 23

Выберите способ изоляции и проверьте, откуда у вас берутся данные.

ЧТО СДЕЛАТЬ

- [ ] 01 Опишите, как сейчас изолированы тесты, работающие с базой.
- [ ] 02 Выберите способ из таблицы для каждого типа тестов.
- [ ] 03 Проверьте стенды на наличие реальных персональных данных.

РЕЗУЛЬТАТ Если персональные данные нашлись — это задача первого приоритета, раньше любых тестов.

24 Моки, стабы, фейки

После неё вы различаете виды тестовых двойников, выбираете нужный под задачу и не превращаете тест в проверку собственных моков.

24.1 Что и когда подменять

24.2 Кейс «Полки»: WireMock для платёжного шлюза

Тестовый двойник — объект, который подменяет настоящую зависимость в тесте. Его используют, когда настоящая зависимость медленная, недетерминированная, платная или ещё не готова. Слово «мок» в обиходе означает любой двойник, но различать виды полезно: от вида зависит, что именно проверяет тест.

КЛАССИКА — Джерард Месарош, «xUnit Test Patterns», 2007

Тестовые двойники

Месарош ввёл термин test double по аналогии с дублёром каскадёра и описал пять видов: dummy, stub, spy, mock и fake. Классификацию популяризировал Мартин Фаулер в статье «Mocks Aren't Stubs».

Ключевое различие: стаб помогает тесту дойти до проверки, а мок сам является проверкой — тест падает, если ожидаемый вызов не случился.

ВИД	ЧТО ДЕЛАЕТ	ПРИМЕР В «ПОЛКЕ»
Dummy	Заполняет параметр, не используется	Пустой логгер в конструкторе
Stub	Возвращает заранее заданный ответ	Сервис погоды всегда отвечает «ясно»
Spy	Записывает вызовы для последующей проверки	Отправитель SMS запоминает сообщения
Mock	Заранее знает ожидаемые вызовы и падает при расхождении	verify: платёжный клиент вызван ровно раз
Fake	Рабочая упрощённая реализация	In-memory репозиторий, WireMock со сценарием

24.1 Что и когда подменять

- Подменяйте внешний мир: платёжный шлюз, SMS, карты, сторонние API.
- Не подменяйте то, чем владеете и что можно поднять: свою базу, свою очередь — для них есть Testcontainers.
- Моки используйте для исходящих команд с побочным эффектом (отправить SMS, списать деньги), стабы — для входящих запросов данных.

24.2 Кейс «Полки»: WireMock для платёжного шлюза

Sandbox платёжного провайдера отвечал по 3–8 секунд и раз в день падал на час. Тимофей сделал фейк на WireMock со сценариями: успешная оплата, отказ банка, таймаут, повторный callback. Интеграционные и API-тесты ходят в фейк, а E2E перед релизом — в настоящий sandbox. Фейк проверяется контрактом против спецификации провайдера, чтобы не разойтись с реальностью.

ПЛОХО

Мокаем всё подряд

Мок репозитория, мок маппера, мок валидатора.

Тест проверяет, что моки вызвали друг друга.

ХОРОШО

Подменяем внешний мир

Настоящая логика и база в контейнере.

Фейк только для платежей и SMS.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Мок расходится с реальностью	Тесты зелёные, продакшен падает	Контрактные проверки фейков
verify на запросы данных	Хрупкость при рефакторинге	Стабы для запросов, моки для команд
Мок своего же кода	Нет проверки интеграции	Testcontainers

ПРАКТИКА 24 после главы 24

Проведите ревизию двойников в своих тестах.

ЧТО СДЕЛАТЬ

01	Найдите все места, где подменяются зависимости.	☐
02	Для каждого определите вид двойника и подменяемую зависимость.	☐
03	Уберите моки собственных компонентов, которые можно использовать вживую.	☐
04	Для внешних фейков добавьте сверку со спецификацией провайдера.	☐

РЕЗУЛЬТАТ Хороший знак — моков становится меньше, а тесты ловят больше.

25 Управление тестовыми пользователями

После неё вы создаёте пользователей под каждый тест, храните служебные учётки безопасно и покрываете роли и права без ручной настройки аккаунтов.

25.1 Три типа пользователей
25.2 Роли и права
25.3 Кейс «Полки»: обход SMS-авторизации

Тестовый пользователь — тоже тестовые данные, но с особенностями: у него есть пароль или токен, роль, права, иногда — двухфакторная авторизация и привязанная карта. Если все тесты логинятся одним аккаунтом, они начинают мешать друг другу: один меняет адрес по умолчанию, другой очищает корзину, третий блокирует аккаунт неверным паролем.

25.1 Три типа пользователей

ТИП	КАК СОЗДАЁТСЯ	ЖИЗНЬ	ПРИМЕР
Одноразовый	Фабрика в начале теста	Один тест	Покупатель для сценария оплаты
Пул	Заранее, N штук, выдаются воркерам	Прогон	Курьеры с привязкой к магазину
Служебный	Вручную, хранится в секретах	Постоянно	Синтетика в продакшене, админ стенда

25.2 Роли и права

В «Полке» пять ролей: покупатель, сборщик, курьер, менеджер магазина, оператор поддержки. Проверка прав — матрица «роль × действие», и её удобно генерировать параметризованным тестом: каждой ячейке соответствует ожидаемый статус ответа.

```
const matrix = [
 ['customer', 'GET /orders/:own',     200],
 ['customer', 'GET /orders/:other',   404],
 ['courier',  'PATCH /orders/:id/status', 200],
 ['picker',   'POST /refunds',        403],
 ['support',  'POST /refunds',        201],
];
for (const [role, call, status] of matrix)
 test(`${role} ${call} → ${status}`, async ({ actor }) ⇒ { … });
```

25.3 Кейс «Полки»: обход SMS-авторизации

Покупатели входят по коду из SMS. Ждать настоящее SMS в тесте нельзя. На тестовых окружениях сервис авторизации принимает для номеров из служебного диапазона +7 900 000-xx-xx фиксированный код, а в продакшене этот диапазон отключён на уровне конфигурации и проверяется отдельным тестом безопасности.

ВАЖНО · ТИПИЧНАЯ ОШИБКА

Добавлять «бэкдор» для тестов, который включён и в продакшене: магический заголовок, отключающий авторизацию, или общий мастер-код. Такие лазейки рано или поздно находят снаружи. Тестовые обходы должны быть выключены конфигурацией продакшена и проверены тестом.

ПЛОХО

Один аккаунт на всех

qa@polka.ru / Qwerty123 в 200 тестах.

Пароль в README, аккаунт периодически блокируется.

ХОРОШО

Пользователь на тест

Фабрика создаёт покупателя с нужной ролью.

Служебные учётки — в хранилище секретов, с ротацией.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Логин через UI в каждом тесте	Медленно, лишний источник падений	Токен через API, сохранённая сессия
Права проверяются только позитивно	Утечки доступа	Матрица ролей с запретами
Тестовые обходы в продакшене	Уязвимость	Отключение конфигом + тест

ПРАКТИКА 25 после главы 25

Соберите матрицу ролей и прав и покройте её тестами.

ЧТО СДЕЛАТЬ

01	Выпишите роли и ключевые действия продукта.	☐
02	Заполните ожидаемый результат для каждой пары.	☐
03	Напишите параметризованный тест по матрице.	☐
04	Проверьте, что тестовые обходы авторизации отключены в продакшене.	☐

РЕЗУЛЬТАТ Матрица прав — один из самых дешёвых и полезных наборов API-тестов.

26 Создание проекта с нуля

После неё вы за один день собираете каркас проекта автотестов: структуру, конфиг, линтер, первый тест, отчёт и запуск в CI.

26.1 План первого дня · 26.2 Минимальный конфиг · 26.3 Кейс «Полки»

Первый день проекта автотестов определяет его следующие годы. Если сразу заложить структуру, конфиг окружений, линтер и запуск в CI, каждый новый тест ложится в готовый порядок. Если начать с одного файла «пока попробовать», через месяц в нём будет 60 тестов, и переделывать придётся на ходу.

26.1 План первого дня

КАРКАС ЗА ДЕНЬ

1. **Инициализация**
   npm init playwright@latest; TypeScript, папка tests, конфиг с проектами chromium и api.
2. **Структура**
   Папки api, fixtures, pages, tests, config, utils по главе 15.
3. **Конфиг окружений**
   Типизированный объект, ENV=local по умолчанию, чтение секретов из переменных.
4. **Качество кода**
   ESLint с правилами для Playwright, Prettier, проверка типов в pre-commit.
5. **Первый тест**
   Smoke: health-check сервиса и открытие главной страницы.
6. **CI**
   Стадия в .gitlab-ci.yml в официальном Docker-образе, отчёт как артефакт.

КАРКАС ЗА ДЕНЬ

✓ **README**

Запуск локально, на стенде, по тегу; где смотреть отчёт.

26.2 Минимальный конфиг

```
export default defineConfig({
 testDir: './tests',
 fullyParallel: true,
 retries: process.env.CI ? 1 : 0,
 reporter: [['list'], ['html'], ['allure-playwright']],
 use: { baseURL: env.baseUrl, trace: 'retain-on-failure' },
 projects: [
  { name: 'api', testMatch: /api\/.*\.spec\.ts/ },
  { name: 'ui',  testMatch: /ui\/.*\.spec\.ts/, use: devices['Desktop Chrome'] },
 ],
});
```

26.3 Кейс «Полки»

Тимофей собрал каркас polka-tests за один день и первым делом довёл до зелёного состояния CI с двумя smoke-тестами. Только после этого команда начала переносить сценарии. За первую неделю в проект пришло 40 тестов от четырёх авторов, и ни один не потребовал переделки структуры.

ПРАВИЛО

Подключайте CI в первый же день, когда тест ещё один. Тест, который не запускается автоматически, не автотест, а скрипт, который однажды кто-то запустил.

ПЛОХО

Сначала тесты, потом порядок

«Напишем 50 тестов, потом разложим по папкам».

Через месяц — рефакторинг на неделю.

ХОРОШО

Сначала каркас

Структура, конфиг, CI и один зелёный тест.

Каждый следующий тест ложится в готовое место.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Нет линтера	Разный стиль, пропущенные await	ESLint с правилами фреймворка
Ретраи по умолчанию 3	Маскируют нестабильность	Не больше 1 и только в CI
Проект без владельца	Никто не отвечает за каркас	Назначить мейнтейнера

ПРАКТИКА 26 после главы 26

Соберите каркас нового проекта автотестов или проверьте существующий по плану первого дня.

ЧТО СДЕЛАТЬ

01	Пройдите семь шагов плана и отметьте, что уже есть.	☐
02	Добавьте недостающее, начиная с CI и конфига окружений.	☐
03	Попросите коллегу добавить тест по README без вашей помощи.	☐

РЕЗУЛЬТАТ Каркас готов, когда новый тест добавляется без вопросов автору проекта.

27 Первый unit-тест

После неё вы пишете первый unit-тест на реальную функцию, выбираете, что проверять, и строите вокруг него привычку писать тесты вместе с кодом.

27.1 Выбор функции
27.2 Кейс «Полки»: форматирование веса
27.3 Красный, зелёный, рефакторинг

Первый unit-тест лучше писать не на «hello world», а на функцию, которая реально может сломаться: расчёт, валидацию, преобразование. Тогда тест сразу приносит пользу, и его ценность видно команде. Хороший кандидат — функция без зависимостей от базы и сети, с понятным входом и выходом.

27.1 Выбор функции

- Есть ветвления и граничные значения.
- Из неё приходили дефекты или её боятся трогать.
- Чистая функция: результат зависит только от аргументов.
- Её поведение можно описать таблицей «вход → выход».

27.2 Кейс «Полки»: форматирование веса

Фронтендер Лиза выбрала функцию formatWeight: она превращает граммы в подпись на карточке товара. В продакшене однажды показывалось «1000 г» вместо «1 кг» и «0,5 кг» вместо «500 г». Тест на Vitest занял 15 минут.

```
import { describe, it, expect } from 'vitest';
import { formatWeight } from './formatWeight';

describe('formatWeight', () ⇒ {
 it.each([
  [450, '450 г'],
  [999, '999 г'],
  [1000, '1 кг'],
  [1250, '1,25 кг'],
  [0, '—'],
 ])('%i g → %s', (grams, expected) ⇒ {
  expect(formatWeight(grams)).toBe(expected);
 });
 it('throws on negative weight', () ⇒ {
  expect(() ⇒ formatWeight(-1)).toThrow(RangeError);
 });
});
```

Два кейса из шести упали сразу: функция возвращала «1000 г» и «1.25 кг» с точкой. Исправление заняло пять минут. Важнее было другое: Лиза увидела, что тест нашёл ошибку быстрее, чем она дошла бы до неё в браузере.

27.3 Красный, зелёный, рефакторинг

Убедитесь, что новый тест умеет падать. Временно сломайте функцию или ожидание: если тест остался зелёным, он ничего не проверяет. В TDD этот шаг встроен в цикл: сначала падающий тест, потом минимальный код, потом улучшение кода под защитой теста.

ПЛОХО

Тест ради теста

it('works', () => { formatWeight(500) })

Нет проверки результата — тест всегда зелёный.

ХОРОШО

Тест на поведение

Таблица вход → выход, граница 999/1000, ноль, ошибка.

Название говорит, что сломалось.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Не видеть красный	Тест может ничего не проверять	Сломать код и убедиться, что тест упал
Копировать логику в тест	Тест повторяет ошибку кода	Ожидание — конкретное значение

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Первый тест на сложный сервис	Моки, настройка, разочарование	Начать с чистой функции

ПРАКТИКА 27 после главы 27

Напишите первый unit-тест на функцию, которую вы боитесь трогать.

ЧТО СДЕЛАТЬ

01	Выберите функцию по четырём признакам.	☐
02	Составьте таблицу «вход → выход» с границами и ошибками.	☐
03	Напишите параметризованный тест и запустите.	☐
04	Сломайте функцию и убедитесь, что тест упал.	☐

РЕЗУЛЬТАТ Добавьте тест в CI — с этого момента функция защищена.

28 Первый API-тест

После неё вы пишете первый API-тест с подготовкой данных, проверкой статуса, схемы и полей, а затем превращаете его в основу для клиента API.

28.1 Анатомия | 28.2 Что проверять | 28.3 Кейс «Полки»: из теста в клиент

Первый API-тест стоит писать на сценарий, который важен бизнесу и затрагивает несколько полей ответа: создание сущности, расчёт, смену статуса. Тест на GET /health полезен для smoke, но ничему не учит. В «Полке» первым стал тест на добавление товара в корзину и пересчёт суммы.

28.1 Анатомия

```
test('adds item and recalculates cart total', async ({ request }) => {
 const token = await loginAsNewCustomer(request);
 const headers = { Authorization: `Bearer ${token}` };

 const res = await request.post('/baskets/v2/items', {
  headers, data: { store: 7, sku: 'milk-1l', qty: 2 },
 });

 expect(res.status()).toBe(200);
 const body = await res.json();
 expect(body).toMatchSchema(basketSchema);
 expect(body.items).toHaveLength(1);
 expect(body.total).toBe(2 * 89);
});
```

28.2 Что проверять

ПРОВЕРКА	ЗАЧЕМ	ПРИМЕР
Статус	Сервис понял запрос так, как мы ждали	200, 201, 422
Схема	Структура не сломалась: поля, типы, обязательность	toMatchSchema(basketSchema)

ПРОВЕРКА	ЗАЧЕМ	ПРИМЕР
Ключевые поля	Бизнес-результат верен	total = 178
Побочный эффект	Изменение действительно сохранилось	GET корзины показывает товар

28.3 Кейс «Полки»: из теста в клиент

После третьего теста Тимофей заметил, что в каждом повторяются логин, заголовки и путь. Он не стал копировать дальше, а вынес их: сначала в фикстуру customer с готовым токеном, потом в клиент, сгенерированный из OpenAPI. Через неделю тот же тест выглядел в три строки: customer.basket.add(milk, 2) и две проверки.

ПРАВИЛО

Пишите первые два-три теста «в лоб», без абстракций. Повторения покажут, какие абстракции нужны на самом деле. Каркас, придуманный до первого теста, обычно угадывает не то.

ПЛОХО

Проверка статуса

expect(res.ok()).toBeTruthy()

200 с пустой корзиной тоже пройдёт.

ХОРОШО

Проверка результата

Статус, схема, состав корзины, сумма.

Повторный GET подтверждает сохранение.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Ожидание из ответа сервиса	Тест сравнивает ошибку с ошибкой	Ожидание считать из входных данных
Тест зависит от цены в каталоге	Падает при смене цены	Создать товар фабрикой с известной ценой
Нет проверки схемы	Лишние или пропавшие поля незаметны	Схема из OpenAPI

ПРАКТИКА 28 после главы 28

Напишите первый API-тест на важный бизнес-сценарий.

ЧТО СДЕЛАТЬ

01	Выберите эндпоинт, который создаёт или меняет сущность.	☐
02	Подготовьте данные фабрикой с известными значениями.	☐
03	Проверьте статус, схему, ключевые поля и сохранение.	☐
04	После третьего теста вынесите повторения в фикстуру или клиент.	☐

РЕЗУЛЬТАТ Сравните время прогона с аналогичной ручной проверкой через Postman.

29 Первый UI-тест

После неё вы пишете первый UI-тест на стабильных локаторах, готовите состояние через API и проверяете то, что видит пользователь, а не внутренности страницы.

29.1 Локаторы по приоритету | 29.2 Кейс «Полки»: оформление заказа | 29.3 Автоожидания

Первый UI-тест — визитная карточка автоматизации: его показывают команде, и по нему судят, стоит ли этим заниматься. Если он падает через раз, доверие теряется надолго. Поэтому первый тест должен быть коротким, стабильным и проверять то, что действительно важно пользователю.

29.1 Локаторы по приоритету

ПРИОРИТЕТ	ЛОКАТОР	ПОЧЕМУ
1	getByRole('button', { name: 'Оформить' })	Так видит страницу пользователь и экранный диктор
2	getByLabel('Телефон'), getByPlaceholder	Привязка к подписи поля
3	getByTestId('cart-total')	Стабильный атрибут, договорённость с фронтендом
4	getByText('Корзина пуста')	Подходит для проверки, хуже для действий
5	CSS и XPath	Только если иначе нельзя

29.2 Кейс «Полки»: оформление заказа

```
test('customer places order from cart', async ({ page, customer }) => {
 await customer.basket.add('milk-1l', 2);          // подготовка через API
 await page.goto('/cart');

 await page.getByRole('button', { name: 'Выбрать время' }).click();
 await page.getByTestId('slot').first().click();
 await page.getByRole('button', { name: 'Оформить заказ' }).click();

 await expect(page.getByRole('heading', { name: 'Заказ принят' })).toBeVisible();
 await expect(page.getByTestId('order-total')).toHaveText('178 ₽');
});
```

Пользователь уже авторизован: фикстура customer создала его через API и подложила сессию в браузер. Корзина наполнена через API. В интерфейсе проходят только три шага, которые и проверяются. Тест идёт 9 секунд; его предшественник на Selenium шёл 52.

29.3 Автоожидания

Playwright перед каждым действием ждёт, пока элемент появится, станет видимым, перестанет двигаться и сможет принять клик. Проверки через expect тоже повторяются до таймаута. Поэтому в тесте нет ни одной паузы, и он одинаково проходит на быстром ноутбуке и на загруженном CI.

ПРАВИЛО

Договоритесь с фронтендом о data-testid для элементов, у которых нет доступной роли и подписи. Атрибут — часть контракта: его не удаляют при рефакторинге без согласования с тестами.

ПЛОХО

Тест повторяет ручной кейс

Логин через форму, поиск, каталог, корзина, оформление.

40 шагов, 90 секунд, падает на любом из них.

ХОРОШО

Тест проверяет одно

Подготовка через API, в UI — три шага.

9 секунд, понятно, что сломалось.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
waitForTimeout	Медленно и нестабильно	Автоожидания и expect
Проверка CSS-классов	Ломается при рестайлинге	Проверять видимое: текст, роль, состояние
Первый тест — самый сложный путь	Нестабильность подрывает доверие	Короткий ключевой сценарий

ПРАКТИКА 29 после главы 29

Напишите первый UI-тест на ключевое действие пользователя.

ЧТО СДЕЛАТЬ

01	Выберите одно действие, которое приносит ценность: оформить, отправить, оплатить.	☐
02	Подготовьте всё остальное через API.	☐
03	Используйте локаторы по ролям и подписям, data-testid — где нужно.	☐
04	Прогоните тест 50 раз подряд и посчитайте падения.	☐

РЕЗУЛЬТАТ Тест готов к команде, если 50 из 50 прогонов зелёные.

30 Структура теста: arrange, act, assert

После неё вы строите каждый тест из трёх явных частей, держите в нём одно действие и одну причину для падения и пишете названия, по которым понятно, что сломалось.

30.1 Кейс «Полки»: до и после · 30.2 Правила

Любой хороший тест состоит из трёх частей. Arrange — подготовка: данные, состояние, зависимости. Act — одно действие, которое проверяем. Assert — проверка результата. Когда части видны, тест читается как спецификация: «если дано вот это и сделать вот это, получится вот это».

КЛАССИКА — Билл Уэйк, 2001; Дэн Норт, BDD, 2006

Arrange-Act-Assert и Given-When-Then

Билл Уэйк описал шаблон 3A как способ сделать тесты единообразными. Дэн Норт в BDD предложил ту же структуру словами Given-When-Then, чтобы сценарии могли читать аналитики и заказчики.

Смысл один: тест — это пример поведения, а у примера есть условие, событие и результат.

30.1 Кейс «Полки»: до и после

```
test('promo PROMO10 gives 10% for cart over 1 500 ₽', async ({ customer }) => {
 // arrange
 const promo = await factory.promo({ code: 'PROMO10', percent: 10, min: 1500 });
 await customer.basket.add(factory.sku({ price: 1600 }));

 // act
 const basket = await customer.basket.applyPromo(promo.code);

 // assert
 expect(basket.discount).toBe(160);
 expect(basket.total).toBe(1440);
});
```

30.2 Правила

Одно действие	Если в act два действия, это два теста. Исключение — E2E, где проверяется цепочка, но и там каждый шаг явный.
Проверки об одном	Несколько assert допустимы, если они описывают один результат: скидку и итог.
Без логики в тесте	Никаких if и циклов в теле теста: тест с ветвлениями сам нуждается в тесте.
Название — спецификация	Что происходит при каком условии: «promo gives 10% for cart over 1 500».

В старом наборе «Полки» был тест testPromo длиной 140 строк: шесть промокодов, по очереди применённых к одной корзине, с проверками между ними. Он падал на третьем промокоде, и никто не знал, работают ли остальные три. Тимофей разделил его на шесть тестов по 12 строк — четыре оказались зелёными, два нашли реальные дефекты.

ПЛОХО

Названия-номера

test('promo test 3')

test('check order')

ХОРОШО

Названия-спецификации

test('rejects expired promo with PROMO_EXPIRED')

test('refunds payment when order cancelled before assembly')

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Тест-сценарий на 100 строк	Падение скрывает остальное	Разбить по действиям
if в тесте	Тест проверяет разное при разных данных	Отдельные тесты или параметризация
Огромный arrange	Неясно, что важно	Фабрики с умолчаниями

ПРАКТИКА 30 после главы 30

Перепишите самый длинный тест своего набора по структуре AAA.

ЧТО СДЕЛАТЬ

01	Найдите тест с несколькими действиями и проверками между ними.	☐
02	Разбейте его на тесты с одним действием.	☐
03	Вынесите подготовку в фабрики и оставьте в arrange только важное.	☐
04	Дайте каждому тесту название-спецификацию.	☐

РЕЗУЛЬТАТ Проверка: по списку названий тестов можно восстановить требования к функции.

31 Чтение и интерпретация результатов

После неё вы читаете вывод тестового прогона, находите причину падения по сообщению, трейсу и логам и отличаете падение проверки от ошибки подготовки.

31.1 Что смотреть при падении
31.2 Хорошее сообщение об ошибке
31.3 Кейс «Полки»: время на разбор

Красный тест — это вопрос, а не ответ. Чтобы понять, что произошло, нужно прочитать сообщение об ошибке, место падения и контекст: что тест успел сделать, что вернул сервис, как выглядела страница. Хороший набор тестов отдаёт всё это сразу, без перезапуска.

31.1 Что смотреть при падении

ПОРЯДОК РАЗБОРА

1. **Где упало**
   Arrange, act или assert? Падение в подготовке — чаще проблема данных или стенда, в проверке — чаще дефект.
2. **Сообщение**
   Ожидалось vs получено. Сравнение объектов показывает, какое поле разошлось.
3. **Контекст**
   Трейс Playwright: шаги, DOM, сеть, консоль. Для API — запрос и ответ целиком.
4. **История**
   Падал ли тест раньше? Падает ли на других ветках?
5. ✓ **Вывод**
   Дефект продукта, дефект теста, данные, окружение — и кто чинит.

31.2 Хорошее сообщение об ошибке

```
✗ promo PROMO10 gives 10% for cart over 1 500 ₽  (1.2s)

 Error: expect(received).toBe(expected)
 Expected: 160
 Received: 0

 at tests/api/cart/cart.promo.spec.ts:14:29   expect(basket.discount).toBe(160)
 request:  POST /baskets/v2/promo  {"code":"PROMO10"}
 response: 200 {"discount":0,"reason":"MIN_NOT_REACHED","subtotal":1499.99}
```

По одному этому выводу видно всё: тест ждал скидку 160, сервис её не дал и объяснил почему — сумма корзины 1 499,99 вместо 1 600. Значит, товар создан не с той ценой, и это дефект подготовки теста, а не промокода. Без запроса и ответа в отчёте пришлось бы перезапускать тест с отладкой.

31.3 Кейс «Полки»: время на разбор

ЧТО ДОБАВИЛИ В ОТЧЁТ	ВРЕМЯ РАЗБОРА ПАДЕНИЯ
Только стек-трейс (было)	~25 мин
+ запрос и ответ API	~10 мин
+ трейс Playwright для UI	~6 мин
+ ссылка на логи сервиса по trace-id	~4 мин

ПРАВИЛО

Тест должен падать так, чтобы причина была видна без перезапуска. Каждая минута, которую разработчик тратит на воспроизведение, снижает шанс, что он вообще будет разбираться.

ПЛОХО

Немое падение

AssertionError: expected true to be false

Неясно, что проверялось и что пришло.

ХОРОШО

Говорящее падение

Ожидалось 160, получено 0, ответ сервиса рядом.

Причина видна за минуту.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
expect(a === b).toBe(true)	Сообщение без значений	Сравнивать значения напрямую
Перезапуск до зелёного	Дефект остаётся	Разобрать первое падение
Нет связи с логами сервиса	Причина на сервере не видна	trace-id в запросах и отчёте

ПРАКТИКА 31 после главы 31

Улучшите диагностику падений в своём наборе.

ЧТО СДЕЛАТЬ

01	Возьмите три последних падения и засеките время разбора каждого.	☐
02	Запишите, чего не хватало: запроса, ответа, скриншота, логов.	☐
03	Добавьте это в отчёт автоматически.	☐
04	Замените проверки вида toBe(true) на сравнение значений.	☐

РЕЗУЛЬТАТ Целевое время разбора типичного падения — до 5 минут.

32 Архитектура REST и HTTP-основы

После неё вы читаете HTTP-запрос и ответ по частям, понимаете методы, статусы, заголовки и свойства REST, которые определяют, что проверять в API-тестах.

32.1 Запрос и ответ · 32.2 Методы и их свойства · 32.3 Классы статусов · 32.4 Кейс «Полки»: двойной заказ

API-тест проверяет обмен HTTP-сообщениями, поэтому без понимания HTTP тест получается поверхностным: проверяет статус и одно поле. Зная протокол, тестировщик видит больше: правильный ли метод, кэшируется ли ответ, идемпотентна ли операция, корректны ли заголовки.

32.1 Запрос и ответ

```
POST /orders HTTP/1.1
Host: api.polka.ru
Authorization: Bearer eyJhbGciOi…
Content-Type: application/json
Idempotency-Key: 7f3c9a1e-…

{"basketId":"b-912","slotId":"s-44","payment":"card"}

HTTP/1.1 201 Created
Location: /orders/o-5521
Content-Type: application/json

{"id":"o-5521","status":"CREATED","total":1440}
```

32.2 Методы и их свойства

МЕТОД	НАЗНАЧЕНИЕ	БЕЗОПАСНЫЙ	ИДЕМПОТЕНТНЫЙ
GET	Получить ресурс	да	да

МЕТОД	НАЗНАЧЕНИЕ	БЕЗОПАСНЫЙ	ИДЕМПОТЕНТНЫЙ
POST	Создать или выполнить действие	нет	нет
PUT	Заменить ресурс целиком	нет	да
PATCH	Изменить часть ресурса	нет	не обязательно
DELETE	Удалить	нет	да

Безопасный метод не меняет состояние; идемпотентный при повторе даёт тот же результат, что и при одном вызове. Эти свойства — готовые сценарии тестов: повторный DELETE не должен падать с 500, повторный PUT не должен создавать дубль, а POST с тем же Idempotency-Key не должен создавать второй заказ.

32.3 Классы статусов

2xx — успех	200 OK, 201 Created (с Location), 202 Accepted (обработка асинхронная), 204 No Content.
3xx — перенаправление	301, 302, 304 Not Modified при кэшировании.
4xx — ошибка клиента	400 неверный запрос, 401 нет аутентификации, 403 нет прав, 404 нет ресурса, 409 конфликт, 422 нарушено бизнес-правило, 429 лимит.
5xx — ошибка сервера	500, 502, 503, 504. В тестах любой 5xx — дефект, даже если клиент прислал мусор.

КЛАССИКА Рой Филдинг, диссертация, 2000

REST

Филдинг описал REST как набор ограничений архитектуры: клиент-сервер, отсутствие состояния на сервере между запросами, кэшируемость, единый интерфейс через ресурсы и методы, слоистость.

На практике большинство «REST API» выполняют часть ограничений, и для тестирования важнее не чистота стиля, а то, что поведение API описано и согласовано.

32.4 Кейс «Полки»: двойной заказ

При плохом мобильном интернете приложение повторяло POST /orders после таймаута, и клиент получал два заказа. API-тест на идемпотентность — два одинаковых запроса с одним ключом — упал сразу: сервис создал o-5521 и o-5522. После исправления второй запрос возвращает 201 с тем же id.

ПЛОХО

Любая ошибка — 400 или 500

Промокод истёк → 500 Internal Server Error.

Клиент не может понять, что делать.

ХОРОШО

Статус по смыслу

Истёкший промокод → 422 и код PROMO_EXPIRED.

Нет прав → 403, нет ресурса → 404.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
200 с ошибкой в теле	Клиенты и мониторинг не видят ошибок	Тест на статус по смыслу
Не проверять заголовки	Пропуск Location, кэша, CORS	Проверять ключевые заголовки
Игнорировать повторы	Дубли при плохой сети	Тесты идемпотентности

ПРАКТИКА 32 после главы 32

Проверьте API своего продукта на соответствие свойствам HTTP-методов.

ЧТО СДЕЛАТЬ

- [ ] 01 Для пяти эндпоинтов выпишите метод, ожидаемые статусы и идемпотентность.
- [ ] 02 Напишите тест на повтор для каждого идемпотентного вызова.
- [ ] 03 Найдите места, где ошибки отдаются как 200 или 500, и заведите дефекты.

РЕЗУЛЬТАТ Тесты на повторы чаще всего находят дефекты, которые проявляются только у клиентов с плохой связью.

33 Проверка статусов и контрактов

После неё вы проверяете контракт API по спецификации, подключаете consumer-driven контракты между сервисами и ловите несовместимые изменения до деплоя.

33.1 Два подхода
33.2 Consumer-driven контракты
33.3 Кейс «Полки»: переименованное поле

Контракт API — договорённость о том, какие запросы сервис принимает и что на них отвечает: пути, методы, параметры, статусы, структура тела. Когда контракт меняется без предупреждения, потребитель ломается. В микросервисах это главный источник инцидентов: каждый сервис по отдельности исправен, а вместе не работают.

33.1 Два подхода

ПОДХОД	ИСТОЧНИК ПРАВДЫ	КАК ПРОВЕРЯЕТСЯ
Provider-driven	Спецификация OpenAPI сервиса	Ответы сервиса валидируются по схеме; изменения схемы проверяются на обратную совместимость
Consumer-driven	Ожидания потребителей	Каждый потребитель описывает, что ему нужно; провайдер прогоняет все ожидания перед деплоем

33.2 Consumer-driven контракты

В подходе Pact потребитель в своих тестах описывает конкретные запросы и ответы, которые он использует. Из теста получается файл контракта, он публикуется в брокер. Провайдер в своём пайплайне скачивает контракты всех потребителей и проверяет, что отвечает именно так. Если провайдер удалит поле, которым пользуется хотя бы один потребитель, его сборка упадёт.

```
// потребитель: мобильный BFF, тест на Pact
provider.given('order o-1 exists in status ASSEMBLING')
 .uponReceiving('get order status')
 .withRequest({ method: 'GET', path: '/orders/o-1' })
 .willRespondWith({ status: 200, body: {
  id: like('o-1'), status: term({ matcher: 'CREATED|ASSEMBLING|DELIVERING|DONE' }),
  eta: iso8601DateTime() } });
```

33.3 Кейс «Полки»: переименованное поле

Команда сервиса заказов переименовала поле eta в estimatedAt, обновила свой фронтенд и задеплоила. Мобильное приложение показывало «время доставки не известно» два дня, пока не вышел его релиз. После внедрения Pact такое изменение ломает сборку сервиса заказов: в брокере 7 потребителей, и мобильный BFF использует eta.

ВНЕДРЕНИЕ КОНТРАКТОВ

1 **Пилот**
Одна пара: мобильный BFF → сервис заказов, 14 взаимодействий.

2 **Брокер**
Pact Broker и проверка can-i-deploy перед каждым деплоем.

3 **Расширение**
За полгода — 180 контрактов между 14 сервисами и двумя BFF.

✓ **Итог**
Инцидентов из-за несовместимых API: 5 за полгода до → 0 за полгода после.

ПРАВИЛО

Контракт фиксирует то, чем потребитель пользуется, а не всё, что отдаёт провайдер. Иначе любое добавление поля ломает контракт, и команда начинает его ненавидеть.

ПЛОХО

Договорились в чате

«Поле eta больше не нужно? Вроде никто не использует».

Узнаём от пользователей.

ХОРОШО

Договорились в контракте

can-i-deploy показывает, кто использует eta.

Изменение — через период совместимости.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Контракт на весь ответ	Ломается при добавлении полей	Только используемые поля
Контракты без брокера	Провайдер не видит потребителей	Брокер и can-i-deploy
Спецификация отстаёт от кода	Проверка по устаревшей схеме	Генерировать OpenAPI из кода или валидировать в тестах

ПРАКТИКА 33 после главы 33

Найдите самый рискованный стык между сервисами и закройте его контрактом.

ЧТО СДЕЛАТЬ

01	Вспомните инциденты из-за несовместимых изменений API за полгода.	☐
02	Выберите пару потребитель → провайдер, где риск выше.	☐
03	Опишите 5–10 взаимодействий потребителя и проверьте их у провайдера.	☐
04	Добавьте проверку перед деплоем провайдера.	☐

РЕЗУЛЬТАТ Пилот на одной паре занимает неделю и сразу показывает, стоит ли масштабировать.

34 Валидация схем и данных

После неё вы проверяете ответы API по JSON Schema, отличаете проверку структуры от проверки смысла и ловите ошибки в типах, форматах и согласованности данных.

34.1 JSON Schema · 34.2 Проверки данных · 34.3 Кейс «Полки»: копейки

Проверка схемы отвечает на вопрос «правильной ли формы ответ»: есть ли обязательные поля, нужного ли они типа, нет ли лишних. Проверка данных отвечает на вопрос «правильные ли в нём значения»: сумма равна сумме позиций, дата доставки позже даты заказа, статус допустим для этого перехода. Нужны обе: схема ловит поломки контракта, данные — поломки логики.

34.1 JSON Schema

```
{
 "type": "object",
 "required": ["id", "status", "total", "items"],
 "additionalProperties": false,
 "properties": {
  "id":     { "type": "string", "pattern": "^o-[0-9]+$" },
  "status": { "enum": ["CREATED", "ASSEMBLING", "DELIVERING", "DONE", "CANCELLED"]
},
  "total":  { "type": "integer", "minimum": 0 },
  "items":  { "type": "array", "minItems": 1, "items": { "$ref": "#/$defs/item" } }
 }
}
```

Схему не пишут вручную, если есть OpenAPI: её берут из спецификации и валидируют ответ библиотекой (Ajv в TypeScript, jsonschema в Python, json-schema-validator на JVM). additionalProperties: false в тестах полезен: он ловит случайно утёкшие поля, например внутренний идентификатор платежа.

34.2 Проверки данных

ПРАВИЛО	ПРОВЕРКА	ЧТО ЛОВИТ
Сумма сходится	total = Σ(price × qty) − discount + delivery	Ошибки округления и скидок
Согласованность дат	deliveredAt ≥ createdAt; слот в рабочих часах магазина	Часовые пояса
Деньги в копейках	total — целое число	Плавающая точка
Формат	Телефон E.164, даты ISO 8601 с зоной	Неверная сериализация
Ссылочная целостность	storeId заказа существует и активен	Сироты после удалений

34.3 Кейс «Полки»: копейки

Сервис заказов отдавал total как число с плавающей точкой, и в 0,3% заказов итог отличался от суммы позиций на копейку: 1 439,99 вместо 1 440. Проверка «total = Σ позиций» в API-тестах на 200 сгенерированных корзинах нашла 4 расхождения за первый прогон. Деньги перевели в целые копейки, а в схему добавили type: integer.

ПРАВИЛО

Схема — первая проверка, а не единственная. Ответ может идеально соответствовать схеме и при этом содержать неправильную сумму.

ПЛОХО

Проверка полей по одному

expect(body.id).toBeDefined()

expect(typeof body.total).toBe('number')

Двадцать строк, а лишнее поле не заметно.

ХОРОШО

Схема + смысл

expect(body).toMatchSchema(orderSchema)

expect(body.total).toBe(sum(body.items) - body.discount)

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Схема вручную	Расходится со спецификацией	Генерировать из OpenAPI
Снимок всего ответа	Падает при любом изменении	Схема + ключевые поля

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Только структура	Логика не проверена	Инварианты данных

ПРАКТИКА 34 после главы 34

Добавьте проверку схемы и инвариантов к своим API-тестам.

ЧТО СДЕЛАТЬ

01	Подключите валидацию ответов по схемам из OpenAPI.	☐
02	Выпишите 3–5 инвариантов данных ключевой сущности.	☐
03	Проверьте инварианты на сотне сгенерированных входов.	☐

РЕЗУЛЬТАТ Инварианты на сгенерированных данных часто находят то, что не найдут руками придуманные примеры.

35 Авторизация и работа с токенами

После неё вы получаете токены в тестах без UI, проверяете аутентификацию и авторизацию как отдельные риски и тестируете истечение, отзыв и подделку токенов.

35.1 Механизмы | 35.2 Кейс «Полки»: фикстура актёров | 35.3 Сценарии безопасности

Аутентификация отвечает на вопрос «кто это», авторизация — «что ему можно». Ошибки в первой пускают чужих, ошибки во второй дают своим лишнее. API-тесты должны проверять обе, а для этого — уметь быстро получать токены любых ролей.

35.1 Механизмы

Сессия и cookie	Сервер хранит сессию, браузер — идентификатор. В тестах — сохранить cookie после логина.
OAuth 2.0 / OpenID Connect	Токен выдаёт сервер авторизации. Для тестов — client credentials или password grant на тестовом клиенте.
JWT	Подписанный токен с полями: sub, roles, exp. Сервис проверяет подпись и срок без обращения к базе.
API-ключ	Статический ключ сервиса или партнёра. Тесты — на отзыв и лимиты.

35.2 Кейс «Полки»: фикстура актёров

```
export const test = base.extend<{ actor: ActorFactory }>({
 actor: async ({ request }, use) => {
  const cache = new Map<string, string>();
  await use(async (role, opts = {}) => {
   const key = role + JSON.stringify(opts);
   if (!cache.has(key)) cache.set(key, await auth.tokenFor(role, opts));
   return api.client(request, cache.get(key)!);
  });
 },
});
```

Токен получается через API сервера авторизации за 80 мс и кэшируется на время воркера. В UI-тестах тот же токен подкладывается в хранилище браузера, и страница открывается уже авторизованной.

35.3 Сценарии безопасности

СЦЕНАРИЙ	ОЖИДАНИЕ
Без токена	401
Истёкший токен (exp в прошлом)	401, а не 500
Подпись подделана или alg: none	401
Токен отозван (logout)	401 при следующем запросе
Чужой ресурс со своим токеном	404 или 403, без утечки факта существования
Роль повышена в теле токена без подписи	401

ВАЖНО · ТИПИЧНАЯ ОШИБКА

Проверять права только в интерфейсе: кнопка «Вернуть деньги» скрыта у сборщика — значит, всё в порядке. Скрытая кнопка не защищает эндпоинт. Права проверяются API-тестами на каждой роли.

ПЛОХО

Один долгоживущий токен

Токен админа на год в переменной CI.

Все тесты идут с максимальными правами.

ХОРОШО

Токен под роль

Фикстура выдаёт токен нужной роли на прогон.

Тесты проверяют и разрешения, и запреты.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Все тесты под админом	Ошибки прав не видны	Минимальная роль для сценария
Логин в каждом тесте	Медленно, лимиты сервера авторизации	Кэш токенов на воркер
Токены в логах	Утечка доступа	Маскирование заголовков

ПРАКТИКА 35 после главы 35

Покройте аутентификацию и авторизацию отдельным набором тестов.

ЧТО СДЕЛАТЬ

01	Сделайте фикстуру, которая выдаёт токен любой роли без UI.	☐
02	Напишите шесть сценариев безопасности из таблицы для ключевого эндпоинта.	☐
03	Проверьте, что токены маскируются в логах и отчётах.	☐

РЕЗУЛЬТАТ Сценарий с истёкшим токеном часто возвращает 500 — это частая находка первого прогона.

36 Негативные и пограничные сценарии

После неё вы систематически находите граничные значения и классы эквивалентности, проверяете ошибки ввода и состояния и применяете генерацию данных для поиска неожиданных случаев.

36.1 Классы эквивалентности и границы
36.2 Негативные сценарии по типам
36.3 Кейс «Полки»: генерация входов

Позитивный сценарий проверяет, что продукт делает нужное. Негативный — что продукт правильно отказывает: на неверный ввод, недопустимое состояние, отсутствие прав. Пограничный — что продукт верно ведёт себя на краях допустимого. Большинство дефектов живёт именно здесь, потому что разработчик думал в первую очередь о нормальном пути.

36.1 Классы эквивалентности и границы

Классы эквивалентности делят входы на группы, внутри которых продукт ведёт себя одинаково: достаточно одного представителя на класс. Граничные значения берут на стыках классов: минимум, минимум − шаг, максимум, максимум + шаг.

ПАРАМЕТР	КЛАССЫ	ГРАНИЦЫ ДЛЯ ТЕСТОВ
Количество товара (1–99)	<1 · 1–99 · >99	0, 1, 99, 100
Сумма для бесплатной доставки (≥1 500 ₽)	<1 500 · ≥1 500	1 499,99, 1 500,00
Длина комментария (≤500)	0 · 1–500 · >500	0, 500, 501
Слот доставки	прошлое · сейчас · будущее · за горизонтом	−1 мин, начало текущего, +14 дней, +15 дней

36.2 Негативные сценарии по типам

Неверный формат	Строка вместо числа, лишние поля, пустое тело, невалидный JSON → 400.
Нарушение правила	Промокод истёк, товара нет в наличии, слот занят → 422 с кодом ошибки.
Недопустимое состояние	Отменить доставленный заказ, оплатить отменённый → 409.
Конкуренция	Два изменения одного объекта одновременно → один успех, второй — 409.

36.3 Кейс «Полки»: генерация входов

Тимофей добавил property-based тесты на fast-check для эндпоинта добавления в корзину: генератор создаёт случайные комбинации количества, цены и скидки в допустимых и недопустимых диапазонах, а тест проверяет инварианты — сервис никогда не отвечает 5xx, итог не отрицательный, недопустимые входы дают 400 или 422. За первую ночь найдено 3 дефекта, включая 500 на количестве 2 147 483 648.

ПРАВИЛО

Любой 5xx в ответ на запрос клиента — дефект. Неверный ввод должен давать 4xx с понятным кодом, а не падение сервера.

ПЛОХО

Только «ввести неверные данные»

Один тест с qty = -1.

Остальные границы не проверены.

ХОРОШО

Таблица границ

0, 1, 99, 100 и нечисловое значение.

Плюс генератор на инварианты.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Проверять только статус ошибки	Неясно, какая ошибка	Код и поле ошибки в ответе
Негативные только в UI	Валидация на сервере не проверена	Негативные — на уровне API
Генерация без seed	Падение не повторить	Сохранять seed в отчёт

ПРАКТИКА 36 после главы 36

Составьте таблицу границ для ключевого эндпоинта и покройте её тестами.

ЧТО СДЕЛАТЬ

- [ ] 01 Выпишите параметры и их допустимые диапазоны.
- [ ] 02 Определите классы эквивалентности и граничные значения.
- [ ] 03 Напишите параметризованный тест по таблице.
- [ ] 04 Добавьте генератор на инвариант «никогда не 5xx».

РЕЗУЛЬТАТ Ночной прогон генератора — дешёвый способ находить то, что никто не придумал.

37 Связь API-тестов с UI- и unit-тестами

После неё вы распределяете проверки одного требования по уровням без дублирования и строите матрицу покрытия, где видно, какой уровень за что отвечает.

37.1 Кто за что отвечает · 37.2 Общий код между уровнями · 37.3 Кейс «Полки»: матрица покрытия

Одно требование почти всегда проверяется на нескольких уровнях, но каждый уровень должен проверять своё. Если правило скидки проверено и в unit, и в API, и в UI на одних и тех же данных, при изменении правила придётся править три теста, а уверенности это не добавит.

37.1 Кто за что отвечает

ТРЕБОВАНИЕ: PROMO10 ДАЁТ 10% ОТ 1 500 ₽	UNIT	API	UI
Расчёт скидки на всех комбинациях	23 кейса	—	—
Граница 1 499,99 / 1 500	2 кейса	—	—
Промокод сохраняется в корзине и заказе	—	1	—
Истёкший, чужой, исчерпанный промокод	—	3	—
Поле ввода, сообщение об ошибке, отображение скидки	—	—	1

Unit проверяет логику во всех вариантах. API — что логика подключена к хранилищу, контракту и правилам доступа. UI — что пользователь видит результат. Комбинации не повторяются на верхних уровнях.

37.2 Общий код между уровнями

В «Полке» API и UI-тесты используют одни фабрики и одного клиента API: UI-тест готовит корзину тем же вызовом, что API-тест. Это даёт две выгоды. Подготовка UI-тестов быстрая и стабильная. А когда меняется API, падают сначала API-тесты с понятным сообщением, и

UI-тесты чинятся той же правкой клиента.

37.3 Кейс «Полки»: матрица покрытия

КАК НИНА ИСКАЛА ДУБЛИ

1 **Требования**
Для модуля корзины — 42 требования из базы знаний аналитиков.

2 **Разметка**
Каждый тест получил аннотацию с id требования: @req CART-17.

3 **Матрица**
Скрипт собрал таблицу «требование × уровень» из аннотаций.

✓ **Итог**
9 требований без тестов, 14 — проверены одинаково на 2–3 уровнях; удалено 37 дублей.

ПРАВИЛО

Проверяйте требование на самом низком уровне, где его можно проверить, и на верхнем — только то, что видно только там.

ПЛОХО

Одна проверка трижды

Скидка 10% проверена в unit, API и UI на корзине 1 600 ₽.

Изменение правила — три правки.

ХОРОШО

Своя проверка на уровне

Unit — расчёт, API — сохранение и запреты, UI — отображение.

Изменение правила — одна правка.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Уровни пишут разные люди без связи	Дубли и дыры	Матрица покрытия
Свой клиент API в UI-тестах	Двойная поддержка	Общий клиент и фабрики
Нет связи с требованиями	Непонятно, что не покрыто	Аннотации @req

ПРАКТИКА 37 после главы 37

Постройте матрицу покрытия для одного модуля.

ЧТО СДЕЛАТЬ

01	Выпишите требования модуля.	☐
02	Разметьте тесты всех уровней идентификаторами требований.	☐
03	Соберите матрицу и найдите дыры и дубли.	☐
04	Удалите дубли на верхних уровнях.	☐

РЕЗУЛЬТАТ Обычно первая матрица находит 10–20% непокрытых требований и столько же дублей.

38 Как писать стабильные тесты

После неё вы устраняете основные источники нестабильности ещё на этапе написания: время, асинхронность, порядок, общее состояние и внешние зависимости.

38.1 Пять источников и защита
38.2 Асинхронные процессы
38.3 Кейс «Полки»: проверка стабильности до мержа

Стабильный тест при одном и том же коде всегда даёт один и тот же результат. Нестабильность почти всегда сводится к одному из пяти источников, и каждый можно исключить заранее, если знать, где он прячется.

38.1 Пять источников и защита

ИСТОЧНИК	ПРИМЕР	ЗАЩИТА
Время	Тест падает в полночь или 31-го числа	Фиксированные часы в unit; явное время в данных
Асинхронность	Событие обработано позже проверки	Ожидание состояния с таймаутом (expect.poll)
Порядок	Тест проходит только после другого	Каждый тест готовит своё; случайный порядок в CI
Общее состояние	Два теста меняют один объект	Уникальные данные на тест
Внешний мир	Сторонний API отвечает медленно	Фейки и эмуляторы

38.2 Асинхронные процессы

В «Полке» после оплаты заказ переходит в статус ASSEMBLING через событие в Kafka — обычно за 200 мс, иногда за 3 секунды. Тест, который проверяет статус сразу после оплаты, будет нестабильным. Правильно — опрашивать с таймаутом и понятным

сообщением.

```
await expect.poll(() ⇒ customer.orders.status(order.id), {
 message: 'order should start assembling after payment',
 timeout: 10_000, intervals: [200, 500, 1000],
}).toBe('ASSEMBLING');
```

38.3 Кейс «Полки»: проверка стабильности до мержа

Новый или изменённый тест в «Полке» перед мержем прогоняется 20 раз подряд в случайном порядке с 4 воркерами — это отдельная задача в пайплайне MR, которая запускается только для изменённых файлов. Если хоть один прогон красный, MR не мержится. За полгода эта проверка остановила 31 нестабильный тест до попадания в основной набор.

ПРАВИЛО

Тест, который не прошёл 20 прогонов подряд, не готов. Найти нестабильность до мержа в десятки раз дешевле, чем потом разбирать её в чужих пайплайнах.

ПЛОХО

Надежда на удачу

expect(status).toBe('ASSEMBLING') сразу после оплаты.

Проходит на ноутбуке, падает на загруженном CI.

ХОРОШО

Ожидание с таймаутом

expect.poll с понятным сообщением.

20 прогонов в случайном порядке перед мержем.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
new Date() в ожидании	Падения на границах суток	Время из данных теста
Фиксированный порядок тестов	Скрытые зависимости	Случайный порядок в CI
Бесконечный таймаут	Зависший прогон	Таймаут + понятное сообщение

ПРАКТИКА 38 после главы 38

Проверьте свои тесты на пять источников нестабильности.

ЧТО СДЕЛАТЬ

01	Найдите в тестах обращения к текущему времени.	☐
02	Найдите проверки асинхронных результатов без ожидания.	☐
03	Запустите набор в случайном порядке и сравните результат.	☐
04	Добавьте в MR проверку 20 прогонов для изменённых тестов.	☐

РЕЗУЛЬТАТ Запуск в случайном порядке почти всегда находит пару скрытых зависимостей.

39 Борьба с flaky-тестами

После неё вы измеряете нестабильность, отправляете flaky-тесты в карантин, находите причину по истории и возвращаете их в набор, не давая красному цвету стать нормой.

39.1 Процесс · 39.2 Кейс «Полки»: 14% → 0,6%

Flaky-тест проходит и падает на одном и том же коде. Один такой тест — мелочь. Двадцать — катастрофа: каждый прогон становится лотереей, команда привыкает нажимать «перезапустить», и однажды настоящий дефект проходит под видом очередного случайного падения.

ФОРМУЛА 39.1 · ВЕРОЯТНОСТЬ ЗЕЛЁНОГО ПРОГОНА

$$P_{\text{green}} = (1 - f)^{n} \tag{39.1}$$

где

f — средняя вероятность ложного падения одного теста — доля

n — число тестов в наборе — шт.

Старый UI-набор «Полки»: f = 1,2%, n = 120 → P = 0,988^{120} ≈ 23%. Три прогона из четырёх были красными без дефекта.

39.1 Процесс

ЦИКЛ РАБОТЫ С FLAKY

1. **Обнаружение**
   Тест упал и прошёл на перезапуске без изменений кода — система отчётов помечает его как flaky.

ЦИКЛ РАБОТЫ С FLAKY

2 **Карантин**
Тест переносится в отдельный набор: он запускается, но не блокирует MR.

3 **Задача**
Автоматически заводится задача на владельца области с историей падений.

4 **Причина**
Разбор по трейсам упавших прогонов: время, асинхронность, данные, порядок, стенд.

5 **Возврат**
После исправления — 50 зелёных прогонов, затем возврат в основной набор.

✓ **Срок**
В карантине не больше двух недель; дальше тест удаляется или переписывается.

39.2 Кейс «Полки»: 14% → 0,6%

МЕСЯЦ	FLAKY ТЕСТОВ	ДОЛЯ	ЧТО СДЕЛАЛИ
0	17 из 120	14%	Замер
2	11 из 310	3,5%	Перенос в Playwright, логин через API
4	9 из 1 400	0,6%	Карантин, фабрики данных, временные стенды
12	27 из 4 530	0,6%	Проверка 20 прогонов перед мержем

Доля держится на 0,6% при росте набора в 40 раз, потому что процесс работает постоянно: новые flaky появляются, но не задерживаются дольше двух недель.

ВАЖНО · ТИПИЧНАЯ ОШИБКА

Лечить нестабильность ретраями: retries: 3 в конфиге. Ретрай скрывает симптом, а тест остаётся нестабильным и замедляет прогон. Ретрай допустим один и только как сигнал для пометки flaky, а не как способ получить зелёный цвет.

ПЛОХО

Привычка перезапускать

«Опять упал, перезапусти».

Никто не знает, сколько тестов нестабильны.

ХОРОШО

Измерение и карантин

Дашборд flaky по областям и владельцам.

Карантин со сроком и задача на причину.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
retries: 3	Маскирует проблему	1 ретрай + пометка flaky
Вечный карантин	Тест не проверяет ничего	Срок 2 недели
Flaky — проблема QA	Разработчики не чинят свою асинхронность	Задача на владельца области

ПРАКТИКА 39 после главы 39

Запустите процесс работы с flaky-тестами.

ЧТО СДЕЛАТЬ

01	Посчитайте долю flaky за последний месяц по истории прогонов.	☐
02	Посчитайте по формуле (39.1) вероятность зелёного прогона.	☐
03	Заведите карантинный набор и правило срока.	☐
04	Разберите три самых частых flaky-теста до причины.	☐

РЕЗУЛЬТАТ Целевой ориентир — доля flaky ниже 1% и зелёный прогон с вероятностью выше 90%.

40 Рефакторинг автотестов

После неё вы улучшаете код тестов без потери проверок, находите запахи тестового кода и проводите рефакторинг маленькими безопасными шагами.

40.1 Запахи тестового кода
40.2 Безопасные шаги
40.3 Кейс «Полки»: фикстура вместо копипасты

Рефакторинг тестов опаснее рефакторинга продукта: у продукта есть тесты, которые подтвердят, что поведение не изменилось, а у тестов их нет. Поэтому главное правило — менять только структуру и проверять, что тесты по-прежнему ловят те же ошибки.

40.1 Запахи тестового кода

ЗАПАХ	ПРИЗНАК	РЕФАКТОРИНГ
Дублирование подготовки	Одинаковые 15 строк в начале 30 тестов	Фабрика или фикстура
Таинственный гость	Тест зависит от данных, которых в нём не видно	Явная подготовка
Чрезмерная проверка	Снимок всего ответа ради одного поля	Проверка ключевого поля
Условная логика	if и циклы в теле теста	Параметризация
Тест-гигант	100+ строк, несколько действий	Разделить по действиям
Магические числа	expect(total).toBe(1440) без объяснения	Вычислить из входных данных

40.2 Безопасные шаги

ПРОТОКОЛ РЕФАКТОРИНГА

1. **Зафиксировать**
   Набор зелёный, известно число тестов и время прогона.
2. **Один запах за раз**
   Отдельный MR на одну трансформацию: проще ревьюить и откатывать.
3. **Мутационная проверка**
   Сломать продукт в одном месте и убедиться, что тест по-прежнему падает.
4. ✓ **Сравнить**
   Число тестов то же, время не выросло, покрытие требований прежнее.

40.3 Кейс «Полки»: фикстура вместо копипасты

В 46 API-тестах заказов повторялся блок из 18 строк: создать покупателя, магазин, товар, корзину, выбрать слот. Тимофей вынес его в фикстуру orderReady с параметрами. Объём папки orders сократился с 3 900 до 2 100 строк. Перед мержем он вручную сломал расчёт доставки на стенде: упали те же 12 тестов, что и до рефакторинга.

ПРАВИЛО

Рефакторинг тестов и изменение проверок — разные MR. Если в одном изменении и структура, и ожидания, ревьюер не сможет понять, не ослабла ли проверка.

ПЛОХО

Большой взрыв

«Перепишу весь набор за спринт».

MR на 9 000 строк, ревью формальное.

ХОРОШО

Маленькие шаги

Одна трансформация на MR, 200–400 строк.

Каждый шаг проверен поломкой продукта.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Абстракции «на будущее»	Сложнее понять тест	Выносить после третьего повтора
Рефакторинг без проверки поломкой	Тест мог перестать ловить ошибку	Мутационная проверка
Смешивать с новыми тестами	Ревью невозможно	Разные MR

ПРАКТИКА 40 после главы 40

Найдите и устраните один запах в своём наборе.

ЧТО СДЕЛАТЬ

01	Пройдитесь по таблице запахов и найдите самый частый у вас.	☐
02	Устраните его в одной папке отдельным MR.	☐
03	Проверьте поломкой продукта, что тесты ловят то же, что раньше.	☐

РЕЗУЛЬТАТ Повторяйте раз в спринт — это дешевле, чем раз в год переписывать набор.

41 Обновление тестов при изменении продукта

После неё вы планируете изменения тестов вместе с задачей, меняете их в том же MR, что и продукт, и не узнаёте о сломанных тестах из ночного прогона.

41.1 Тесты в определении готовности
41.2 Кейс «Полки»: новый экран оплаты

Продукт меняется каждый день, и тесты должны меняться вместе с ним. Если тесты обновляют «потом», между изменением и обновлением всегда есть окно, когда набор красный. Команда привыкает, что «эти тесты сейчас сломаны, это нормально», и пропускает настоящие падения.

41.1 Тесты в определении готовности

```
Задача готова, когда:
 код прошёл ревью
 unit и интеграционные тесты изменённой логики — в том же MR
 API, UI и контракты обновлены — в том же MR или связанном, смёрженном раньше
 новые data-testid согласованы с автоматизатором
 пайплайн зелёный, без новых тестов в карантине
```

41.2 Кейс «Полки»: новый экран оплаты

Команда оплаты переделывала экран: вместо выпадающего списка способов — карточки, плюс новый способ «оплата при получении». На планировании Алиса и Тимофей разобрали, какие тесты затронет изменение: 7 UI-тестов, 3 E2E и 2 контракта. В оценку задачи добавили 6 часов на тесты, а data-testid для карточек согласовали ещё на макете.

КАК ИЗМЕНЕНИЕ ПРОШЛО ЧЕРЕЗ ТЕСТЫ

1 **Планирование**
Список затронутых тестов по тегу @payments и матрице покрытия.

КАК ИЗМЕНЕНИЕ ПРОШЛО ЧЕРЕЗ ТЕСТЫ

2 **Разработка**
Компонент PaymentMethods в page objects переписан вместе с экраном.

3 **MR**
Код экрана, обновлённые объекты и 4 новых теста — в одном MR.

✓ **Итог**
Ни одного красного ночного прогона из-за изменения.

ПРАВИЛО

Изменение продукта, которое ломает тесты, не мержится, пока тесты не обновлены. Иначе «временно красный» набор становится постоянным.

ПЛОХО

Тесты догоняют

«Мы выкатили, автоматизаторы поправят».

Неделю регресс красный, настоящие падения тонут.

ХОРОШО

Тесты вместе с кодом

Затронутые тесты известны на планировании.

Обновляются в том же MR.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Тесты не в оценке задачи	Нет времени на обновление	Часы на тесты — часть оценки
Отключить тест «до исправления»	Забыт навсегда	Карантин со сроком
Автоматизатор узнаёт последним	Тесты ломаются неожиданно	Тег области и уведомление владельцу

ПРАКТИКА 41 после главы 41

Встройте обновление тестов в процесс разработки.

ЧТО СДЕЛАТЬ

01	Добавьте тесты в определение готовности задачи.	☐
02	На ближайшем планировании определите затронутые тесты для каждой задачи.	☐
03	Посчитайте, сколько ночных падений за месяц были из-за необновлённых тестов.	☐

РЕЗУЛЬТАТ Цель — ноль падений из-за того, что тесты не успели за продуктом.

42 Управление техническим долгом в автотестах

После неё вы видите долг в автотестах в цифрах, ведёте его реестр, выделяете постоянную долю времени на погашение и объясняете его цену руководству.

42.1 Как измерить долг · 42.2 Реестр долга · 42.3 Кейс «Полки»: 15% времени

Технический долг в тестах копится незаметно: устаревшие тесты, дубли, медленная подготовка, тесты в карантине, старый фреймворк рядом с новым. Каждый пункт по отдельности не страшен. Вместе они превращают набор в то, что команда терпит, а не использует.

42.1 Как измерить долг

ПОКАЗАТЕЛЬ	КАК СЧИТАТЬ	ПОРОГ ТРЕВОГИ
Время прогона	Медиана за неделю, тренд	Рост >10% за месяц
Flaky	Доля тестов с ложными падениями	>1%
Карантин	Тестов в карантине дольше 2 недель	>0
Время на поддержку	Часы автоматизаторов на починку, а не на новые тесты	>30%
Устаревший стек	Тестов на старом фреймворке	Без плана миграции

42.2 Реестр долга

```
ТД-14 · медленная подготовка заказов в E2E
 симптом:  +3,5 мин к каждому E2E-прогону (12 тестов × 18 с)
 цена:     ~25 прогонов в день → 87 мин ожидания в день
 решение:  служебный эндпоинт подготовки заказа в нужном статусе
 оценка:   2 дня · владелец: Тимофей · приоритет: высокий
```

42.3 Кейс «Полки»: 15% времени

Нина договорилась с руководством, что 15% времени автоматизаторов идёт на долг постоянно, а не «когда будет время». Раз в две недели они берут верхний пункт реестра по соотношению цены и оценки. За полгода закрыли 19 пунктов; время ночного прогона снизилось со 104 до 70 минут, хотя тестов стало на 900 больше.

ФОРМУЛА 42.1 · ЦЕНА ДОЛГА В ЧАСАХ

$$D = t \cdot r \cdot k \tag{42.1}$$

где

- t — лишние минуты на прогон — мин
- r — прогонов в месяц — шт.
- k — доля прогонов, которых кто-то ждёт — доля

```
ТД-14: 3,5 мин · 550 прогонов · 0,6 ≈ 1 155 мин ≈ 19 ч ожидания в месяц против 16 ч на исправление
```

ПЛОХО

Долг без цифр

«Тесты медленные, надо бы переписать».

Руководство не видит причины выделять время.

ХОРОШО

Долг в часах

«19 часов ожидания в месяц против 2 дней работы».

Решение принимается за минуту.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Долг «когда будет время»	Время не появляется	Постоянная доля 10–20%
Нет реестра	Долг невидим	Реестр с ценой и оценкой
Переписать всё с нуля	Полгода без новых тестов	Погашать по пунктам

ПРАКТИКА 42 после главы 42

Заведите реестр долга в автотестах.

ЧТО СДЕЛАТЬ

01	Посчитайте пять показателей из таблицы для своего набора.	☐
02	Запишите 5–10 пунктов долга с симптомом, ценой и оценкой.	☐
03	Посчитайте цену верхних пунктов по формуле (42.1).	☐
04	Договоритесь о постоянной доле времени на погашение.	☐

РЕЗУЛЬТАТ Цена в часах ожидания — самый понятный аргумент для руководства.

43 Зачем запускать автотесты в пайплайне

После неё вы объясняете ценность автотестов в CI через скорость обратной связи и стоимость дефекта и связываете их с метриками DORA.

43.1 Цена дефекта по этапам · 43.2 Кейс «Полки»: метрики до и после

Автотест, который запускают вручную, проверяет продукт тогда, когда о нём вспомнили. Автотест в пайплайне проверяет каждое изменение, и разработчик узнаёт о поломке, пока ещё помнит, что менял. Чем раньше найден дефект, тем дешевле его исправить: в MR это минуты, на стенде — часы, в продакшене — инцидент.

43.1 Цена дефекта по этапам

ГДЕ НАЙДЕН	КТО ТРАТИТ ВРЕМЯ	ЦЕНА В «ПОЛКЕ»
В MR, автотест	Автор, 15 минут	≈ 300 ₽
На стенде, ручное тестирование	QA, автор, переключение контекста	≈ 4 000 ₽
В релизном регрессе	QA, автор, перенос релиза	≈ 12 000 ₽
В продакшене	Поддержка, дежурный, автор, компенсации клиентам	≈ 60 000 ₽

КЛАССИКА — Николь Форсгрен, Джез Хамбл, Джин Ким, «Accelerate», 2018

Метрики DORA

Исследование DevOps Research and Assessment выделило четыре метрики, по которым сильные команды отличаются от слабых: частота деплоя, время от коммита до продакшена, доля изменений с инцидентом и время восстановления.

Один из главных выводов: скорость и стабильность не противоречат друг другу. Лучшие команды деплоят чаще и ломают реже, и автоматизированное тестирование входит в число практик, которые это обеспечивают.

43.2 Кейс «Полки»: метрики до и после

МЕТРИКА DORA	ДО	ЧЕРЕЗ ГОД
Частота деплоя	раз в 2 недели	5–8 раз в день
Время от коммита до продакшена	~9 дней	~3 часа
Доля изменений с инцидентом	~21%	~4%
Время восстановления	~5 часов	~40 минут

Время восстановления сократилось не только благодаря тестам: маленькие изменения легко откатить. Но без автоматического регресса маленькие изменения были бы невозможны — каждое пришлось бы ждать три дня ручной проверки.

ПРАВИЛО

Пайплайн — единственный путь кода в продакшен. Если есть способ задеплоить в обход тестов «в срочном случае», им будут пользоваться в каждом срочном случае, то есть постоянно.

ПЛОХО

Тесты по запросу

«Запусти регресс перед релизом».

Дефект найден через неделю после изменения.

ХОРОШО

Тесты на каждое изменение

Каждый MR проходит проверки автоматически.

Дефект найден, пока автор помнит контекст.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Деплой в обход пайплайна	Непроверенный код в продакшене	Один путь для всех
Тесты только перед релизом	Поздняя обратная связь	Запуск на каждом MR
Красный пайплайн можно мержить	Тесты теряют смысл	Блокировка мержа

ПРАКТИКА 43 после главы 43

Посчитайте цену дефекта по этапам и метрики DORA своей команды.

ЧТО СДЕЛАТЬ

01	Оцените цену дефекта на каждом этапе в часах и рублях.	☐
02	Посчитайте четыре метрики DORA за последний квартал.	☐
03	Проверьте, есть ли пути деплоя в обход тестов.	☐

РЕЗУЛЬТАТ Эти цифры — основа разговора о том, какие тесты и где запускать.

44 Интеграция с CI-системами

После неё вы подключаете автотесты к GitLab CI, GitHub Actions или Jenkins, запускаете их в контейнерах, кэшируете зависимости и сохраняете артефакты.

44.1 Кейс «Полки»: задача в GitLab CI
44.2 Что важно в настройке
44.3 Раннеры

CI-система запускает задачи по событиям: пуш, MR, мерж, расписание. Для автотестов важны четыре вещи: одинаковое окружение на каждом запуске, быстрый старт, параллельность и сохранение результатов. Все популярные системы — GitLab CI, GitHub Actions, Jenkins, TeamCity — это умеют; различия в синтаксисе и экосистеме.

44.1 Кейс «Полки»: задача в GitLab CI

```
api-tests:
 stage: test
 image: mcr.microsoft.com/playwright:v1.56.0-noble
 needs: [deploy-preview]
 variables: { ENV: preview, BASE_URL: $PREVIEW_URL }
 cache: { key: { files: [package-lock.json] }, paths: [.npm/] }
 script:
  - npm ci --cache .npm --prefer-offline
  - npx playwright test --project=api --shard=$CI_NODE_INDEX/$CI_NODE_TOTAL
 parallel: 4
 artifacts:
  when: always
  paths: [playwright-report/, allure-results/, test-results/]
  reports: { junit: results.xml }
  expire_in: 14 days
```

44.2 Что важно в настройке

Образ	Официальный образ с браузерами и нужной версией: никаких установок в каждом запуске.

Кэш	Зависимости кэшируются по хэшу lock-файла; при неизменном lock-файле установка занимает секунды.
Артефакты when: always	Отчёт сохраняется и при падении — именно тогда он нужнее всего.
JUnit-отчёт	GitLab и GitHub показывают упавшие тесты прямо в MR.
Секреты	Защищённые и маскированные переменные, доступные только в нужных ветках.

44.3 Раннеры

Общие облачные раннеры удобны, но UI-тестам часто не хватает ресурсов: браузер на 2 vCPU работает медленно, и появляются таймауты. «Полка» перешла на собственные автомасштабируемые раннеры в Kubernetes: 4 vCPU и 8 ГБ на задачу, до 40 задач параллельно, стоимость — около 210 тыс. ₽ в месяц, что входит в 0,9 млн ₽ годового бюджета на инфраструктуру CI.

ПЛОХО

Установка в каждом запуске

apt-get install chromium && npm install

+4 минуты к каждой задаче, версии плавают.

ХОРОШО

Готовый образ и кэш

Образ Playwright нужной версии, npm ci из кэша.

Старт задачи — 25 секунд.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Артефакты только при успехе	Нет отчёта, когда он нужен	when: always
Слабые общие раннеры для UI	Таймауты и flaky	Ресурсы под задачу
Копипаста задач в каждом репозитории	Расхождения и ручные обновления	Общие шаблоны include

ПРАКТИКА 44 после главы 44

Проверьте задачу автотестов в своём CI по пяти пунктам.

ЧТО СДЕЛАТЬ

01	Используется ли образ с зафиксированной версией?	☐
02	Кэшируются ли зависимости по lock-файлу?	☐
03	Сохраняются ли артефакты при падении?	☐
04	Видны ли упавшие тесты в MR?	☐
05	Замерьте время от старта задачи до первого теста.	☐

РЕЗУЛЬТАТ Если до первого теста проходит больше минуты — начните с образа и кэша.

45 Разделение тестов по стадиям

После неё вы раскладываете тесты по стадиям пайплайна — коммит, MR, мерж, деплой, ночь — так, чтобы быстрые проверки шли первыми, а медленные не тормозили разработчика.

45.1 Стадии «Полки» 45.2 Выбор тестов для MR

Запускать все тесты на каждом изменении слишком долго, запускать только unit — слишком рискованно. Решение — стадии: каждая следующая медленнее, шире и реже. Быстрая обратная связь даётся там, где разработчик её ждёт, а полная проверка — там, где она нужна перед продакшеном.

45.1 Стадии «Полки»

СТАДИЯ	ЧТО ИДЁТ	ВРЕМЯ	БЛОКИРУЕТ
Pre-commit (локально)	Линтер, типы, unit изменённых модулей	<30 с	коммит
MR	Unit и интеграция сервиса, контракты, API-тесты затронутых областей	9 мин	мерж
Мерж в main	Полный API, UI-регресс, can-i-deploy	25 мин	деплой на stage
После деплоя на stage	Smoke, 12 E2E	9 мин	деплой в продакшен
После деплоя в продакшен	Smoke, синтетика	2 мин	откат
Ночь	Всё + мутации, кросс-браузер, мобильные	70 мин	ничего, утренний разбор

45.2 Выбор тестов для MR

На стадии MR идут не все API-тесты, а только затронутые. Скрипт смотрит на изменённые

файлы, по карте «сервис → теги» выбирает области и запускает тесты с этими тегами плюс smoke. Если изменились общие библиотеки или клиент API — запускается весь API-набор.

КАК СОКРАТИЛИ MR-СТАДИЮ

1 **Было**
Все 640 API-тестов на каждый MR: 14 минут.

2 **Карта**
14 сервисов связаны с 22 тегами областей.

3 **Выбор**
В среднем 120 тестов на MR: 3 минуты.

✓ **Страховка**
Полный набор на мерже ловит то, что пропустил выбор: за полгода — 3 случая.

ПРАВИЛО

Время стадии MR — главная метрика для разработчика. Выше 10–15 минут люди переключаются на другую задачу, и цена обратной связи резко растёт.

ПЛОХО

Всё в одной стадии

Один job на 70 минут на каждый MR.

Разработчики мержат, не дождавшись.

ХОРОШО

Воронка стадий

9 минут на MR, 25 на мерже, ночью — всё.

Каждая стадия блокирует следующий шаг.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Ночной прогон никто не смотрит	Дефекты лежат неделями	Утренний разбор с владельцем
E2E на каждом MR	Медленно и нестабильно	E2E после деплоя на stage
Выбор тестов без страховки	Пропуск регрессий	Полный набор на мерже

ПРАКТИКА 45 после главы 45

Разложите свои тесты по стадиям пайплайна.

ЧТО СДЕЛАТЬ

01	Выпишите все наборы тестов и их время.	☐
02	Распределите их по стадиям: pre-commit, MR, мерж, деплой, ночь.	☐
03	Определите, что блокирует каждая стадия.	☐
04	Настройте выбор тестов по изменённым областям для MR.	☐

РЕЗУЛЬТАТ Цель — MR-стадия не дольше 10 минут без потери полного регресса до продакшена.

46 Параллельный запуск тестов

После неё вы ускоряете прогон за счёт воркеров и шардов, считаете предел ускорения и готовите тесты и данные к параллельности.

46.1 Воркеры и шарды · 46.2 Подготовка к параллельности · 46.3 Кейс «Полки»

Параллельность — самый быстрый способ сократить время прогона без удаления тестов. Тесты делятся между воркерами на одной машине и между шардами на разных машинах CI. Но ускорение не бесконечно: его ограничивают самый долгий тест, общая подготовка и ресурсы стенда.

46.1 Воркеры и шарды

Воркер	Отдельный процесс на той же машине. Playwright по умолчанию запускает воркеров по числу ядер.
Шард	Отдельная задача CI, которая получает свою часть набора: --shard=2/4.
Балансировка	Распределение по времени прошлых прогонов, а не по числу тестов, — чтобы шарды заканчивали одновременно.

ФОРМУЛА 46.1 · ВРЕМЯ ПАРАЛЛЕЛЬНОГО ПРОГОНА

$$T_p = T_o + \max\left(\frac{T_s}{N}, t_{max}\right) \quad (46.1)$$

где

T_o	—	подготовка, общая для всех: старт, установка, авторизация	мин
T_s	—	суммарное время всех тестов	мин
N	—	воркеров × шардов	шт.
t_{max}	—	самый долгий тест или файл	мин

```
UI-регресс «Полки»: T₀ = 1,5; Tₛ = 38; N = 16; t_max = 2,1 → Tp = 1,5 + max(2,4; 2,1) ≈ 3,9 мин
```

Формула показывает два предела. Если t_max больше T_s/N, добавлять воркеры бесполезно — нужно делить длинный тест. Если T_0 велико, его надо сокращать, иначе на каждом шарде он повторяется.

46.2 Подготовка к параллельности

- Каждый тест создаёт свои данные с уникальным префиксом — главное условие.
- Никаких глобальных настроек, которые меняет один тест: выключатели функций, цены, тарифы.
- Тесты, которые меняют общее состояние, помечаются @serial и идут в отдельном проекте.
- Стенд выдерживает нагрузку: 16 воркеров — это 16 одновременных пользователей и больше.

46.3 Кейс «Полки»

При переходе с 1 на 8 воркеров упали 23 теста из 640: они использовали общий тестовый магазин и меняли его часы работы. Тимофей перевёл их на магазин-на-тест через фабрику; 4 теста настроек остались в @serial. Затем стенд начал отвечать 503 при 16 воркерах — увеличили число реплик API-шлюза на preview-стендах с 1 до 3.

ПЛОХО

Параллельность включили — и всё

workers: 16 без подготовки данных.

Десятки случайных падений, параллельность выключают обратно.

ХОРОШО

Сначала изоляция

Данные на тест, @serial для общего состояния.

Затем рост воркеров с замером стенда.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Шарды по числу тестов	Один шард заканчивает на 10 минут позже	Балансировка по времени
Один гигантский файл	Не делится между воркерами	fullyParallel или разбиение
Стенд не масштабирован	503 и таймауты	Нагрузка стенда в плане

ПРАКТИКА 46 после главы 46

Посчитайте предел ускорения своего набора и подготовьте его к параллельности.

ЧТО СДЕЛАТЬ

01	Замерьте T_0, T_s и t_max по последнему прогону.	☐
02	Посчитайте T_p для 4, 8 и 16 воркеров по формуле (46.1).	☐
03	Найдите тесты, которые меняют общее состояние, и изолируйте их.	☐
04	Увеличивайте параллельность шагами, следя за ошибками стенда.	☐

РЕЗУЛЬТАТ Если t_max ограничивает ускорение, начните с разделения самого длинного теста.

47 Отчёты и артефакты выполнения

После неё вы собираете отчёт, из которого причина падения видна без перезапуска, храните артефакты разумное время и связываете тесты с требованиями и логами.

47.1 Что сохранять · 47.2 Кейс «Полки»: Allure TestOps

Отчёт о прогоне нужен трём людям. Разработчику — понять, почему упал его MR. Автоматизатору — увидеть тренды: что стало медленнее, что стало нестабильным. Руководителю — убедиться, что релиз проверен. Один и тот же отчёт должен отвечать всем трём на разных уровнях детализации.

47.1 Что сохранять

АРТЕФАКТ	ДЛЯ ЧЕГО	КОГДА	СРОК ХРАНЕНИЯ
JUnit XML	Упавшие тесты в интерфейсе MR	всегда	14 дней
HTML / Allure	Разбор: шаги, вложения, история	всегда	14 дней, релизные — 1 год
Трейс Playwright	DOM, сеть, консоль по шагам	при падении	14 дней
Видео и скриншоты	Визуальная причина	при падении	7 дней
Запрос и ответ API	Диагностика API-тестов	при падении	14 дней
trace-id	Ссылка на логи сервиса	всегда	в отчёте

47.2 Кейс «Полки»: Allure TestOps

Раньше отчёт был HTML-файлом в артефактах задачи, и чтобы понять, падал ли тест раньше, приходилось открывать прошлые пайплайны. После перехода на Allure TestOps

результаты всех стадий собираются в одну базу: у каждого теста видна история, процент успешных прогонов, среднее время и связанные требования. Тест, который упал три раза за неделю на разных ветках, подсвечивается как кандидат во flaky.

```
test('cancels order and refunds card', async ({ customer }) => {
 await allure.link('CART-17', 'https://wiki.polka/req/CART-17');
 await allure.owner('payments-team');
 await test.step('place order', async () => { … });
 await test.step('cancel before assembly', async () => { … });
 await test.step('refund arrives', async () => { … });
});
```

ПРАВИЛО

У каждого упавшего теста в отчёте должен быть владелец. Падение без владельца ждёт, пока кто-нибудь заметит; падение с владельцем приходит конкретному человеку.

ПЛОХО

Лог консоли

3 000 строк вывода в задаче CI.

Чтобы найти упавший тест, нужен поиск по странице.

ХОРОШО

Отчёт для людей

Упавшие сверху, шаги, трейс, запрос и ответ.

История теста и владелец рядом.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Видео всех тестов всегда	Гигабайты в день, медленная выгрузка	Только при падении
Отчёты по стадиям раздельно	Нет общей картины	Единое хранилище результатов
Артефакты навсегда	Растущий счёт за хранение	Срок хранения по типу

ПРАКТИКА 47 после главы 47

Проверьте, отвечает ли ваш отчёт на вопросы трёх читателей.

ЧТО СДЕЛАТЬ

01	Откройте последний упавший прогон и найдите причину, засекая время.	☐
02	Добавьте недостающие артефакты из таблицы.	☐
03	Разметьте тесты владельцами и требованиями.	☐
04	Настройте сроки хранения артефактов.	☐

РЕЗУЛЬТАТ Отчёт хорош, если разработчик находит причину, не открывая лог задачи.

48 Автотесты как критерий качества релиза

После неё вы формулируете критерии выпуска на основе автотестов — quality gates — и решаете, что блокирует релиз, а что идёт в отчёт.

48.1 Quality gates «Полки» 48.2 Что не должно блокировать 48.3 Кейс «Полки»: первый откат по smoke

Когда релизы выходят каждый день, решение «выпускать или нет» не может принимать человек на совещании. Его принимает пайплайн по заранее согласованным правилам. Такие правила называют quality gates: набор условий, при нарушении которых сборка не идёт дальше.

48.1 Quality gates «Полки»

ГЕЙТ	УСЛОВИЕ	ДЕЙСТВИЕ
Тесты MR	Все тесты стадии зелёные, без новых в карантине	Блок мержа
Покрытие изменений	Новый код доменных модулей покрыт ≥ 80%	Блок мержа
Контракты	can-i-deploy = да для всех потребителей	Блок деплоя
E2E на stage	12 из 12 зелёных	Блок продакшена
Smoke в продакшене	Все проверки зелёные за 2 минуты	Автоматический откат
Ночной прогон	Падения разобраны к 11:00	Отчёт, не блок

48.2 Что не должно блокировать

Гейт, который часто блокирует без причины, команда научится обходить. Поэтому в гейты не включают тесты в карантине, нестабильные метрики (общее покрытие проекта, время прогона) и проверки, которые не связаны с изменением. Их показывают в отчёте и обсуждают на разборе.

48.3 Кейс «Полки»: первый откат по smoke

Через месяц после ввода гейтов smoke в продакшене упал на проверке поиска: новая версия сервиса каталога не подхватила индекс. Пайплайн откатил деплой через 3 минуты после выкатки. По логам успели затронуть около 40 поисковых запросов. Раньше такую ошибку замечали по жалобам в поддержку через 30–60 минут.

КАК ВВОДИЛИ ГЕЙТЫ

1. **Наблюдение**
   Месяц гейты только сообщают, но не блокируют: проверяем, как часто сработали бы ложно.
2. **Чистка**
   Нестабильные проверки убраны или исправлены: ложных срабатываний < 2%.
3. **Включение**
   Гейты блокируют; обход только через согласование техлида с записью причины.
4. ✓ **Итог**
   За полгода — 2 обхода гейтов, оба с разбором.

ПЛОХО

Релиз по ощущениям

«Вроде всё протестировали, выпускаем».

Решение на совещании, критерии разные каждый раз.

ХОРОШО

Релиз по правилам

Гейты согласованы заранее и записаны.

Пайплайн пропускает или останавливает сам.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Гейт на общее покрытие	Блок из-за чужого кода	Покрытие изменений
Обход без записи	Гейты превращаются в формальность	Обход через согласование и разбор
Гейты без периода наблюдения	Много ложных блоков на старте	Месяц в режиме отчёта

ПРАКТИКА 48 после главы 48

Сформулируйте quality gates для своего пайплайна.

ЧТО СДЕЛАТЬ

01	Выпишите стадии и решите, что каждая блокирует.	☐
02	Для каждого гейта сформулируйте условие и действие.	☐
03	Включите гейты в режиме отчёта на месяц и посчитайте ложные срабатывания.	☐
04	Договоритесь о процедуре обхода.	☐

РЕЗУЛЬТАТ Гейты работают, если их почти никогда не обходят и почти никогда не срабатывают ложно.

49 Виды отчётов по автотестам

После неё вы выбираете формат отчёта под читателя — разработчика, команду, руководство — и не перегружаете каждого чужими деталями.

49.1 Три уровня
49.2 Кейс «Полки»: ночное сообщение

Один отчёт для всех не работает. Разработчику нужен список упавших тестов его MR с причинами. Команде — тренды за неделю. Руководству — ответ на вопрос «можно ли выпускать и становится ли лучше». Если показать руководителю трейс Playwright, он перестанет открывать отчёты; если показать разработчику дашборд трендов, он не найдёт свой тест.

49.1 Три уровня

ОТЧЁТ	ЧИТАТЕЛЬ	СОДЕРЖАНИЕ	КАНАЛ
Прогона	Автор MR, дежурный	Упавшие тесты, шаги, трейсы, запросы	MR, ссылка в CI
Ночной	Владельцы областей	Новые падения, flaky, медленные тесты	Чат команды в 9:00
Недельный	Команда, техлиды	Тренды: время, flaky, покрытие требований, карантин	Дашборд, ретро
Квартальный	Руководство	DORA, дефекты в продакшене, экономия	Слайд на обзоре

49.2 Кейс «Полки»: ночное сообщение

```
Ночной прогон · 31 окт · 4 512 тестов · 71 мин · ✓ 4 498  ✗ 9  ⏸ 5 в карантине

Новые падения (3):
 @payments  refund arrives within 10 s    → @payments-team   впервые
 @slots     night slot not offered        → @slots-team      впервые
 @search    typo tolerance 'малако'       → @catalog-team    2-я ночь
Повторные (6): см. отчёт · Медленнее на 20%+: 4 теста
Отчёт: allure.polka.dev/launch/88412
```

Сообщение короткое и адресное: каждое падение с владельцем и признаком «впервые» или «повтор». Тесты в карантине не смешиваются с падениями. Владельцы отвечают в треде до 11:00 — это часть гейта ночного прогона из главы 48.

ПРАВИЛО

Отчёт для руководства — в терминах бизнеса: релизы, инциденты, время до продакшена, деньги. Число тестов и процент покрытия — внутренние метрики, они руководителю ничего не говорят.

ПЛОХО

Число тестов как успех

«За квартал написали 1 200 автотестов».

Руководитель не понимает, хорошо это или плохо.

ХОРОШО

Эффект как успех

«Дефектов в продакшене 18 → 7 в месяц, релиз ежедневно».

«Экономия на регрессе ≈ 0,7 млн ₽ за квартал».

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Один отчёт для всех	Никто не читает	Отчёт под читателя
Ночной отчёт без владельцев	Падения никто не берёт	Владелец у каждого падения
Только текущее состояние	Не видно трендов	Недельный дашборд

ПРАКТИКА 49 после главы 49

Соберите отчёты для трёх читателей.

ЧТО СДЕЛАТЬ

01	Определите, кто читает отчёты о тестах и зачем.	☐
02	Сделайте ночное сообщение с владельцами и признаком «впервые».	☐
03	Соберите недельный дашборд из пяти трендов.	☐
04	Подготовьте один слайд для руководства в терминах бизнеса.	☐

РЕЗУЛЬТАТ Проверка: каждый читатель находит свой ответ меньше чем за минуту.

50 Интерпретация падений

После неё вы классифицируете падения по причинам, разбираете групповые падения по общему корню и направляете каждое падение тому, кто может его исправить.

50.1 Четыре причины | 50.2 Групповые падения | 50.3 Кейс «Полки»: 64 красных теста

Падение теста имеет одну из четырёх причин: дефект продукта, дефект теста, проблема данных или проблема окружения. От причины зависит, кто чинит и насколько срочно. Ошибка классификации стоит дорого: если дефект продукта списать на стенд, он уйдёт в продакшен.

50.1 Четыре причины

ПРИЧИНА	ПРИЗНАКИ	КТО ЧИНИТ
Дефект продукта	Воспроизводится стабильно; связан с недавним изменением кода; ответ сервиса неверен по смыслу	Команда сервиса
Дефект теста	Продукт работает верно по требованию; тест ждёт устаревшего поведения	Автор теста или владелец области
Данные	Падает в arrange; связано с конкретной записью; проходит с новыми данными	Автор фабрики
Окружение	Падают десятки тестов разных областей; 502, 503, таймауты соединения	Дежурный по инфраструктуре

50.2 Групповые падения

Если в прогоне упало 80 тестов, разбирать их по одному бессмысленно — почти наверняка причина одна. Allure и аналогичные системы группируют падения по сообщению и стек-трейсу. В «Полке» правило такое: больше 20 падений в разных областях — сначала проверяем стенд, потом последние изменения общих библиотек.

50.3 Кейс «Полки»: 64 красных теста

РАЗБОР УТРЕННЕГО ПРОГОНА

1. **Группировка**
   64 падения, 3 группы: 58 — 401 Unauthorized, 4 — таймаут, 2 — разные.

2. **Главная группа**
   58 тестов 7 областей с 401: сервер авторизации на стенде обновили, срок токена сократили до 5 минут.

3. **Решение**
   Фикстура токенов теперь обновляет токен по exp; 58 тестов зелёные.

4. **Остальное**
   4 таймаута — медленный стенд; 2 разных — один дефект продукта, один дефект теста.

✓ **Итог**
Разбор занял 25 минут, а не полдня.

ВАЖНО · ТИПИЧНАЯ ОШИБКА

Начинать разбор с первого теста в списке. Сначала посмотрите на картину целиком: сколько упало, в каких областях, есть ли общее сообщение. Индивидуальный разбор — только для одиночных падений.

ПЛОХО

«Стенд опять лежит»

Любое падение по умолчанию — окружение.

Дефекты продукта проходят как «нестабильность».

ХОРОШО

Классификация по признакам

Причина определяется по признакам из таблицы.

Каждая категория уходит своему владельцу.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Разбор по одному при массовом падении	Часы на одну причину	Группировка
Нет категории причины в отчёте	Нет статистики	Метка категории у каждого разбора
Чинит тот, кто заметил	Неэффективно и с ошибками	Маршрутизация по причине

ПРАКТИКА 50 после главы 50

Разберите падения за последнюю неделю по четырём причинам.

ЧТО СДЕЛАТЬ

- [ ] 01 Выгрузите все падения за неделю.
- [ ] 02 Классифицируйте каждое по таблице.
- [ ] 03 Посчитайте долю каждой причины.
- [ ] 04 Для самой частой причины предложите системное исправление.

РЕЗУЛЬТАТ Если окружение даёт больше трети падений, следующим шагом стоит заняться стендами, а не тестами.

51 Отличие дефекта от нестабильного теста

После неё вы по истории и признакам отличаете настоящий дефект от flaky-теста, ловите нестабильность самого продукта и не прячете гонки в карантин.

51.1 Признаки
51.2 Нестабильность продукта
51.3 Кейс «Полки»: «нестабильный» тест возврата

Самая опасная ошибка в работе с падениями — назвать дефект нестабильностью. Тест, который падает в 3% прогонов, может быть плохо написан, а может честно ловить гонку в продукте, которая в продакшене случается у 3% пользователей. Карантин в таком случае прячет настоящую проблему.

51.1 Признаки

ПРИЗНАК	СКОРЕЕ ДЕФЕКТ	СКОРЕЕ FLAKY-ТЕСТ
Начало падений	Совпадает с изменением продукта	Без связи с изменениями
Воспроизводимость	Стабильно или при определённых условиях	Случайно
Ответ сервиса	Неверный по смыслу: 500, неправильная сумма	Верный, но тест не дождался
Где падает	В act или assert	В ожидании элемента, в arrange
Похожие жалобы	Есть в поддержке или мониторинге	Нет

51.2 Нестабильность продукта

Иногда нестабилен не тест, а продукт: гонка при параллельных запросах, кэш, который отдаёт устаревшее значение, событие, которое иногда теряется. Признак — тест написан правильно (ждёт состояния, изолирован по данным), а падает всё равно. Такой тест нужно не чинить, а превращать в воспроизводимый: повторить сценарий 100 раз, добавить параллельность, собрать логи сервиса по trace-id.

51.3 Кейс «Полки»: «нестабильный» тест возврата

Тест «возврат денег при отмене» падал в 4% прогонов, и его дважды отправляли в карантин. Тимофей запустил его 200 раз с логами: в 8 случаях событие отмены приходило в платёжный сервис раньше события оплаты, и возврат не создавался. В продакшене за месяц нашлось 37 заказов, отменённых в первые секунды после оплаты, где клиент не получил деньги автоматически. Дефект исправили, тест вернули в основной набор.

ПРАВИЛО

Перед карантином ответьте на вопрос: правильно ли написан тест? Если да — это дефект продукта, пока не доказано обратное.

ПЛОХО

Карантин по умолчанию

«Падает иногда — в карантин».

Гонка в продукте живёт месяцами.

ХОРОШО

Сначала проверить продукт

100–200 повторов с логами сервиса.

Карантин — только для доказанно плохого теста.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Не смотреть логи сервиса	Причина остаётся неизвестной	trace-id и логи в разборе
Увеличить таймаут	Прячет медленную деградацию	Сначала понять, почему долго
Не сверять с поддержкой	Не видно, что проблема у клиентов	Поиск похожих жалоб

ПРАКТИКА 51 после главы 51

Проверьте тесты в карантине на скрытые дефекты продукта.

ЧТО СДЕЛАТЬ

01	Для каждого теста в карантине проверьте признаки из таблицы.	☐
02	Для тестов, написанных правильно, повторите сценарий 100 раз с логами.	☐
03	Сверьте с обращениями в поддержку и мониторингом.	☐

РЕЗУЛЬТАТ Хотя бы один тест из карантина почти всегда оказывается настоящим дефектом.

52 Коммуникация результатов с командой

После неё вы сообщаете о падениях так, чтобы их брали в работу, пишете баг-репорт по результатам автотеста и разбираете крупные пропуски без поиска виноватых.

52.1 Баг-репорт по автотесту · 52.2 Каналы · 52.3 Разбор пропусков

Автотест нашёл дефект — половина работы. Вторая половина — чтобы дефект исправили. Для этого сообщение должно дойти до нужного человека, быть понятным без контекста и показывать, насколько это важно. Сообщение «тесты упали, посмотрите» не берёт в работу никто.

52.1 Баг-репорт по автотесту

```
Возврат не создаётся при отмене в первые 2 с после оплаты
Найден:     ночной прогон 88401, тест payments/refund.cancel-early.spec.ts
Частота:    8 из 200 повторов; в проде 37 заказов за октябрь
Шаги:       оплатить заказ → отменить через 0–2 с → ждать возврат 30 с
Ожидание:   возврат на сумму заказа в статусе CREATED
Факт:       возврата нет; order-cancelled обработан раньше payment-captured
Доказательства: трейс, логи по trace-id 4f1c…, запрос в Kafka UI
Влияние:    клиент ждёт ручного возврата 1–3 дня; ~37 обращений в месяц
Владелец:   @payments-team · Приоритет: высокий
```

52.2 Каналы

СИТУАЦИЯ	КАНАЛ	СКОРОСТЬ
Упал тест в моём MR	Комментарий пайплайна в MR	сразу
Новое падение в ночном прогоне	Тред в чате области с упоминанием владельца	до 11:00
Дефект продукта	Задача в трекере по шаблону	в день обнаружения

СИТУАЦИЯ	КАНАЛ	СКОРОСТЬ
Откат по smoke в продакшене	Канал инцидентов, дежурный	сразу
Тренды и долг	Ретро, недельный дашборд	раз в 2 недели

52.3 Разбор пропусков

Когда дефект дошёл до продакшена, полезно понять, почему тесты его не поймали. В «Полке» разбор идёт по формату без виноватых: что случилось, на каком уровне это можно было поймать раньше всего, почему не поймали, что меняем. Каждый разбор заканчивается конкретным тестом или изменением процесса с владельцем.

ПРАВИЛО

В сообщении о падении всегда три вещи: что сломалось для пользователя, кто владелец и ссылка на доказательства. Без любой из них сообщение будет висеть.

ПЛОХО

Сообщение без адреса

«Ночью упало 9 тестов, гляньте кто-нибудь».

Никто не считает это своей задачей.

ХОРОШО

Адресное сообщение

«@payments-team: возврат не создаётся при ранней отмене, 37 заказов в октябре, трейс по ссылке».

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Поиск виноватых на разборе	Люди скрывают ошибки	Формат без виноватых
Технический жаргон в задаче для продакта	Приоритет занижен	Влияние на пользователя первой строкой
Дефект только в чате	Теряется	Задача в трекере

ПРАКТИКА 52 после главы 52

Наладьте путь от падения до исправления.

ЧТО СДЕЛАТЬ

01	Сделайте шаблон баг-репорта по результатам автотеста.	☐
02	Определите канал и срок реакции для каждой ситуации.	☐
03	Проведите разбор последнего пропуска в продакшен без поиска виноватых.	☐

РЕЗУЛЬТАТ Метрика пути — время от падения до задачи у владельца.

53 Метрики автотестирования

После неё вы выбираете небольшой набор метрик, который показывает пользу, скорость и здоровье автотестов, и не превращаете метрики в цели, которые портят поведение.

53.1 Три группы метрик
53.2 Экономика автоматизации

Метрики нужны, чтобы видеть, приносит ли автоматизация пользу и не деградирует ли она. Ошибка — собирать всё подряд или выбрать одну метрику и сделать её целью. Число тестов и процент покрытия легко нарастить, ничего не улучшив.

53.1 Три группы метрик

ГРУППА	МЕТРИКА	«ПОЛКА» ДО → ЧЕРЕЗ ГОД
Польза	Дефекты в продакшене в месяц	18 → 5
Польза	Доля дефектов, найденных автотестами до продакшена	~25% → 78%
Польза	Часы ручного регресса на релиз	96 → 4
Скорость	Время MR-стадии	— → 9 мин
Скорость	Время от коммита до продакшена	~9 дней → ~3 часа
Здоровье	Доля flaky	14% → 0,6%
Здоровье	Время на поддержку от времени автоматизаторов	~60% → 25%
Здоровье	Тестов в карантине дольше 2 недель	— → 0

ФОРМУЛА 53.1 · ЭФФЕКТИВНОСТЬ ОБНАРУЖЕНИЯ

$$DDE = \frac{D_{\text{auto}}}{D_{\text{auto}} + D_{\text{prod}}} \tag{53.1}$$

где

D_{auto} — дефектов, найденных автотестами до продакшена — шт.

D_{prod} — дефектов, найденных в продакшене — шт.

```
Октябрь «Полки»: 18 найдено автотестами, 5 — в продакшене → DDE = 18 / 23 ≈ 78%
```

53.2 Экономика автоматизации

Нина свела годовой эффект в одну таблицу для руководства. Вложения — 7,92 млн ₽ на двух SDET и 0,9 млн ₽ на инфраструктуру CI. Эффект — 3,0 млн ₽ на ручном регрессе и 9,36 млн ₽ на дефектах в продакшене: (18 - 5) × 60 000 × 12. Итого 12,36 млн ₽ против 8,82 млн ₽, без учёта главного — ежедневных релизов.

КЛАССИКА Чарльз Гудхарт, 1975; формулировка Мэрилин Стратерн, 1997

Закон Гудхарта

Когда мера становится целью, она перестаёт быть хорошей мерой. Если команду оценивают по числу тестов, она пишет много тривиальных тестов; если по покрытию — тесты без проверок.

Поэтому метрики пользы стоит смотреть в связке с метриками здоровья, и ни одна не должна быть единственной целью.

ПЛОХО

Метрики объёма

«Цель квартала — 2 000 автотестов и 80% покрытия».

Растут тесты на геттеры.

ХОРОШО

Метрики эффекта

«Дефектов в продакшене меньше 8 в месяц, MR-стадия до 10 минут».

Тесты пишутся там, где ловят дефекты.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Число тестов как KPI	Тесты ради числа	Метрики пользы
Двадцать метрик	Никто не смотрит	5–7 ключевых
Метрики без трендов	Не видно деградации	Динамика по неделям

ПРАКТИКА 53 после главы 53

Соберите набор из 5–7 метрик автотестирования.

ЧТО СДЕЛАТЬ

01	Выберите по 2 метрики пользы, скорости и здоровья.	☐
02	Посчитайте их текущие значения.	☐
03	Посчитайте DDE по формуле (53.1) за последний месяц.	☐
04	Посчитайте годовой эффект и вложения в рублях.	☐

РЕЗУЛЬТАТ Экономика в рублях нужна один раз в квартал; метрики здоровья — каждую неделю.

54 Взаимодействие QA, разработчиков и аналитиков

После неё вы встраиваете автотесты в работу всей команды: критерии приёмки становятся тестами, а аналитик, разработчик и QA договариваются о проверках до начала разработки.

54.1 Три амиго · 54.2 Кейс «Полки»

Автотесты лучше всего работают, когда о проверках думают до написания кода. Если тестировщик узнаёт о функции на стенде, он автоматизирует то, что уже сделано, а не то, что требовалось. Если аналитик, разработчик и QA вместе обсуждают примеры заранее, тесты пишутся параллельно с кодом, а неясности в требованиях всплывают до разработки.

54.1 Три амиго

Короткая встреча перед взятием истории в работу: аналитик рассказывает, зачем она нужна, разработчик — как собирается делать, QA — как это может сломаться. Результат — список конкретных примеров, которые станут критериями приёмки и автотестами.

```
История: оплата при получении
Примеры (три амиго, 25 мин):
 заказ до 5 000 ₽ → доступен способ «при получении»
 заказ 5 000,01 ₽ → способа нет, подсказка «доступно до 5 000 ₽»
 новый клиент без заказов → способа нет
 клиент отказался от 2 заказов при получении → способ отключён на 30 дней
 курьер отмечает оплату → заказ DONE, чек отправлен
Уровни: правила — unit; доступность и отказы — API; выбор способа — 1 UI
```

54.2 Кейс «Полки»

Встречу трёх амиго ввели для историй больше двух дней. На встрече по оплате при получении всплыл вопрос, которого не было в требованиях: что делать с клиентом, который отказывается от заказов. Алиса уточнила у бизнеса и добавила правило блокировки. Тимофей начал писать API-тесты по примерам в первый день разработки, и к моменту готовности кода 9 из 11 тестов уже ждали его.

РОЛЬ	ВКЛАД В АВТОТЕСТЫ
Аналитик	Примеры и граничные случаи, связь требований с тестами
Разработчик	Unit и интеграционные тесты, testid, тестовые хуки, ревью системных тестов
QA / SDET	Стратегия, системные тесты, инфраструктура, исследовательское тестирование

ПЛОХО

Эстафета

Аналитик → разработчик → QA по очереди.

Вопросы к требованиям — на стенде, в конце спринта.

ХОРОШО

Вместе с начала

Примеры согласованы до разработки.

Тесты пишутся параллельно с кодом.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
QA узнаёт о задаче на стенде	Поздние вопросы, долгий цикл	Три амиго до разработки
Критерии приёмки абстрактные	Нельзя превратить в тест	Конкретные примеры с числами
Тесты пишутся после релиза	Регресс не защищён	Тесты в определении готовности

ПРАКТИКА 54 после главы 54

Проведите встречу трёх амиго для ближайшей истории.

ЧТО СДЕЛАТЬ

01	Выберите историю больше двух дней.	☐
02	Соберите аналитика, разработчика и QA на 20–30 минут.	☐
03	Запишите 5–10 конкретных примеров с числами.	☐
04	Распределите примеры по уровням тестов.	☐

РЕЗУЛЬТАТ Посчитайте, сколько вопросов к требованиям всплыло на встрече, а не на стенде.

55 Ответственность за автотесты

После неё вы распределяете владение тестами по областям продукта, договариваетесь, кто чинит упавший тест, и не оставляете набор «общим», то есть ничьим.

55.1 Модели ответственности
55.2 Кейс «Полки»: CODEOWNERS для тестов

У тестов, за которые отвечают все, нет владельца. Падение висит, потому что каждый думает, что им займётся кто-то другой. Владение тестами должно быть так же явно, как владение кодом: у каждой области есть команда, которая отвечает за её тесты, и человек, к которому идут с вопросами.

55.1 Модели ответственности

МОДЕЛЬ	КТО ПИШЕТ	КТО ЧИНИТ	РИСК
Отдельная QA-команда	QA	QA	Разработка не чувствует тесты своими; узкое горлышко
Разработчики пишут всё	Разработчики	Разработчики	Нет экспертизы стратегии и инфраструктуры
Смешанная (whole team)	Разработчики — unit и интеграция; SDET — системные и инфраструктура	Тот, чьё изменение сломало, с помощью владельца	Требует договорённостей

55.2 Кейс «Полки»: CODEOWNERS для тестов

```
# polka-tests/CODEOWNERS
/tests/api/orders/     @orders-team   @timofey
/tests/api/payments/   @payments-team @timofey
/tests/ui/checkout/    @web-team      @timofey
/fixtures/  /api/     @timofey       @qa-platform
/config/  .gitlab-ci.yml  @qa-platform
```

Правило «Полки»: если тест упал из-за твоего изменения, чинишь ты — в том же MR. Если тест упал в ночном прогоне без связанного изменения, его берёт владелец области. Тимофей и группа QA-платформы отвечают за общие фикстуры, клиенты и CI и помогают командам, но не чинят их тесты за них.

ПРАВИЛО

Кто сломал — тот чинит. Это правило важнее любой схемы ответственности: оно делает стоимость поломки видимой тому, кто её создал.

ПЛОХО

Тесты — работа QA

«Ваши тесты упали, почините».

Разработчики не читают тесты и не знают, что они проверяют.

ХОРОШО

Тесты — часть продукта

Команда отвечает за тесты своей области.

SDET помогает и развивает платформу.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Один автоматизатор на всю компанию	Уходит — тесты умирают	Владение по областям
Нет владельца у общих фикстур	Изменения ломают всех	Платформенная группа
Отключить тест вместо починки	Дыра в покрытии	Только карантин со сроком и задачей

ПРАКТИКА 55 после главы 55

Назначьте владельцев тестов по областям.

ЧТО СДЕЛАТЬ

01	Разбейте тесты на области продукта.	☐
02	Назначьте команду-владельца и контактное лицо для каждой.	☐
03	Заведите CODEOWNERS для репозитория тестов.	☐
04	Договоритесь о правиле «кто сломал — тот чинит».	☐

РЕЗУЛЬТАТ Проверка: на любое падение можно за секунду ответить, кто его берёт.

56 Code review автотестов

После неё вы ревьюите тесты по собственному чек-листу — проверяет ли тест то, что заявлено, устойчив ли он и понятен ли — и привлекаете разработчиков к ревью системных тестов.

56.1 Чек-лист ревьюера · 56.2 Кейс «Полки»: ревью разработчиком

Ревью тестов часто сводится к «вроде нормально». Но тест с ошибкой хуже, чем отсутствие теста: он создаёт ложную уверенность. Поэтому ревьюер теста отвечает на другие вопросы, чем ревьюер продукта: проверяет ли тест то, что написано в названии, упадёт ли он, если продукт сломается, и не упадёт ли, если продукт не сломан.

56.1 Чек-лист ревьюера

ВОПРОС	НА ЧТО СМОТРЕТЬ
Проверяет то, что заявлено?	Название ↔ assert; нет ли теста без проверок
Упадёт при дефекте?	Ожидания вычислены из входа, а не скопированы из ответа
Не упадёт без дефекта?	Нет пауз, общих данных, зависимостей от порядка и времени
Правильный уровень?	Можно ли проверить ниже: unit вместо API, API вместо UI
Понятен без автора?	Структура AAA, важные данные видны, нет магических чисел
Не дублирует?	Нет ли такой же проверки на этом или соседнем уровне
Диагностика?	При падении будет видна причина

56.2 Кейс «Полки»: ревью разработчиком

В «Полке» MR с системными тестами области требует аппрув двух человек: автоматизатора и разработчика команды-владельца. Разработчики сначала сопротивлялись — «это не наш код». Через два месяца выяснилось, что именно они находят самые полезные замечания: тест проверяет поведение, которое уже изменилось в разработке, или не учитывает асинхронную обработку, о которой автоматизатор не знал.

ЗАМЕЧАНИЯ НА РЕВЬЮ ЗА КВАРТАЛ

1 **Неверный уровень**
23% замечаний: проверку стоит перенести ниже.

2 **Стабильность**
21%: паузы, общие данные, отсутствие ожиданий.

3 **Слабые проверки**
19%: статус без тела, ожидание из ответа.

✓ **Остальное**
Читаемость, дубли, именование.

ПЛОХО

Ревью стиля

«Переименуй переменную, поставь точку с запятой».

Тест без проверок прошёл ревью.

ХОРОШО

Ревью смысла

«Этот assert пройдёт и при сломанной скидке — ожидание скопировано из ответа».

Линтер делает стиль, человек — смысл.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Аппрув без запуска	Тест может не проходить	Пайплайн как условие аппрува
MR на 2 000 строк тестов	Ревью поверхностное	До 400 строк
Только QA ревьюит QA	Нет знаний о продукте	Разработчик области в ревью

ПРАКТИКА 56 после главы 56

Введите чек-лист ревью тестов.

ЧТО СДЕЛАТЬ

01	Адаптируйте чек-лист под свой стек и добавьте его в шаблон MR.	☐
02	Подключите разработчиков-владельцев к ревью системных тестов.	☐
03	Через месяц посчитайте замечания по категориям.	☐

РЕЗУЛЬТАТ Категория с наибольшим числом замечаний — тема следующего внутреннего обучения.

57 Автотесты как часть культуры качества

После неё вы смещаете ответственность за качество с QA на всю команду, используете автотесты как живую документацию и поддерживаете привычки, без которых система деградирует.

57.1 Привычки
57.2 Кейс «Полки»: год спустя

Культура качества — это то, как команда ведёт себя, когда никто не проверяет: пишет ли разработчик тест на исправленный баг, останавливается ли мерж при красном пайплайне в пятницу вечером, разбирают ли падение или нажимают «перезапустить». Автотесты не создают культуру, но делают её видимой и поддерживают.

57.1 Привычки

Тест на каждый баг	Исправление дефекта начинается с теста, который его воспроизводит. Дефект не возвращается.
Красный — стоп	Красный main — главный приоритет команды, как упавший продакшен.
Тесты как документация	Новичок читает тесты области, чтобы понять правила: названия-спецификации делают это возможным.
Сдвиг влево	Вопросы о проверках — на этапе требований, а не на стенде.
Сдвиг вправо	Синтетика и мониторинг продолжают проверку в продакшене.

КЛАССИКА Эдвардс Деминг, «Out of the Crisis», 1982

Качество нельзя проверить, его нужно встроить

Один из 14 принципов Деминга: перестаньте полагаться на массовую инспекцию, встройте качество в продукт с самого начала. Инспекция в конце находит брак, но не предотвращает его.

Для разработки это означает: ручной регресс в конце цикла — инспекция; автотесты в каждом MR, примеры до разработки и тест на каждый баг — встроенное качество.

57.2 Кейс «Полки»: год спустя

ПРИВЫЧКА	ДО	ЧЕРЕЗ ГОД
Исправления багов с тестом	~10%	94%
Время красного main	часы, иногда дни	медиана 18 мин
Тесты, написанные разработчиками	~30% (только unit)	~70% всех тестов
Истории с тремя амиго	0	все от 2 дней

Нина считает главным результатом не цифры тестов, а то, что фраза «это проблема QA» исчезла из обсуждений. QA-команда из шести человек не выросла, но её работа сместилась: меньше ручных проверок, больше стратегии, исследовательского тестирования и помощи командам.

ПЛОХО

QA как фильтр

«Тестировщики найдут, если что».

Качество — ответственность последнего в цепочке.

ХОРОШО

Качество как свойство команды

«Мой код без теста не готов».

QA — эксперт и тренер, а не фильтр.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Красный main в пятницу — «в понедельник»	Норма красного цвета	Красный — стоп в любой день
Метрики качества только у QA	Команда их не видит	Дашборд для всех
Героизм перед релизом	Выгорание, ошибки	Маленькие релизы каждый день

ПРАКТИКА 57 после главы 57

Оцените привычки качества в своей команде.

ЧТО СДЕЛАТЬ

01	Посчитайте долю исправлений багов с тестом за последний месяц.	☐
02	Посчитайте медиану времени красного main.	☐
03	Выберите одну привычку и договоритесь о ней на ретро.	☐
04	Через месяц сравните цифры.	☐

РЕЗУЛЬТАТ Одна закреплённая привычка полезнее пяти объявленных.

58 Автотесты без ценности

После неё вы находите тесты, которые не ловят дефектов и не защищают от рисков, оцениваете ценность теста и удаляете лишнее без страха.

58.1 Признаки теста без ценности | 58.2 Как оценить ценность | 58.3 Кейс «Полки»: чистка unit-тестов

Каждый тест стоит денег: время на написание, время прогона на каждом MR, время на поддержку при изменениях. Тест, который никогда не упадёт на настоящем дефекте, — чистый расход. Таких тестов в любом большом наборе 10–30%, и их удаление делает набор быстрее без потери защиты.

58.1 Признаки теста без ценности

ПРИЗНАК	ПРИМЕР
Проверяет то, что не может сломаться	Геттер, конструктор, константа
Проверяет фреймворк, а не код	Что Spring внедрил зависимость
Не может упасть	Нет assert или assert на мок, который сам настроен
Проверяет то, что уже проверено ниже	UI-тест расчёта, покрытого 23 unit-кейсами
Проверяет мёртвую функцию	Функция отключена флагом год назад
Не падал ни разу за год и не связан с риском	Тест на редко меняемый справочник без инцидентов

58.2 Как оценить ценность

ФОРМУЛА 58.1 · ЦЕННОСТЬ ТЕСТА ЗА ГОД

$$V = n_{\mathrm{d}} \cdot c_{\mathrm{d}} - (c_{\mathrm{w}} + r \cdot t \cdot c_{\mathrm{min}} + c_{\mathrm{s}}) \tag{58.1}$$

где

n_{d}	—	дефектов, которые тест поймал или поймает (оценка по риску)	шт.
c_{d}	—	цена дефекта, если бы его не поймали	₽
r, t	—	прогонов в год и время прогона теста	шт., мин
c_{min}, c_{s}	—	цена минуты ожидания и поддержки за год	₽

```
Точный расчёт не нужен: формула напоминает, что тест с n_d ≈ 0 всегда имеет
отрицательную ценность
```

58.3 Кейс «Полки»: чистка unit-тестов

Из 900 исходных unit-тестов бэкенда Марат с командой удалили 412: 260 проверяли DTO и маппинг полей один в один, 98 — конфигурацию Spring, 54 — функции, отключённые флагами. Перед удалением прогнали мутационное тестирование: удалённые тесты убивали 3% мутантов, которых не убивали оставшиеся, и для этих мест написали 11 новых тестов. Прогон бэкенда сократился с 6 минут до 2,5.

ПРАВИЛО

Удаление теста — нормальное инженерное действие, а не потеря. Тест, который ничего не защищает, отнимает время у каждого MR.

ПЛОХО

Страх удаления

«Пусть будет, вдруг пригодится».

Набор растёт, прогон медленнее каждый месяц.

ХОРОШО

Удаление по критериям

Тест без риска и без падений за год — кандидат.

Мутации подтверждают, что защита не потеряна.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Тесты ради покрытия	Растёт число без пользы	Тесты от рисков
Никогда не удалять	Медленный набор	Квартальная чистка
Удалять без проверки	Потеря защиты	Мутационное тестирование

ПРАКТИКА 58 после главы 58

Найдите и удалите тесты без ценности в одном модуле.

ЧТО СДЕЛАТЬ

01	Пройдите тесты модуля по признакам из таблицы.	☐
02	Отметьте кандидатов на удаление.	☐
03	Запустите мутационное тестирование с ними и без них.	☐
04	Удалите тесты, без которых мутационный счёт не падает.	☐

РЕЗУЛЬТАТ Замерьте время прогона до и после — это лучший аргумент для следующей чистки.

59 Дублирование тестов

После неё вы находите дубли внутри уровня и между уровнями, понимаете, откуда они берутся, и сокращаете набор без потери покрытия требований.

59.1 Виды дублирования
59.2 Кейс «Полки»: поиск дублей

Дубли появляются естественно: разные люди покрывают одну функцию, тест копируют и меняют одну строку, после инцидента добавляют проверку, которая уже была на другом уровне. Каждый дубль замедляет прогон и удваивает работу при изменении требования, а уверенности не добавляет.

59.1 Виды дублирования

Точные копии	Два теста с одинаковыми шагами и проверками в разных файлах. Находятся поиском по похожести кода.
Вариации без нового класса	Пять тестов с qty = 2, 3, 4, 5, 6 — все из одного класса эквивалентности.
Межуровневые	Одно правило проверено в unit, API и UI на одних данных.
Подмножества	E2E-сценарий полностью включает шаги трёх UI-тестов.

59.2 Кейс «Полки»: поиск дублей

КАК НАШЛИ 37 ДУБЛЕЙ В КОРЗИНЕ

1. **Матрица покрытия**
   Аннотации @req показали 14 требований, проверенных одинаково на 2–3 уровнях (глава 37).

КАК НАШЛИ 37 ДУБЛЕЙ В КОРЗИНЕ

2 **Покрытие кода по тестам**
Для каждого API-теста — список затронутых строк; пары с совпадением >95% — кандидаты.

3 **Классы эквивалентности**
Параметризованные кейсы сверены с таблицей границ; 11 лишних вариаций.

✓ **Итог**
Удалено 37 тестов, объединено 18 в 5 параметризованных; покрытие требований не изменилось.

ВАЖНО · ТИПИЧНАЯ ОШИБКА

Копировать тест, чтобы проверить «ещё один случай», и менять в копии одну строку. Через полгода таких копий десять, и правка правила требует десяти изменений. Новый случай — новая строка в параметризации.

ПЛОХО

Копия с одной правкой

cart.promo.spec.ts и cart.promo2.spec.ts.

Отличаются одним значением промокода.

ХОРОШО

Параметризация

Одна таблица промокодов и ожиданий.

Новый случай — одна строка.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Новый E2E на каждый инцидент	Дублирует нижние уровни	Тест на нижнем уровне
Вариации одного класса	Медленно без новой информации	Один представитель класса + границы
Нет матрицы покрытия	Дубли невидимы	Аннотации требований

ПРАКТИКА 59 после главы 59

Найдите дубли в одной области продукта.

ЧТО СДЕЛАТЬ

01	Постройте матрицу покрытия требований по уровням.	☐
02	Найдите тесты с почти совпадающим покрытием кода.	☐
03	Сверьте параметризованные кейсы с классами эквивалентности.	☐
04	Удалите или объедините дубли.	☐

РЕЗУЛЬТАТ Проверьте, что матрица покрытия требований после чистки не изменилась.

60 Хрупкие UI-тесты

После неё вы находите причины хрупкости UI-тестов, договариваетесь с фронтендом о стабильных точках привязки и используете визуальное тестирование там, где проверка DOM бессильна.

60.1 Причины и лечение
60.2 Договорённость с фронтендом
60.3 Визуальное тестирование
60.4 Кейс «Полки»: редизайн карточки товара

Хрупкий тест ломается при изменении продукта, которое не меняет поведения: перенесли кнопку, поменяли класс, переименовали заголовок. Хрупкость — не то же самое, что нестабильность. Нестабильный тест падает случайно; хрупкий — предсказуемо, при каждом рефакторинге интерфейса. Итог один: команда перестаёт доверять UI-тестам.

60.1 Причины и лечение

ПРИЧИНА	ПРИМЕР	ЛЕЧЕНИЕ
Локаторы от структуры	div > div:nth-child(3) > span	Роли, подписи, data-testid
Локаторы от стилей	.btn-primary.mt-2	То же
Проверка текста целиком	Весь абзац с условиями доставки	Ключевая часть или testid
Проверка вёрстки через DOM	Ширина блока = 320pt	Визуальное сравнение
Длинные сценарии	40 шагов через 6 экранов	Подготовка через API, короткие тесты
Копии локаторов	Один селектор в 30 тестах	Page objects и компоненты

60.2 Договорённость с фронтендом

```
Соглашение о testid «Полки»
 формат:     <область>-<элемент>[—<уточнение>]: cart-total, slot-item, promo-input
 где:        интерактивные элементы без роли и подписи, значения для проверок
 нельзя:     удалять или переименовывать без задачи в @qa-platform
 проверка:   ESLint-правило запрещает CSS-локаторы в polka-tests
```

60.3 Визуальное тестирование

Некоторые ошибки нельзя увидеть через DOM: кнопка перекрыта баннером, текст вылез за карточку, иконка пропала. Для них есть сравнение скриншотов с эталоном: toHaveScreenshot в Playwright или облачные сервисы. В «Полке» визуальных тестов 24 — по ключевым экранам в трёх размерах. Они идут ночью, а эталоны обновляет автор изменения интерфейса через отдельный MR.

60.4 Кейс «Полки»: редизайн карточки товара

Год назад редизайн карточки товара сломал 41 UI-тест из 120, и их чинили неделю. Через год после перехода на компонентные объекты и testid похожий редизайн корзины затронул один файл — компонент CartItem — и 3 визуальных эталона. Правка заняла 40 минут.

ПЛОХО

Тест знает вёрстку

Локаторы от классов и вложенности.

Редизайн = неделя починки тестов.

ХОРОШО

Тест знает смысл

Роли, подписи и testid по соглашению.

Редизайн = правка одного компонента.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Визуальные тесты на каждый экран	Шум от мелких изменений	Только ключевые экраны
testid без соглашения	Удаляются при рефакторинге	Формат и правило изменения
Проверки стилей через DOM	Хрупко и не ловит видимое	Визуальное сравнение

ПРАКТИКА 60 после главы 60

Измерьте хрупкость своего UI-набора.

ЧТО СДЕЛАТЬ

01	Посчитайте, сколько тестов сломал последний редизайн или крупная правка интерфейса.	☐
02	Найдите все CSS и XPath-локаторы.	☐
03	Согласуйте с фронтендом соглашение о testid.	☐
04	Добавьте линтер, запрещающий CSS-локаторы.	☐

РЕЗУЛЬТАТ Цель — изменение интерфейса без изменения поведения затрагивает один объект компонента.

61 Отсутствие стратегии

После неё вы пишете стратегию автоматизации на одну-две страницы: цели, риски, уровни, ответственность, метрики — и сверяете с ней каждое решение о тестах.

61.1 Признаки отсутствия стратегии
61.2 Стратегия «Полки» на одной странице
61.3 Кейс «Полки»

Без стратегии автоматизация растёт как придётся: каждый пишет те тесты, которые умеет, там, где удобно. Через год набор большой, но непонятно, от чего он защищает, а на вопрос «можно ли выпускать» по-прежнему отвечает ручной регресс. Стратегия — не документ на 40 страниц, а короткий набор решений, с которыми согласна команда.

61.1 Признаки отсутствия стратегии

- Никто не может назвать, зачем автоматизация, кроме «чтобы были автотесты».
- Тесты пишутся там, где проще, а не там, где риск.
- Уровни выбираются по привычке автора.
- Нет договорённости, что блокирует релиз.
- Успех измеряется числом тестов.

61.2 Стратегия «Полки» на одной странице

```
Цель:          релиз каждый день; дефектов в продакшене < 6 в месяц к Q4
Главные риски: оплата и возвраты · слоты и гонки · стыки 14 сервисов · цены и скидки
Уровни:        логика — unit; стыки — интеграция и контракты; сценарии — API;
               интерфейс — короткие UI; 12 критичных путей — E2E; синтетика в проде
Не автоматизируем: вёрстку новых фич до стабилизации, разовые миграции, исследование
Ответственность: команды — тесты своих областей; QA-платформа — каркас, CI, обучение
Гейты:         MR 10 мин · мерж 25 мин · E2E на stage · smoke в проде с откатом
Метрики:       дефекты в проде, DDE, MR-время, flaky < 1%, доля поддержки < 30%
Пересмотр:     раз в квартал на обзоре качества
```

61.3 Кейс «Полки»

Первую версию стратегии Нина написала за неделю после интервью с техлидами и разбора инцидентов за полгода. Ключевым оказался раздел «не автоматизируем»: он снял споры о том, нужно ли покрывать UI-тестами каждую новую фичу. Каждый квартал стратегию сверяют с метриками: в третьем квартале, например, приоритет сместился с API на контракты, потому что половина инцидентов пришла со стыков.

ПРАВИЛО

Стратегия начинается с рисков продукта, а не с инструментов. Инструменты — следствие: сначала решите, что защищать, потом — на каком уровне и чем.

ПЛОХО

Стратегия-инструмент

«Наша стратегия — Selenium и Java».

Неясно, что и зачем проверяется.

ХОРОШО

Стратегия-решения

Цели, риски, уровни, ответственность, гейты, метрики.

Одна страница, пересмотр раз в квартал.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Стратегия на 40 страниц	Никто не читает	Одна-две страницы
Стратегия без «не делаем»	Споры о каждой фиче	Явный список исключений
Написали и забыли	Расходится с реальностью	Квартальный пересмотр

ПРАКТИКА 61 после главы 61

Напишите стратегию автоматизации на одну страницу.

ЧТО СДЕЛАТЬ

- [ ] **01** Сформулируйте цель автоматизации в терминах бизнеса.
- [ ] **02** Выпишите 3–5 главных рисков продукта по инцидентам.
- [ ] **03** Распределите риски по уровням тестов.
- [ ] **04** Добавьте раздел «не автоматизируем», ответственность, гейты и метрики.
- [ ] **05** Согласуйте с техлидами и назначьте дату пересмотра.

РЕЗУЛЬТАТ Хорошая стратегия помещается на один экран и меняет хотя бы одно текущее решение.

62 Игнорирование поддержки тестов

После неё вы планируете время на поддержку автотестов, замечаете признаки деградации набора и не допускаете момента, когда тесты проще выбросить, чем починить.

62.1 Стадии деградации
62.2 Сколько времени закладывать
62.3 Кейс «Полки»: старый набор

Автотесты — не разовая инвестиция. Продукт меняется, и тесты требуют постоянной работы: обновлений, починки, рефакторинга, чистки. Команды, которые считают только стоимость написания, через полтора-два года обнаруживают, что набор красный, медленный и никому не нужен. Обычно тогда принимают решение «переписать с нуля» — и цикл повторяется.

62.1 Стадии деградации

КАК УМИРАЕТ НАБОР

1. **Отставание**
   Тесты не успевают за продуктом; несколько падений «известны».
2. **Привычка**
   Команда перезапускает пайплайн, пока не позеленеет.
3. **Обход**
   Красные тесты отключают, гейты обходят «временно».
4. **Недоверие**
   Релиз снова проверяют вручную; автотесты — формальность.
5. ✓ **Конец**
   Набор выбрасывают; через год начинают новый.

62.2 Сколько времени закладывать

ЗРЕЛОСТЬ НАБОРА	ДОЛЯ ВРЕМЕНИ НА ПОДДЕРЖКУ	КОММЕНТАРИЙ
Первые месяцы	10–15%	Набор маленький, всё свежее
Устоявшийся, хорошая архитектура	20–30%	Норма для живого продукта
Большой, без слоёв и фабрик	50–70%	Сигнал: нужен рефакторинг, а не новые тесты

62.3 Кейс «Полки»: старый набор

Старые 120 Selenium-тестов «Полки» прошли все четыре стадии. Их писали два автоматизатора, которые ушли из компании, и после их ухода никто не отвечал за поддержку. К началу истории 14% тестов были нестабильны, ещё 22 отключены, и релиз всё равно проверяли вручную. Нина посчитала, что за три года в набор вложено около 4 млн ₽, и почти весь эффект потерян.

ВАЖНО · ТИПИЧНАЯ ОШИБКА

Считать автоматизацию проектом с датой окончания. Проект автоматизации заканчивается тогда же, когда продукт: никогда. Бюджет на поддержку нужен столько же лет, сколько живёт продукт.

ПЛОХО

Написали и ушли

«Подрядчик сделает 500 автотестов за квартал».

Через год половина красная, подрядчика нет.

ХОРОШО

Поддержка в плане

Время на поддержку — 20–30% постоянно.

Владельцы областей отвечают за свои тесты.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Бюджет только на написание	Деградация за 1–2 года	Поддержка в бюджете
Нет метрик здоровья	Деградация незаметна	Flaky, карантин, доля поддержки
Переписать с нуля	Повторение цикла	Погашать долг по пунктам

ПРАКТИКА 62 после главы 62

Определите стадию своего набора и план поддержки.

ЧТО СДЕЛАТЬ

01	Сопоставьте признаки своего набора со стадиями деградации.	☐
02	Посчитайте долю времени автоматизаторов на поддержку.	☐
03	Если вы на стадии 2 и выше — составьте план возврата с реестром долга из главы 42.	☐

РЕЗУЛЬТАТ Стадии 1–2 лечатся за месяц; стадии 3–4 — за квартал и дороже.

63 Рост автотестовой базы

После неё вы управляете ростом набора: держите время прогона в бюджете, развиваете общую платформу и не даёте числу тестов расти быстрее их пользы.

63.1 Бюджет времени · 63.2 Кейс «Полки»: рост за год · 63.3 Платформа

Набор растёт вместе с продуктом, и это нормально. Проблема — когда время прогона, нестабильность и стоимость поддержки растут быстрее, чем польза. Это происходит незаметно: каждый новый тест добавляет всего 10 секунд, но через год их тысяча.

63.1 Бюджет времени

Самый действенный механизм — бюджет времени стадий. MR-стадия «Полки» должна укладываться в 10 минут, стадия мержа — в 25. Если новый тест выводит стадию за бюджет, его автор вместе с QA-платформой ищет, что ускорить или перенести в ночной прогон. Бюджет превращает рост набора из стихийного процесса в управляемый.

63.2 Кейс «Полки»: рост за год

Квартал	Тестов	MR-стадия	Flaky	Дефекты в проде/мес
Q1	1 400	14 мин	3,5%	15
Q2	2 600	11 мин	1,1%	11
Q3	3 700	9 мин	0,7%	7
Q4	4 530	9 мин	0,6%	5

Тестов стало втрое больше, а MR-стадия ускорилась: выбор тестов по изменённым областям, параллельность и удаление бесполезных тестов перекрыли рост. Это и есть здоровый рост: польза растёт, а цена для разработчика — нет.

63.3 Платформа

- Общие фикстуры, фабрики и клиенты API — в отдельном пакете с версиями.
- Шаблоны CI-задач, которые команды подключают одной строкой.
- Генератор нового тестового модуля: структура, конфиг, пример теста.
- Документация и внутреннее обучение: как писать тест в «Полке» за 30 минут.

ПЛОХО

Рост без бюджета

Каждый месяц MR-стадия +1 минута.

Через год — 25 минут, разработчики не ждут.

ХОРОШО

Рост в бюджете

Бюджет стадии фиксирован.

Новые тесты оплачиваются ускорением или переносом.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Число тестов как цель роста	Тесты без пользы	Польза и бюджет времени
Каждая команда со своим каркасом	Дублирование и разный уровень	Общая платформа
Нет ночного прогона для тяжёлого	Всё тянется в MR	Ночь для медленных наборов

ПРАКТИКА 63 после главы 63

Введите бюджет времени для стадий пайплайна.

ЧТО СДЕЛАТЬ

01	Зафиксируйте текущее время каждой стадии.	☐
02	Установите бюджет для MR и мержа.	☐
03	Настройте предупреждение при превышении бюджета.	☐
04	Постройте таблицу роста за последние кварталы, как в кейсе.	☐

РЕЗУЛЬТАТ Если время растёт быстрее пользы — начните с выбора тестов по изменениям.

64 Работа с большими проектами

После неё вы организуете автотесты в проекте с десятками команд и тысячами тестов: модули, версии общих библиотек, распределённое владение и единые стандарты.

64.1 Модульность набора
64.2 Изменения общих библиотек
64.3 Стандарты
64.4 Кейс «Полки»: вторая волна роста

В большом проекте автотесты перестают быть делом одной команды. Десять команд пишут тесты в общем наборе, общие фикстуры меняются каждую неделю, а изменение в клиенте API может сломать тесты у всех. Здесь работают те же принципы, что и в коде продукта: модульность, явные интерфейсы, версии и стандарты.

64.1 Модульность набора

ЭЛЕМЕНТ	КАК ОРГАНИЗОВАН	ВЛАДЕЛЕЦ
Ядро	Конфиг, авторизация, отчёты, базовые фикстуры — пакет @polka/test-core с версиями	QA-платформа
Клиенты API	Генерируются из OpenAPI каждого сервиса, публикуются при изменении спецификации	Команда сервиса
Модули областей	tests/<область> со своими фикстурами и page objects	Команда области
Общие компоненты UI	Компонентные объекты дизайн-системы	Команда дизайн-системы

64.2 Изменения общих библиотек

Изменение ядра без версий ломает всех сразу. В «Полке» test-core публикуется с семантическими версиями: патчи и минорные обновления подхватываются автоматически через бот, мажорные — с инструкцией по миграции и периодом, когда старая версия ещё поддерживается. Перед публикацией новой версии CI ядра прогоняет тесты всех областей против неё.

64.3 Стандарты

Линтер	Общие правила ESLint для всех модулей: запрет CSS-локаторов, пауз, проверок без await.
Шаблон теста	Генератор создаёт файл со структурой AAA, тегами и аннотациями владельца.
Гильдия	Раз в две недели автоматизаторы и разработчики-энтузиасты разбирают сложные случаи и меняют стандарты.

64.4 Кейс «Полки»: вторая волна роста

Когда «Полка» выросла с 38 до 70 инженеров и с 6 до 11 команд, первым сломалось ядро: изменение фикстуры авторизации уронило тесты четырёх команд за одно утро. После перехода на версии и прогон всех модулей в CI ядра таких случаев за полгода не было.

ПЛОХО

Общая папка utils

Любой меняет общие хелперы.

Правка одного ломает тесты всех.

ХОРОШО

Ядро с версиями

Изменения ядра проверяются на всех модулях.

Мажорные изменения — с миграцией.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Общие фикстуры без владельца	Хаос изменений	Платформенная команда
Разные стандарты у команд	Нельзя помочь соседу	Общий линтер и шаблон
Изменение ядра без прогона модулей	Массовые поломки	CI ядра против всех модулей

ПРАКТИКА 64 после главы 64

Выделите ядро набора и защитите его изменения.

ЧТО СДЕЛАТЬ

01	Определите, какой код используется всеми областями.	☐
02	Вынесите его в пакет с версиями и владельцем.	☐
03	Настройте прогон тестов всех областей при изменении ядра.	☐
04	Введите общий линтер и шаблон теста.	☐

РЕЗУЛЬТАТ Проверка: изменение ядра не ломает ни одной области без предупреждения.

65 Оптимизация времени прогона

После неё вы находите, на что на самом деле уходит время прогона, и сокращаете его по порядку отдачи: подготовка, параллельность, выбор тестов, медленные тесты, кэш.

65.1 Кейс «Полки»: разбор стадии мержа
65.2 Порядок оптимизации

Время прогона — это не только время тестов. В него входят ожидание раннера, скачивание образа, установка зависимостей, деплой стенда, подготовка данных, сами тесты и выгрузка отчётов. Часто тесты занимают меньше половины, и оптимизировать нужно не их.

65.1 Кейс «Полки»: разбор стадии мержа

ЭТАП	БЫЛО, МИН	СТАЛО, МИН	ЧТО СДЕЛАЛИ
Ожидание раннера	4,0	0,5	Автомасштабируемые раннеры
Образ и зависимости	3,5	0,4	Образ Playwright, кэш npm
Деплой стенда	9,0	4,0	Только изменённые сервисы, прогретые образы
Подготовка данных	6,0	1,5	Служебные эндпоинты вместо UI и цепочек API
API-тесты	14,0	6,0	8 воркеров × 2 шарда
UI-тесты	11,0	5,5	16 воркеров, логин через сохранённую сессию
Отчёты	3,0	1,0	Видео только при падении
Итого	50,5	**~19 + запас**	**Бюджет стадии — 25 минут**

65.2 Порядок оптимизации

ПО УБЫВАНИЮ ОТДАЧИ

1. **Измерить**
   Время каждого этапа и каждого теста за неделю; топ-20 самых долгих тестов.
2. **Инфраструктура**
   Ожидание раннера, образы, кэш: обычно −20–40% без изменения тестов.
3. **Подготовка**
   Данные через API или служебные эндпоинты, сессии вместо логина.
4. **Параллельность**
   Воркеры и шарды с балансировкой по времени (глава 46).
5. **Выбор тестов**
   Только затронутые области на MR (глава 45).
6. ✓ **Медленные тесты**
   Разбить, перенести ниже по пирамиде или в ночной прогон.

ПРАВИЛО

Не оптимизируйте вслепую. Один замер времени по этапам экономит недели работы над тем, что занимает 5% прогона.

ПЛОХО

Ускорение тестов наугад

Неделя на оптимизацию локаторов.

Прогон быстрее на 40 секунд из 50 минут.

ХОРОШО

Ускорение по замеру

Разбор по этапам показал 9 минут деплоя и 4 минуты очереди.

Первые два шага дали −12 минут.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Удалять тесты ради скорости	Потеря защиты	Сначала инфраструктура и подготовка
Видео и трейсы всех тестов	Минуты на выгрузку	Только при падении

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Один шард на всё	Нет масштабирования	Шарды по времени

ПРАКТИКА 65 после главы 65

Разберите время одной стадии по этапам.

ЧТО СДЕЛАТЬ

01	Замерьте каждый этап стадии за неделю, как в таблице кейса.	☐
02	Найдите два этапа с наибольшим временем.	☐
03	Примените шаги оптимизации по порядку.	☐
04	Сравните время до и после.	☐

РЕЗУЛЬТАТ Обычно первые два шага дают больше, чем все последующие вместе.

66 Поддержка нескольких продуктов

После неё вы строите автотесты для нескольких продуктов или брендов на общей платформе, отделяете общее от специфичного и не дублируете тесты для каждой копии.

66.1 Варианты многопродуктовости
66.2 Кейс «Полки»: «Полка Бизнес»

Когда у компании появляется второй продукт, есть соблазн скопировать набор тестов первого и поправить. Через год копии расходятся, исправление в одной не попадает в другую, и поддержка обходится вдвое дороже. Правильный путь — общая платформа и продуктовые модули, которые на ней стоят.

66.1 Варианты многопродуктовости

СИТУАЦИЯ	ЧТО ОБЩЕЕ	ПОДХОД
Белые метки (один код, разные бренды)	Почти всё	Один набор, бренд — параметр конфига
Продукты на общей платформе	Авторизация, платежи, профиль	Общие модули + продуктовые модули
Разные продукты, общая компания	Каркас, CI, стандарты	Общая платформа тестов, отдельные наборы

66.2 Кейс «Полки»: «Полка Бизнес»

Компания запустила «Полку Бизнес» — доставку для офисов с юрлицами, счетами и лимитами. Корзина, слоты и каталог — общие, оплата и личный кабинет — свои. Тимофей не стал копировать набор. Общие сценарии параметризовали продуктом: тест корзины идёт дважды, для покупателя и для офиса. Специфичные — счета, лимиты, роли сотрудников — живут в модуле tests/business.

```
projects: [
 { name: 'b2c', use: { product: 'polka',    baseURL: env.b2c } },
 { name: 'b2b', use: { product: 'business', baseURL: env.b2b } },
]
// tests/api/cart/* — запускаются в обоих проектах
// tests/business/* — только в b2b: testMatch по папке
```

Через полгода общий набор вырос на 30% вместо 100% при копировании. Исправление фикстуры корзины одновременно чинит тесты обоих продуктов, а дефект в общем сервисе ловится тестами того продукта, где он проявился раньше.

ПЛОХО

Копия набора

polka-tests и polka-business-tests.

Через год — два разных каркаса и двойная поддержка.

ХОРОШО

Платформа и модули

Общие тесты параметризованы продуктом.

Специфичное — в своём модуле.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Копирование набора	Расхождение, двойная поддержка	Платформа + модули
if (product === ...) в тестах	Тесты нечитаемы	Различия — в конфиге и фикстурах
Общие тесты только для одного продукта	Дефекты второго не видны	Запуск общих тестов в обоих проектах

ПРАКТИКА 66 после главы 66

Разделите общее и специфичное для своих продуктов.

ЧТО СДЕЛАТЬ

01	Выпишите функции, общие для продуктов, и специфичные.	☐
02	Параметризуйте общие тесты продуктом через проекты или конфиг.	☐
03	Вынесите специфичные тесты в отдельные модули.	☐

РЕЗУЛЬТАТ Проверка: исправление общей фикстуры одним MR чинит тесты всех продуктов.

67 Автотесты в микросервисной архитектуре

После неё вы собираете стратегию тестирования микросервисов — тесты сервиса в изоляции, контракты между сервисами, минимум сквозных проверок и наблюдаемость в продакшене — и видите итог пути «Полки».

67.1 Слои стратегии · 67.2 Асинхронное взаимодействие · 67.3 Независимый деплой · 67.4 Итог пути «Полки»

В монолите один набор тестов проверяет один артефакт. В микросервисах каждый сервис деплоится отдельно, и главный вопрос меняется: не «работает ли система», а «можно ли выкатить этот сервис, не сломав остальные». Ответ на него дают не E2E-тесты, которых всегда будет мало, а тесты сервиса в изоляции плюс контракты с соседями.

67.1 Слои стратегии

СЛОЙ	ЧТО ПРОВЕРЯЕТ	ГДЕ ИДЁТ	«ПОЛКА»
Unit	Логика внутри сервиса	Сборка сервиса	3 200
Интеграция сервиса	Сервис + его база, очередь, кэш в контейнерах	Сборка сервиса	420
Компонентные (сервис целиком)	API сервиса с подменёнными соседями	MR сервиса	входят в 640 API
Контракты	Совместимость потребителей и провайдеров	MR провайдера, can-i-deploy	180
Системные API	Цепочки через несколько сервисов	Мерж, stage	входят в 640 API
E2E	12 критичных путей	После деплоя на stage	12 + 8 мобильных
Синтетика и мониторинг	Продакшен	Каждые 5 минут	6 сценариев

КЛАССИКА Тоби Клеменсон, «Testing Strategies in a Microservice Architecture», 2014

Тестирование микросервисов

Клеменсон описал слои тестов микросервисов: unit, интеграционные, компонентные, контрактные и сквозные — и показал, почему компонентные тесты сервиса с подменёнными соседями должны нести основную нагрузку.

Сквозные тесты он советовал держать минимальными и формулировать как пользовательские пути, а не как проверку каждой функции через всю систему.

67.2 Асинхронное взаимодействие

Половина взаимодействий в «Полке» идёт через события в Kafka. Для них работают те же идеи: интеграционный тест проверяет, что сервис публикует событие нужной схемы и корректно обрабатывает входящее, включая повтор и нарушенный порядок; схема событий хранится в реестре и проверяется на совместимость, как контракт HTTP; системный тест ждёт результата через expect.poll.

67.3 Независимый деплой

ПУТЬ ИЗМЕНЕНИЯ СЕРВИСА СЛОТОВ

1. **MR**
   Unit, интеграция, компонентные API-тесты сервиса, проверка контрактов потребителей: 7 мин.
2. **Мерж**
   Публикация образа, can-i-deploy, системные API-тесты затронутых областей: 12 мин.
3. **Stage**
   Smoke и 12 E2E: 9 мин.
4. **Продакшен**
   Канареечный деплой на 5% трафика, smoke, метрики ошибок 10 минут, затем 100%.

✓ **Итог**
От мержа до 100% продакшена — около 45 минут без участия человека.

67.4 Итог пути «Полки»

ПОКАЗАТЕЛЬ	НАЧАЛО	ЧЕРЕЗ ГОД
Автотестов	1 020	4 530
Ручной регресс	3 дня, 96 ч	4 ч исследовательского
Релизы	раз в 2 недели	5–8 в день
Дефекты в продакшене	18 в месяц	5 в месяц
Flaky	14% UI	0,6% всех
MR-стадия	—	9 мин
Годовой эффект	—	12,36 млн ₽ при вложениях 8,82 млн ₽

ПРАВИЛО

В микросервисах проверяйте каждый сервис так, как будто остальные — внешние системы, а совместимость обеспечивайте контрактами. Сквозные тесты подтверждают, что пазл собран, но не могут проверить каждую деталь.

ПЛОХО

Распределённый монолит в тестах

Чтобы проверить один сервис, поднимаем все 14.

Деплой одного сервиса ждёт полного E2E-прогона.

ХОРОШО

Независимые сервисы

Сервис проверяется в изоляции и контрактами.

Деплоится сам, когда его гейты зелёные.

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Общий E2E-набор как гейт каждого сервиса	Деплои блокируют друг друга	Компонентные тесты + контракты
События без схемы	Тихие поломки потребителей	Реестр схем и проверка совместимости
Нет синтетики в продакшене	Поломки находят клиенты	Сценарии каждые 1–5 минут

ПРАКТИКА 67 после главы 67

Опишите путь одного сервиса от MR до продакшена и проверьте, может ли он деплоиться независимо.

ЧТО СДЕЛАТЬ

01	Распишите слои тестов для одного сервиса, как в таблице.	☐
02	Проверьте, есть ли компонентные тесты с подменёнными соседями.	☐
03	Проверьте, закрыты ли контрактами все его потребители и провайдеры.	☐
04	Посчитайте время от мержа до продакшена и найдите этапы, требующие человека.	☐

РЕЗУЛЬТАТ Цель — любой сервис выкатывается сам, когда его тесты и контракты зелёные.

A Шаблоны документов

Шаблоны из глав книги в виде, готовом к копированию: заполните их для своего продукта.

A.1 Стратегия автоматизации на одну страницу
A.2 Решение о выборе инструмента (ADR)
A.3 Баг-репорт по результатам автотеста
A.4 Пункт реестра технического долга
A.5 Матрица ролей и прав
A.6 Таблица границ параметра
A.7 Quality gates

A.1 Стратегия автоматизации на одну страницу

```
Цель:              <в терминах бизнеса: частота релизов, дефекты в продакшене>
Главные риски:     <3–5 рисков по инцидентам за полгода>
Уровни:            логика — unit · стыки — интеграция, контракты · сценарии — API
                   интерфейс — UI · критичные пути — E2E · продакшен — синтетика
Не автоматизируем: <список исключений>
Ответственность:   <кто пишет и чинит по уровням и областям>
Гейты:             <стадия → условие → действие>
Метрики:           <2 пользы · 2 скорости · 2 здоровья>
Пересмотр:         <дата, раз в квартал>
```

A.2 Решение о выборе инструмента (ADR)

```
ADR-<N> · <дата> · выбор <инструмента> для <уровня>
Контекст:    <язык продукта, кто пишет тесты, проблемы текущего решения>
Кандидаты:   <2–3 варианта>
Критерии:    <критерий · вес> × <оценки кандидатов> → итог
Пилот:       <сценарии, число прогонов, время, flaky, время диагностики>
Решение:     <что выбрали и почему>
Последствия: <миграция, обучение, что перестаём использовать и когда>
```

A.3 Баг-репорт по результатам автотеста

```
<Что сломалось для пользователя — одной строкой>
Найден:          <прогон, тест, ссылка на отчёт>
Частота:         <N из M прогонов; данные продакшена, если есть>
Шаги:            <минимальная последовательность>
Ожидание / факт: <…> / <…>
Доказательства:  <трейс, логи по trace-id, запрос и ответ>
Влияние:         <сколько пользователей, деньги, обращения>
Владелец · приоритет
```

A.4 Пункт реестра технического долга

```
ТД-<N> · <короткое название>
 симптом:   <что происходит, в цифрах>
 цена:      <минуты × прогоны × доля ожидания = часы в месяц>
 решение:   <что сделать>
 оценка:    <дни> · владелец: <имя> · приоритет: <высокий | средний | низкий>
```

A.5 Матрица ролей и прав

РОЛЬ	ДЕЙСТВИЕ	ОЖИДАЕМЫЙ СТАТУС	ТЕСТ
<роль>	<метод и путь>	<200 | 403 | 404>	<файл теста>
…	…	…	…

A.6 Таблица границ параметра

ПАРАМЕТР	ДОПУСТИМО	КЛАССЫ	ГРАНИЦЫ	ОЖИДАНИЕ ВНЕ ГРАНИЦ
<имя>	<диапазон>	<классы эквивалентности>	<мин, мин-шаг, макс, макс+шаг>	<статус и код ошибки>

A.7 Quality gates

СТАДИЯ	УСЛОВИЕ	ДЕЙСТВИЕ ПРИ НАРУШЕНИИ	БЮДЖЕТ ВРЕМЕНИ
MR	<…>	блок мержа	<мин>

СТАДИЯ	УСЛОВИЕ	ДЕЙСТВИЕ ПРИ НАРУШЕНИИ	БЮДЖЕТ ВРЕМЕНИ
Мерж	<...>	блок деплоя на stage	<мин>
Stage	<...>	блок продакшена	<мин>
Продакшен	<...>	откат	<мин>
Ночь	<...>	отчёт и разбор к <время>	<мин>

Б Чек-листы по этапам

Проверочные списки для ключевых этапов автоматизации. Отмечайте пункты по порядку: каждый следующий опирается на предыдущие.

Б.1 Старт автоматизации · Б.2 Каркас проекта · Б.3 Новый тест перед мержем · Б.4 Пайплайн · Б.5 Ежемесячное здоровье набора

Б.1 Старт автоматизации

- [ ] Посчитана стоимость ручного регресса (6.1).
- [ ] Инциденты за полгода разложены по уровню, где их можно было поймать (глава 7).
- [ ] Написана стратегия на одну страницу и согласована с техлидами (глава 61).
- [ ] Выбран стек по матрице и пилоту, решение записано в ADR (главы 8, 13).
- [ ] Назначены владельцы областей и платформы (55).

Б.2 Каркас проекта

- [ ] Структура по функциональным областям, соглашения об именах и тегах (глава 15).
- [ ] Конфиг окружений с иерархией источников, секреты вне репозитория (глава 19).
- [ ] Lock-файлы, образы по версии, бот обновлений (20).
- [ ] Запуск локально за 15 минут по README (глава 21).
- [ ] CI с первого дня, артефакты при падении (главы 26, 44).

Б.3 Новый тест перед мержем

- [ ] Выбран самый низкий уровень, на котором можно проверить требование (глава 37).

- [ ] Структура AAA, одно действие, название-спецификация (глава 30).
- [ ] Данные создаются фабрикой, нет захардкоженных id (глава 18).
- [ ] Нет пауз, есть ожидания состояния (глава 38).
- [ ] Тест видел красный цвет при сломанном продукте (глава 27).
- [ ] 20 прогонов подряд в случайном порядке — зелёные (глава 38).
- [ ] Ревью автоматизатора и разработчика-владельца (глава 56).

Б.4 Пайплайн

- [ ] Стадии разделены по времени и блокировкам (глава 45).
- [ ] MR-стадия укладывается в 10 минут, выбор тестов по изменённым областям (глава 45).
- [ ] Отчёт с владельцами, трейсами, запросами и ответами (глава 47).
- [ ] Quality gates прошли месяц наблюдения (глава 48).
- [ ] Smoke в продакшене с автоматическим откатом (глава 48).

Б.5 Ежемесячное здоровье набора

- [ ] Доля flaky ниже 1%, нет тестов в карантине дольше 2 недель (глава 39).
- [ ] Время стадий в бюджете (глава 63).
- [ ] Доля времени на поддержку не выше 30% (глава 62).
- [ ] Реестр долга обновлён (глава 42).
- [ ] Чистка тестов без ценности и дублей в одной области; метрики пользы обновлены (главы 53, 58, 59).

B Глоссарий

Термины книги по-русски, рядом английский оригинал, как они встречаются в документации и вакансиях.

AAA (arrange, act, assert)	Структура теста: подготовка, действие, проверка.
ADR (architecture decision record)	Короткая запись архитектурного решения: контекст, варианты, выбор, последствия.
Can-i-deploy	Проверка в брокере контрактов: совместима ли версия сервиса со всеми потребителями и провайдерами.
Компонентный тест (component test)	Тест сервиса целиком через его API с подменёнными соседями.
Контрактный тест (contract test)	Проверка, что провайдер отвечает так, как ожидают потребители.
DDE (defect detection effectiveness)	Доля дефектов, найденных до продакшена.
DORA	Четыре метрики поставки: частота деплоя, время до продакшена, доля изменений с инцидентом, время восстановления.
E2E (end-to-end)	Сквозной тест бизнес-процесса через все участвующие системы.
Фабрика данных (test data factory)	Код, создающий тестовые сущности с разумными значениями по умолчанию.
Фейк (fake)	Рабочая упрощённая реализация зависимости: база в памяти, эмулятор API.

Фикстура (fixture)	Готовое состояние или объект, который фреймворк передаёт в тест.
Flaky-тест	Тест, который проходит и падает на одном и том же коде.
Идемпотентность (idempotency)	Свойство операции давать тот же результат при повторе.
Интеграционный тест (integration test)	Проверка кода вместе с настоящей зависимостью: базой, очередью, внешним API.
Карантин (quarantine)	Набор нестабильных тестов, которые запускаются, но не блокируют пайплайн.
Класс эквивалентности (equivalence class)	Группа входов, на которых продукт ведёт себя одинаково.
Мок (mock)	Двойник, который проверяет, что ожидаемый вызов произошёл.
Мутационное тестирование (mutation testing)	Оценка тестов через внесение маленьких изменений в код.
Page Object	Объект, скрывающий устройство страницы или компонента за методами.
Пирамида тестирования (test pyramid)	Пропорция: много быстрых низкоуровневых тестов, мало медленных верхних.
Quality gate	Условие пайплайна, без выполнения которого сборка не идёт дальше.
Регресс (regression testing)	Проверка, что новое не сломало старое.
Sanity	Узкая проверка конкретного исправления и его окружения.

SDET (software development engineer in test)	Инженер, который разрабатывает автотесты и инфраструктуру тестирования.
Шард (shard)	Часть набора тестов, выполняемая на отдельной машине CI.
Синтетический мониторинг (synthetic monitoring)	Регулярный прогон сценариев в продакшене от служебного пользователя.
Smoke	Быстрая проверка, что сборка живая и её стоит тестировать дальше.
Стаб (stub)	Двойник, возвращающий заранее заданные ответы.
Testcontainers	Библиотека для запуска настоящих зависимостей в Docker из тестов.
Тестовый двойник (test double)	Объект, подменяющий настоящую зависимость в тесте.
Три амиго (three amigos)	Встреча аналитика, разработчика и QA до разработки истории.
Unit-тест	Быстрый изолированный тест небольшой единицы поведения.
Воркер (worker)	Отдельный процесс, выполняющий часть тестов параллельно.

КОНЕЦ ГАЙДА

Теперь проверьте себя

Тест по теоретическим основам и шаблоны к главам лежат на странице гайда.

- [ ] Пройдите тест по гайду — онлайн
- [ ] Возьмите шаблоны документов — с. 218
- [ ] Пройдите чек-листы по этапам — с. 221

Цены рекламы, тарифы и доступность сервисов указаны как ориентиры и устаревают быстрее остального текста. Числовая модель сквозного продукта вымышлена, но внутренне непротиворечива.